

Two-Factor Authentication Can Harden Servers Against Offline Password Search

Xavier Boyen¹, Stanislaw Jarecki², Phillip Nazarian², Jiayu Xu³, and Tianyu Zheng⁴

¹ Queensland University of Technology, Australia xavier.boyen@qut.edu.au

² University of California, Irvine, USA {sjarecki,pnazaria}@uci.edu

³ Oregon State University, USA xujiay@oregonstate.edu

⁴ University of California, Los Angeles, USA tyzheng@ucla.edu

Abstract. We propose a novel notion of Two-Factor Authenticated Key Exchange (TFA-KE), defined in the universal composability model (UC), which extends asymmetric PAKE (aPAKE) by a 2nd authentication factor in the form of a t -bit one-time code computed by a personal device based on a clock or counter. Our notion strengthens the security of standard integration of aPAKE with short authentication codes by additionally slowing down offline brute-force password search in case of server compromise by a factor of 2^t . In other words, our TFA-KE notion uses t -bit authentication codes not only to improve on-line security of password authentication, as is the current practice, but also to strengthen password security on server corruption, whilst retaining the ability of aPAKE to avoid the common but deplorable practice of relying on “secure-channel” encryption for password protection.

We show a generic framework for implementing TFA-KE, with two efficient instantiations. Our key enabling tool is a *tight one-way function* (TOWF) with an algebraic structure that allows for its evaluation on a secret-shared input. We initiate the study of such functions, and we provide two proposals which we show to be tightly one-way in the Generic Group Model. Tightness means that a function evaluation on an input sampled from domain $\mathcal{X}$ takes $\Omega(|\mathcal{X}|)$ time to invert, which in our application implies that offline password search attacks are slowed to $\Omega(|D| \cdot 2^t)$ for passwords sampled from dictionary D .

1 Introduction

Two-Factor Authentication. Passwords are the most common method of user verification across various digital systems. However, password re-use (see e.g. [23]), which means that a password compromise at one service implies the compromise of the same user at other services, together with effective low entropy of passwords chosen by a large fraction of users (see e.g. [3]), motivates adoption of ‘two-factor’ authentication, where in addition to a password the user proves that it has access to a personal device which can compute a one-time *authentication code* expected by the authentication server. Since the user typically

reads the code displayed by her device, e.g. an authenticator app on her cell phone, and must type it as a protocol input together with her password, authentication codes are short bitstrings of fixed length t , e.g. encoded as 6-digit numerical strings ($t = 20$) or 6-digit alphanumeric strings ($t = 30$).

Current two-factor authentication implementations decouple the two components of two-factor authentication, i.e. password verification and authenticator code verification. Password verification can be implemented with any asymmetric Password Authenticated Key Exchange (aPAKE) [13,16], where the server holds a (salted) password hash rather than the password itself, while the authenticator code verification is typically implemented using a Pseudorandom Function R whose range is t -bit strings: The one-time authenticator code corresponding to time period or counter T is defined as $\text{code}_T = R_{\text{key}}(T)$ where key is a symmetric key shared by the user’s personal device and the server, and the user and the server can follow any equality-testing protocol, e.g. a (symmetric) PAKE, to verify that they hold matching code values.⁵

The main benefit of two-factor authentication implemented as above is that it provides a second layer of defense to on-line authentication in the face of password weakness: Even if a user’s password leaks, without access to the user’s device the adversary can authenticate only with probability $1/2^t$, i.e. by guessing the correct authentication code. However, this second authentication factor does not help in the all-too-common case of server compromise: Since the server stores a password hash, the adversary can recover the password given the hash using a (brute-force) Offline Dictionary Attack (ODA), which takes time $\Omega(|D|)$ where D is the presumed dictionary from which the user picked her password.

Two-Factor Authentication with 2^t -factor ODA slow-down. We show that two-factor authentication can improve *both online and offline* security of password authentication. Namely, we show Two-Factor Authenticated Key Exchange (TFA-KE) schemes where the server-held *password file* FILE allows for remote verification of the user’s password pw and the correct t -bit authentication code code_T corresponding to time/counter T , but the brute-force search for pw which is possible if FILE is revealed due to a server compromise, requires $\Omega(|D| \cdot 2^t)$ computation instead of $\Omega(|D|)$ for password-only authentication.⁶ Note that for the $t = 20$ or $t = 30$ cases mentioned above, our TFA-KE will slow

⁵ For historical reasons, in today’s practice on the internet, password authentication, with or without a second factor, merely consists of sending the user secrets across a PKI-authenticated SSL/TLS “secure channel”, which places much of the security burden upon authenticating the server *prior* to password verification—completely missing out on the *raison d’être* of PAKE and especially aPAKE. To be clear, in this paper we do not assume any pre-existing secure channel nor PKI.

⁶ Note that both bounds are optimal: In an aPAKE, password file FILE allows the adversary to offline test any password pw^* by locally running the aPAKE on client input pw^* and server input FILE, leading to ODA in time $\mathcal{O}(|D|)$. Likewise, in a TFA-KE, leaking FILE allows the adversary to offline test any $(\text{pw}^*, \text{code}^*)$ pair, by running the TFA-KE on client input $(\text{pw}^*, \text{code}^*)$ and server input (FILE, T) . Since for any T there are 2^t possible code_T values, the ODA search takes $\mathcal{O}(|D| \cdot 2^t)$ time.

ODA by considerable factors of respectively $2^{20} \approx 10^6$ or $2^{30} \approx 10^9$, and an increase of ODA time by a factor of 10^6 means that an attack which took a minute will now take 2 years.

What makes this problem non-trivial is that two-factor authentication must be secure under *lunch-time attack* on the user’s personal device, i.e. a leaked code code_T corresponding to counter T cannot help to authenticate to a server who runs on any other counter $T^* \neq T$. This effectively implies that a code generation function $\text{code}_T = R_{\text{key}}(T)$ must be a PRF.⁷ Our question is therefore: Is there a PRF function R_{key} and a password file generation algorithm $\text{Init}(\text{pw}, \text{key})$ which enable an efficient protocol Auth that authenticates client C and server S with respective inputs (pw, code) and (FILE, T) , if and only if $\text{FILE} = \text{Init}(\text{pw}, \text{key})$ and $\text{code} = R_{\text{key}}(T)$? Moreover, leaking FILE must enable recovery of pw only via a brute-force search that runs Auth on all candidate pairs $(\text{pw}^*, \text{code}^*)$. If Init and R are arbitrary one-way functions then it is not even clear that you can solve it with (two-party) secure computation [28,14], because the combined inputs of both parties, $(\text{pw}, \text{code}, \text{FILE}, T)$, do not contain key key which allows for efficient verification of this condition. On the other hand, if key is present in FILE then leakage of the latter would let an adversary compute $\text{code}_T = R_{\text{key}}(T)$ and run an ODA that tests password candidates with $\mathcal{O}(|D|)$ computation.

Contribution I: Defining Two-Factor Authenticated Key Exchange.

We propose a notion of Two-Factor Authenticated Key Exchange (TFA-KE), stated in the framework of universal composability (UC), which extends the UC asymmetric PAKE (aPAKE) model of Gentry, MacKenzie, and Ramzan [13] by the second authentication factor based on t -bit one-time authentication codes computed by the user’s personal device. Our UC TFA-KE notion is a strict upgrade of UC aPAKE, and it defaults to UC aPAKE if $t = 0$. We model the authentication code generation function $\text{code}_T = R(T)$ as a random function, which implies resilience to lunch-time attacks (in particular, an attacker who knows the correct password but does not know code_T can authenticate to either party with probability at most $1/2^t$), and we model an Offline Dictionary Attack (ODA) possible after compromise of the password file FILE as tests of explicit $(\text{pw}^*, \text{code}^*)$ candidates provided by the adversary (on some adversarially chosen counter T), which implies an $\Omega(|D| \cdot 2^t)$ computation bound on an ODA.

Contribution II: Generic TFA-KE scheme which enables efficient instantiations.

We show a generic framework for implementing TFA-KE, together with two efficient instantiations based on two proposals for *tight* one-way functions with algebraic properties (see Contribution III below). Our construction stems from the idea of Shirvanian et al. [25] which exhibited a TFA scheme with an $\Omega(|D| \cdot 2^t)$ bound on ODA after server compromise, but assuming a significantly simpler setting of Public Key Infrastructure (PKI). Password (and two-factor) authentication in the PKI setting reflects the current web authentication

⁷ Value code_{T^*} must be unpredictable given any set of pairs (T, code_T) for $T \neq T^*$, and it must be indistinguishable from a random t -bit string to achieve a 2^t factor of security increase in on-line authentication.

practice, where the client sends her password pw (and the one-time authenticator code) on an SSL channel authenticated by the server’s PKI certificate.⁸

The idea of [25] was to set $\text{key} = (k, s)$ where k is a key of an arbitrary PRF function, e.g. a block cipher, and s is a t -bit random pad, set $\text{code}_T = R_{(k,s)}(T) = s \oplus \text{PRF}_k(T)$, and let $\text{Init}(\text{pw}, (k, s))$ output $\text{FILE} = (k, H(\text{pw}, s))$ where H is a Random Oracle (RO) hash function. An authentication, which in the PKI setting is performed locally by the server on the local input FILE and message (pw, code) received on the SSL channel, parses $\text{FILE} = (k, F)$ and accepts iff $F = H(\text{pw}, \text{code} \oplus \text{PRF}_k(T))$. Note that $R_{(k,s)}$ is a PRF, and that the password file $(k, H(\text{pw}, s))$ allows for testing only explicitly provided (pw^*, s^*) candidates, implying an $\Omega(|D| \cdot 2^t)$ bound on ODA.

Our TFA-KE proposal starts from the above idea, and observes that pairs $(s_C, s_S) = (\text{code}_T, \text{PRF}_k(T))$ form a secret-sharing of pad s , and that this sharing can be implemented with group addition if set $\{0, 1\}^t$ forms an abelian group, e.g. $\mathbb{Z}_{2^t}$. We then generalize RO hash H to any *tight one-way* (and collision-resilient) function f (see below), and we use a *Conditional Key Encapsulation Mechanism* (CKEM) protocol to establish a secure key between the two parties if their respective inputs (pw, s_C) and (F, s_S) satisfy condition $F = f(\text{pw}, s_C + s_S)$. Further, to argue that $R_{(k,s)}$ acts like a random function even after server corruption we implement PRF_k using a hash function modeled as a programmable RO, and to enable extraction of an adversarial server’s inputs we add a commitment to F , a check that CKEM runs on a committed value, and a non-interactive zero-knowledge proof of knowledge of f pre-image of the committed value.

The core of this TFA-KE protocol is the CKEM for a condition, denoted $\text{SSEM}[f]$ for “secret-shared evaluation match”, that $F = f(\text{pw}, s)$ for a secret-shared s , client-held pw , and server-held committed F . This CKEM can be implemented using generic secure computation, but more efficient instantiations follow from tight one-way functions (OWF) f specifically designed to enable more efficient CKEM realizations, as we discuss below.

Contribution III: Tight OWF’s with Algebraic Properties. We use the UC notion of a Tight One-Way Function (TOWF) of Bradley, Jarecki, and Xu [4], which models a one-way function f on domain $\mathcal{X}$ (which we re-formulate to include collision-resilience) s.t. for any subset S of $\mathcal{X}$, an adversary who receives $f(x)$ for x randomly sampled in S , must bear $\Omega(|S|)$ computation cost to recover x from $f(x)$. A Random Oracle hash is an example of a TOWF, but instantiating f as a generic hash would allow only for a generic secure computation realization of CKEM for condition $\text{SSEM}[f]$. Instead, we show two examples of TOWF’s which allow for more efficient realizations of CKEM for $\text{SSEM}[f]$.

⁸ Relying on PKI in password and two-factor authentication has numerous security problems, including ‘phishing attacks’ where the client sends her password to an attacker who poses as the legitimate web service, and the fact that a spyware on the legitimate server can gain access to cleartext passwords sent to that server.

If argument x were secret-shared over f domain⁹ $\mathcal{X}$ then we could get an efficient CKEM for condition SSEM[f] if $f(x_C) \cdot f(x_S) = f(x_C + x_S)$ for some operation $\cdot$ on the f co-domain, but such a homomorphism could also speed-up the ODA. For example, function $f(x) = g^x$ where $\mathbb{G} = \langle g \rangle$ is a multiplicative group of prime order p , satisfies the above if $\mathcal{X} = \mathbb{Z}_p$, but given $F = g^x$ one can find x in $\mathcal{O}(|\mathcal{X}|^{1/2})$ time using Shank’s algorithm, which finds a collision between values $(g^{2^n})^{x_L}$ and $F \cdot g^{-x_R}$ for all n -bit x_L and x_R and $n = \lceil |p|/2 \rceil$.

We initiate the study of tight OWF’s with algebraic properties that allow for efficient secure computation of f evaluation and/or equality-checking, on secret-shared inputs. We propose two such functions:

$$f_{\text{Vand}}(x) = \prod_{i=1}^N g_i^{(x^i)} \quad f_{\text{PoNR}}(x) = \prod_{i=1}^N g_i^{(\prod_{x[j]=1} K_{i,j})}$$

where N is a ‘repetition parameter’, g_i ’s are elements of group $\mathbb{G}$ that are part of the function description, and (in f_{PoNR}) $K_{i,j}$ ’s are *public* random elements in $\mathbb{Z}_p$ that are also part of the function description. We call the first function ‘Vandermonde’ because the Vandermonde matrix comes up in its analysis, and we call the second function a ‘product of Naor-Reingold PRFs’ [22].

We show that in the Generic Group Model (GGM) both functions realize a UC notion $\mathcal{F}_{\text{OWF}}$ of a *tight* OWF, which implies that for any domain subset S , it takes $\Omega(|S|)$ time to recover x given $f(x)$. Consequently, if (pw, s) is random in $D \times \{0, 1\}^t$ then it takes $\Omega(|D| \cdot 2^t)$ time to recover pw from $f(\text{pw}, s)$. In more detail, we say that f realizes $\mathcal{F}_{\text{OWF}}$ with *rate* r if an n -time attacker can effectively test if $F = f(x^*)$ on at most $\mathcal{O}(n^r)$ candidates x^* , and we call f *tight* if it realizes $\mathcal{F}_{\text{OWF}}$ with rate 1. We show that both functions f_{Vand} and f_{PoNR} realize $\mathcal{F}_{\text{OWF}}$ with rate $r \leq 1 + d/N$ where $d = 2$ for f_{Vand} and $d \approx 3.3$ for f_{PoNR} , and with rate 1 if $N = |p|$. We note that even though these are the best bounds we could prove, either function *might* realize $\mathcal{F}_{\text{OWF}}$ with rate 1 even for $N = 2$.

Contribution IV: CKEMs for Tight OWF Candidates. Both functions f_{Vand} and f_{PoNR} enable efficient realizations of CKEM for condition SSEM.

To simplify this technical overview assume $x = \text{pw}|s$ is additively secret-shared as $x = x_C + x_S \bmod p$ between client C and server S. The Conditional KEM for the condition that S-held F satisfies $F = f_{\text{Vand}}(x)$ can proceed by computing group element $F' = f_{\text{Vand}}(x_C + x_S)$ under an encryption which is homomorphic over group $\mathbb{G}$, i.e. ElGamal (the algebraic structure of f_{Vand} allows for this 2-party computation using $\mathcal{O}(N)$ group elements), and then folding key establishment into a test for plaintext equality of two ElGamal ciphertexts, c and c' , based on the fact that c/c' raised to a random power encrypts 1 iff $F = F'$. If F -commitment is implemented with the same ElGamal encryption then it is not hard to add a verification that the server runs the CKEM on committed F .

⁹ In the explanation below we simplify notation by assuming that the whole argument x is secret-shared between C and S, whereas our actual TOWF proposals split the argument into pw and s and treat them differently since only s is secret-shared.

If x is an n -bit string which is xor-shared, i.e. $x = x_C \oplus x_S$, then the CKEM for the condition that F satisfies $F = f_{\text{PoNR}}(x)$ can be similarly done by computing $F' = f_{\text{PoNR}}(x_C \oplus x_S)$ under ElGamal encryption, but this time we use a 2-party computation similar to the Oblivious PRF protocol for the Naor-Reingold (NR) PRF given by Freedman et al. [11]: Their OPRF protocol lets e.g. S compute the NR PRF $f_K(x)$ on S -held x and C -held key K , using $n = |x|$ Oblivious Transfer (OT) instances, where S uses the i -th bit of x on the i -th OT instance. In our case x is xor-shared between C and S (and key K is public!), but the OPRF of [11] is almost the same if x is xor-shared between the two parties: S still uses the i -th bit of its input x_S as the choice bit in the i -th OT, but C flips its OT inputs according to the i -th bit of its share x_C .

Relation to Threshold (a)PAKEs. Our ODA-strengthened TFA-KE can be compared to *threshold* PAKE [19], which in the 2-server setting shares password information between two servers s.t. compromise of either one leaks no information and hence prevents an ODA attack. If device D acted as threshold server S_1 and server S as S_2 , we would have a TFA with perfect ODA security upon compromise of S . However, existing 2-server threshold PAKE solutions assume that all parties can communicate freely, while in our setting $S_1 = D$ has severe bandwidth constraints. Indeed, our TFA-KE can be considered as a novel type of threshold *asymmetric* PAKE [15], which shares a password hash between $S_1 = D$ and $S_2 = S$, but where S_1 is allowed to send only *a single t -bit message*, on a secure channel to the user/client, and where in such a setting the upper-bound on the ODA upon compromise of S_2 is $\mathcal{O}(|D| \cdot 2^t)$.

On Modularity, Composability, and the Generic Group Model. As explained above, we modularize our TFA-KE constructions into two building-blocks: a tight OWF and a corresponding CKEM. We do so for three reasons: (1) To present each building-block more clearly in isolation, because they are individually complex; (2) To highlight the generality of our approach and how it can be instantiated with different building-blocks; and (3) To minimize our dependence on the GGM. The tightness of our OWF's is proven using the GGM, but the security of our CKEM's is not, which formalizes the intuition that *our TFA-KE protocols rely on the GGM only for their security against ODA*.

Still, the combination of the GGM with the universal composability model raises some questions. In particular, we recognize that our tight OWF's may not compose with other protocols that use the same group and also model it as generic in their security arguments. This deficiency is substantially mitigated by our use of a *multi-session* ideal OWF functionality, which at least implies that it is safe for all OWF instances to use the same group. (Likewise our ideal TFA-KE functionality, and the analysis of a TFA-KE scheme, uses a multi-session setting which considers any number of TFA-KE instances, involving any number of clients, servers, and password accounts at any given server.) We also emphasize that our way of modeling OWF's has merits regardless of the implications for composability. For example, it affords us a clean and natural way to reason about tightness as a property related to simulation rate.

Extensions and Questions. Our results open up several interesting questions and directions for future work. For example, aPAKE has been extended to *strong/salted* aPAKE [16] to prevent offline attacks from being precomputed before the time of server compromise. To similarly upgrade our TFA-KE protocols we would require a salted tight OWF with an analogous property. Gu et al. [15] showed how to use a threshold Oblivious PRF (OPRF) to implement a threshold aPAKE, where the ODA attack requires compromising t ‘auxilliary servers’ in addition to the ‘target server’. For TFA-KE, a similar threshold OPRF technique should achieve a corresponding upgrade to the TFA-KE security guarantees. Another attractive TFA-KE improvement would be *forward security* of codes. For example, if the server were to ‘ratchet’ their code-generating PRF key k after each session by hashing it and deleting the old value, then the leakage of code for time T (i.e. lunch-time attack) would not help an offline attacker who compromises the server at time $T + 1$ or later.

There is also significant room for further exploration in the study of tight OWFs with algebraic properties. Our analyses leave a gap between the proven tightness of our proposed functions and the best-known explicit attacks against them. There is also the opportunity to propose and consider entirely new tight OWF candidates with the requisite properties that enable efficient 2-party secure evaluation or equality-checking on secret-shared inputs.

2 Preliminaries

Hardness Assumptions. We defer the definitions of the Discrete Logarithm (DL) and Decisional Diffie-Hellman (DDH) problems to Appendix A.

Commitment Schemes. A commitment scheme $\text{COM} : \mathcal{M} \times \mathcal{D} \rightarrow \mathcal{C}$ is an algorithm that on message $m \in \mathcal{M}$ and randomness $d \in \mathcal{D}$, outputs a commitment $C \in \mathcal{C}$. We do not define a separate algorithm for decommitment verification. To decommit a commitment C , the committer simply reveals m, d , and the verifier checks whether $\text{COM}(m; d) = C$.

A commitment scheme is required to be hiding and binding. We defer the formal definitions of these two properties to Appendix A.

When a commitment scheme is used with randomly sampled $d \leftarrow_{\$} \mathcal{D}$ (as is typical), we will sometimes write d as an output of COM rather than an input for notational convenience.

ElGamal and Pedersen Commitments. For committing to messages in $\mathcal{M} = \mathbb{G}$, we use the ElGamal commitment scheme denoted ElG.Commit , i.e. standard ElGamal encryption under a random public key. For committing to messages in $\mathcal{M} = \mathbb{Z}_p^N$, we use the (compact) Pedersen commitment scheme denoted Ped^N .

We include full definitions of ElG.Commit and Ped^N in Appendix A. Where the meaning is clear from context, we will sometimes abbreviate ElG.Commit as simply ElG .

Non-Interactive Zero-Knowledge Proofs. Figure 1 is $\mathcal{F}_{\text{NIZK}}$, the ideal functionality for a non-interactive zero-knowledge proof (NIZK) system. The functionality is parameterized by relation $\mathcal{R}$, which is a set of statement-witness pairs. Our protocols use $\mathcal{F}_{\text{NIZK}}$ for various relations, and we discuss in Appendix F that all of these NIZKs have efficient realizations from standard techniques.

- On (PROVE, ssid, $\mathbb{x}$, $\mathbb{w}$) from party P: If $\mathcal{R}(\mathbb{x}, \mathbb{w}) = 1$ then send (PROVE, P, ssid, $\mathbb{x}$) to $\mathcal{A}^*$, await response (PROOF, ssid, π) from $\mathcal{A}^*$, save record $(\mathbb{x}, \mathbb{w}, \pi)$, and send (PROOF, ssid, π) to P.
- On (VERIFY, ssid, $\mathbb{x}$, π) from party V: If there exists no record $(\mathbb{x}, \mathbb{w}, \pi)$ then send (VERIFY, V, ssid, $\mathbb{x}, \pi$) to $\mathcal{A}^*$, await response (WITNESS, ssid, $\mathbb{w}$), and save record $(\mathbb{x}, \mathbb{w}, \pi)$. In either case, send (VERIFICATION, ssid, $\mathcal{R}(\mathbb{x}, \mathbb{w})$) to V.

Fig. 1. $\mathcal{F}_{\text{NIZK}}[\mathcal{R}]$: Non-interactive zero-knowledge proof functionality

Committed Oblivious Transfer. Introduced in [8], committed oblivious transfer (COT) is a strengthened variant of standard oblivious transfer that allows parties to prove correct behavior relative to committed values. We defer a full definition and discussion of COT to Appendix A.

3 Tight One-Way Functions

Figure 2 is $\mathcal{F}_{\text{OWF}}$, an ideal functionality for a one-way function (OWF) with domain $\mathcal{X}$. This functionality allows honest parties to evaluate the OWF at will, with each evaluation tagged by a session identifier sid . Once a function output is leaked to an attacker, they are able to test guessed inputs against it and learn whether or not their guesses are correct.

Messages with $x, x^* \notin \mathcal{X}$ are ignored by the functionality.

- On (EVAL, sid, x) from any honest party P: if this is the first such message for sid, record (sid, P, x) marked **uncompromised**, and send (EVAL, sid, P) to $\mathcal{A}^*$.
- On (TEST, sid, x^*) from P where there exists record (sid, P, x) : if $x^* = x$, return “correct”; otherwise return “wrong”.
- On (LEAKVAL, sid) from $\mathcal{A}^*$: mark record (sid, P, x) **compromised** (if it exists).
- On (ADVTEST, sid, x^*) from $\mathcal{A}^*$ where there exists record (sid, P, x) marked **compromised**: if $x^* = x$, return “correct”; otherwise return “wrong”.

Fig. 2. $\mathcal{F}_{\text{OWF}}[\mathcal{X}]$: One-way function functionality, parameterized by domain $\mathcal{X}$

Crucially, all adversarial function evaluations must be performed via interaction with the ideal functionality. Therefore, $\mathcal{F}_{\text{OWF}}$ is only realizable in a non-plain

model of computation such as the Random Oracle Model (ROM) or Generic Group Model (GGM). In these models, it is assumed that there exist computations that cannot be performed “locally” without interacting with an ideal oracle. In order to invert the OWF, then, a real-world attacker must perform some such interactions (e.g. random oracle queries or generic group operations), which cause activation of the simulator **SIM**. If the OWF realizes $\mathcal{F}_{\text{OWF}}$, there must exist a **SIM** that can answer these oracle interactions while performing at most a polynomial number of queries to **ADVTEST**. The smaller this polynomial’s degree, the *tighter* the OWF, i.e. the more computational work the attacker must perform in order to invert the function.

Definition 1 (OWF Tightness). *OWF simulator **SIM** has rate r if it performs at most $\mathcal{O}(n^r)$ queries to $\mathcal{F}_{\text{OWF}}$ after n activations. Similarly, OWF f has tightness r if it realizes $\mathcal{F}_{\text{OWF}}$ with a simulator of rate r .*¹⁰

We will say a OWF is *tight* if it has tightness $r = 1$, which is optimal. In this case, there is no way of inverting the function that is faster (up to a constant factor) than a naive brute-force attack.

Comparison to [4]. Our $\mathcal{F}_{\text{OWF}}$ functionality is based on the similar functionality given in [4], but with two key additions.

First, we explicitly model a **LEAKVAL** message in the functionality, rather than leave this leakage as an implicit prerequisite of the **ADVTEST** message. Looking ahead, we will require our simulator to simulate leaked OWF outputs for the adversary, so it is helpful for the event to be modeled clearly.

Second, we add the **TEST** command to the honest parties’ interface. Without it, $\mathcal{F}_{\text{OWF}}$ could be realized trivially, e.g. by the constant function $f(\text{sid}, x) = 0$, which can be simulated without ever querying **ADVTEST**. To avert such possibilities, [4] augment their ideal functionality with a separately defined *collision-resistance* property. Our **TEST** interface achieves the same effect: if the environment finds two inputs x_1, x_2 which map to the same OWF output, then $(\text{EVAL}, \text{sid}, x_1)$ followed by $(\text{TEST}, \text{sid}, x_2)$ will return “correct” in the real world but “wrong” in the ideal world. Thus, our functionality actually models a *collision-resistant* OWF. Looking ahead, this stronger property is indeed needed by our TFA-KE application, to ensure that each online session corresponds to just a single (pw, code) guess from each party.

Example of a Tight OWF. In the ROM, our tight OWF definition has an immediate realization: $f_{\text{RO}}(\text{sid}, x) := H(\text{sid}, x)$. To see this, consider simulator **SIM** which, in the epoch before $(\text{LEAKVAL}, \text{sid})$ occurs, simply samples random $H(\text{sid}, x^*)$ values. Upon $(\text{LEAKVAL}, \text{sid})$, **SIM** queries $(\text{ADVTEST}, \text{sid}, x^*)$ for each previously sent x^* . If one of them x returns “correct”, then **SIM** sends the previously sampled $H(\text{sid}, x)$ as the leaked value; otherwise, **SIM** samples and sends a new random value y . On each subsequent $H(\text{sid}, x^*)$ call, **SIM** queries

¹⁰ Some previous works [20,21] have instead defined simulation rate as a function $r(n)$ s.t. simulator **SIM** has rate $r(n)$ if it performs at most $n \cdot r(n)$ functionality queries after n activations.

(ADVTEST, sid, x^*) and programs $H(\text{sid}, x^*) := y$ if it returns “correct”; otherwise, SIM continues to sample random values. SIM performs exactly 1 ADVTEST query per H activation; thus, f_{RO} is tight.

Unfortunately, this simple OWF is not amenable to our TFA application. Looking ahead to Section 5, we will require an online evaluation protocol that runs on a function input $x = (\text{pw}, s)$ that is secret-shared between the client and server. (In particular, s is secret-shared and pw is held by the client.) Since neither party can reveal their share, use of a black-box hash function seems to require generic MPC techniques, which would impose major efficiency costs. Our challenge, then, is to find tight OWFs with an algebraic structure that enables efficient computation on secret-shared inputs.¹¹

3.1 Proposed Function f_{Vand}

We propose the following function $f_{\text{Vand}} : \{0, 1\}^* \times \{0, 1\}^* \times \{0, 1\}^t \rightarrow \mathbb{G}$ as a tight OWF.¹² Tailored to our TFA-KE use case, we treat function inputs as tuples $(\text{sid}, \text{pw}, s)$ where sid is the session identifier, pw is a password, and s is a t -length bitstring. f_{Vand} is parameterized by group $\mathbb{G}$ of prime order p , a “repetition number” $N \in \mathbb{Z}^+$, and hash functions $H_{\mathbb{G}} : \{0, 1\}^* \rightarrow \mathbb{G}$ and $H_p : \{0, 1\}^* \rightarrow \{0, 1\}^{\lfloor \log p \rfloor - t - 1}$.

Definition 2 (f_{Vand}).

$$f_{\text{Vand}}(\text{sid}, \text{pw}, s) := \prod_{i \in [N]} H_{\mathbb{G}}(\text{sid}, i)^{k_i(\text{pw}, s)}$$

$$\text{where } k_i(\text{pw}, s) := (H_p(\text{pw})|0|s)^i$$

Here, $(H_p(\text{pw})|0|s)$ is the $\mathbb{Z}_p$ element represented by the concatenation of bitstrings $H_p(\text{pw})$, 0, and s .

We will find it useful in our analysis to decompose f_{Vand} into a pre-processing step f_{pre} and a main step f_{Vand}^* .

Definition 3 (f_{pre}).

$$\begin{aligned} f_{\text{pre}}^{\mathcal{HP}} : \{0, 1\}^* \times \{0, 1\}^* \times \{0, 1\}^t &\rightarrow \{0, 1\}^* \times \mathcal{HP} \times \{0, 1\}^t \\ f_{\text{pre}}^{\mathcal{HP}}(\text{sid}, \text{pw}, s) &:= (\text{sid}, H_p(\text{pw}), s) \text{ where } H_p : \{0, 1\}^* \rightarrow \mathcal{HP} \end{aligned}$$

Defining $f_{\text{Vand}}^*(\text{sid}, \text{hpw}, s) := \prod_{i \in [N]} H_{\mathbb{G}}(\text{sid}, i)^{(\text{hpw}|0|s)^i}$, we have $f_{\text{Vand}} = f_{\text{Vand}}^* \circ f_{\text{pre}}^{\mathcal{HP}}$ for $\mathcal{HP} = \{0, 1\}^{\lfloor \log p \rfloor - t - 1}$.

¹¹ Notably, the tight OWF proposed by [4] does not have this structure. That work seeks a tight OWF with an algebraic structure that enables efficient computation between two parties, one holding a high-entropy salt and the other holding a low-entropy function input (e.g. a password). Crucially, the Boneh-Boyen OWF they propose is only tight when function inputs are hashed, and it is therefore unsuited to our TFA-KE application.

¹² The name of f_{Vand} refers to the Vandermonde matrix that arises in the analysis of its tightness.

Whenever a new $\mathbb{G}$ element is sent to the environment, the simulator picks a unique random string to represent it (except when this behavior is overridden by the code below). The simulator maintains internal descriptions for these $\mathbb{G}$ elements in the form of two tables of maps α and β . $\alpha[\gamma]$ is a map $(\text{sid}, i) \mapsto \mathbb{Z}_p$ and $\beta[\gamma]$ is a map $\text{sid} \mapsto \mathbb{Z}_p$ such that $\gamma \in \mathbb{G}$ is described as:

$$\gamma = \prod_{\text{sid}, i} H_{\mathbb{G}}(\text{sid}, i)^{\alpha[\gamma](\text{sid}, i)} \cdot \prod_{\text{sid}} (y_{\text{sid}})^{\beta[\gamma](\text{sid})} \quad (1)$$

The simulator also maintains a table $X : (\text{sid}, \text{pw}, s) \mapsto \{0, 1\}$ of possible OWF inversions that are known to be either correct or wrong.

- On fresh query $H_{\mathbb{G}}(\text{sid}, i)$, define new $\gamma \in \mathbb{G}$ and return it. $\alpha[\gamma](\text{sid}, i) := 1$.
- Sample $H_p(\text{pw})$ uniformly at random for every $\text{pw} \in \{0, 1\}^*$.
- On $(\text{LEAKVAL}, \text{sid})$, define new $y_{\text{sid}} \in \mathbb{G}$ and send it to $\mathcal{A}^*$. $\beta[y_{\text{sid}}](\text{sid}) := 1$.
- On group operation $\gamma_1 \cdot \gamma_2$, define new $\mathbb{G}$ element γ_3 and return it. $\alpha[\gamma_3] := \alpha[\gamma_1] + \alpha[\gamma_2]$ and $\beta[\gamma_3] := \beta[\gamma_1] + \beta[\gamma_2]$.

Whenever a new $\mathbb{G}$ element γ is defined (by $H_{\mathbb{G}}$ query, LEAKVAL event, or group operation), the simulator checks whether its internal description is identical to any previous γ' ; if so, the same unique string is used as its representation. If not, the simulator proceeds to check whether the internal description of γ is *conditionally* equivalent to any previous γ' , conditioned on possible OWF inversions:

$$\begin{aligned} & \forall_{\text{sid} \text{ s.t. } \beta[\gamma](\text{sid}) = \beta[\gamma'](\text{sid})} \\ & \quad \forall_{i \in [N]} \quad \alpha[\gamma](\text{sid}, i) = \alpha[\gamma'](\text{sid}, i) \\ & \quad \wedge \\ & \quad \forall_{\text{sid} \text{ s.t. } \beta[\gamma](\text{sid}) \neq \beta[\gamma'](\text{sid})} \quad \exists_{\text{pw}, s} \\ & \quad \quad \forall_{i \in [N]} \quad \alpha[\gamma](\text{sid}, i) - \alpha[\gamma'](\text{sid}, i) = k_i(\text{pw}, s) \cdot (\beta[\gamma](\text{sid}) - \beta[\gamma'](\text{sid})) \end{aligned} \quad (2)$$

If Condition 2 is true with some $(\text{sid}_1, \text{pw}_1, s_1), (\text{sid}_2, \text{pw}_2, s_2), \dots, (\text{sid}_n, \text{pw}_n, s_n)$ satisfying the second clause, then consider each of those as a possible OWF inversion. If all these inversions are correct, then $\gamma = \gamma'$; otherwise, $\gamma \neq \gamma'$. If there is insufficient information in X to determine whether $\gamma = \gamma'$, perform the relevant $(\text{ADVTEST}, \text{sid}, (\text{pw}, s))$ queries until receiving one “wrong” or receiving all “correct”. Record responses in X to avoid redundant queries in the future.

Fig. 3. SIM_{Vand} : Simulator for f_{Vand}

Theorem 1. *Function f_{Vand} realizes $\mathcal{F}_{\text{OWF}}[\{0,1\}^* \times \{0,1\}^t]$, assuming hash function $H_{\mathbb{G}}$ is modeled as a random oracle, hash function H_p is modeled as a non-programmable random oracle, $\mathbb{G}$ is modeled as a generic group, at most $q_H \leq 2^{(\lceil \log p \rceil - t - 1 - \kappa)/2}$ queries are made to H_p , at most $q_{\mathbb{G}}$ $\mathbb{G}$ elements are seen by the environment, and $q_H q_{\mathbb{G}}^2 \leq 2^{\lceil \log p \rceil - t - 1 - \kappa}$.*

Proof. To prove the security of f_{Vand} we provide in Figure 3 the simulator SIM_{Vand} , which operates in the GGM and ROM. In particular, SIM_{Vand} must appropriately program the responses to every $H_{\mathbb{G}}$ evaluation, H_p evaluation, and $\mathbb{G}$ group operation while making a minimal number of ADVTEST queries to $\mathcal{F}_{\text{OWF}}$. For $H_{\mathbb{G}}$ and H_p , SIM_{Vand} simply samples responses at random.

For group operations, the simulation strategy is more complex. SIM_{Vand} maintains an internal description of every $\mathbb{G}$ element the environment has seen and constantly checks for pairs of these group elements that represent tests for possible OWF inversions (Condition 2). If two group elements do represent a test, then SIM_{Vand} performs the relevant ADVTEST queries to evaluate it. If the test succeeds, then the simulator correctly programs the two associated $\mathbb{G}$ elements to be identical. Notice that a “test pair” for sid is only possible after the $(\text{LEAKVAL}, \text{sid})$ event; before that event, $\beta[\gamma](\text{sid}) = 0$ for all γ .

Thus, SIM_{Vand} perfectly emulates the real world, except in the case of three “bad events” which we argue to be negligibly probable.

1. Discrete Logarithm break. SIM_{Vand} does not bother to define the discrete logarithms between $\mathbb{G}$ elements output by $H_{\mathbb{G}}$. The only relationship between $\mathbb{G}$ elements that is respected by SIM_{Vand} is that of the OWF f_{Vand} , which is checked in Condition 2. In the real world, it is possible for the environment to compute a discrete logarithm relation between $H_{\mathbb{G}}$ responses and thereby find alternative descriptions for group elements. In the ideal world, SIM_{Vand} will erroneously program these alternative descriptions as if they describe unique group elements.

The classical GGM result of Shoup [26] is easily extended to show that this event occurs with probability $< q_{\mathbb{G}}^2/p$. Under the assumption that $q_{\mathbb{G}}^2 < 2^{\log p - \kappa}$, this probability is $< 2^{-\kappa}$.

2. Hash Premonition. When checking Condition 2, SIM_{Vand} only considers pws that have been queried to H_p . It is possible for a later H_p query to sample a $\text{hpw} \leftarrow_{\$} \mathcal{HP} = \{0,1\}^{\lceil \log p \rceil - t - 1}$ that retroactively satisfies Condition 2 for a pair γ, γ' of $\mathbb{G}$ elements that SIM_{Vand} already judged to be unequal.

For every $\gamma, \gamma' \in \mathbb{G}$ and every sid s.t. $\beta[\gamma](\text{sid}) \neq \beta[\gamma'](\text{sid})$, there is a single $\text{hpw} \in \mathcal{HP}$ that satisfies Condition 2. Thus, the probability that a new H_p query retroactively satisfies Condition 1 for any γ, γ' is $\leq q_{\mathbb{G}}^2/|\mathcal{HP}|$. The probability of the event across q_H H_p queries is $\leq q_H q_{\mathbb{G}}^2/|\mathcal{HP}|$. Under the assumption that $q_H q_{\mathbb{G}}^2 \leq 2^{\lceil \log p \rceil - t - 1 - \kappa}$, this probability is $\leq 2^{-\kappa}$.

3. OWF Collision. As mentioned above, $\mathcal{F}_{\text{OWF}}$ requires that a OWF be collision-resistant: If the environment can find $x_1 \neq x_2$ and sid s.t. $f(\text{sid}, x_1) = f(\text{sid}, x_2)$, then it is easy to distinguish the real and ideal worlds via the TEST command. This collision-resistance is a property of the OWF itself, i.e. it is independent of the simulator.

Lemma 1. *Function f_{Vand}^* is collision-resistant, assuming $H_{\mathbb{G}}$ is modeled as a random oracle and DL is hard in $\mathbb{G}$.*

To prove Lemma 1, suppose there exists algorithm $\mathcal{A}$ that outputs $(\text{hpw}, s) \neq (\text{hpw}', s')$ and sid s.t. $f_{\text{Vand}}^*(\text{sid}, \text{hpw}, s) = f_{\text{Vand}}^*(\text{sid}, \text{hpw}', s')$. The two evaluations use the same bases, but $\forall_{i \in [N]} (\text{hpw}|0|s)^i \neq (\text{hpw}'|0|s')^i$. Then there is a straight-forward and well-known “DL relation” reduction (e.g. [18]) from $\mathcal{A}$ that solves DL in $\mathbb{G}$.

By Lemma 1, in the case that no DL break (i.e. bad event 1) occurs, $f_{\text{Vand}} = f_{\text{Vand}}^* \circ f_{\text{pre}}$ collision is only possible in the case of f_{pre} collision. Under the assumption that $q_H \leq 2^{(\lfloor \log p \rfloor - t - 1 - \kappa)/2}$, the probability of such a collision is $< 2^{-\kappa}$ by a simple birthday bound.

It is now proven that f_{Vand} realizes $\mathcal{F}_{\text{OWF}}$, but its tightness still remains to be analyzed.

Theorem 2 (f_{Vand} Tightness). *Assume that at most $q_H \leq 2^{(\lfloor \log p \rfloor - t - 1 - \kappa)/2}$ queries are made to H_p and at most q_G $\mathbb{G}$ elements are seen by the environment.*

- If $N \geq \log q_G$, then SIM_{Vand} has rate $r = 1$, i.e. f_{Vand} is tight.
- If $N < \log q_G$, then SIM_{Vand} has rate $r = 1 + \frac{1}{\lfloor N/2 \rfloor}$.

Proof. To begin, split the ADVTEST queries made by SIM_{Vand} into those which return “correct” and those which return “wrong”. Once a “correct” value is found, no other queries are ever needed for that sid . Therefore, the total number of “correct” ADVTEST queries is upper-bounded by the number of $(\text{LEAKVAL}, \text{sid})$ activations of SIM_{Vand} , i.e. the rate of such queries is 1.

To upper-bound the number of “wrong” queries, it is helpful to think of an *attack graph* $\mathcal{G}$. This undirected graph has at most q_G vertices, one for each $\mathbb{G}$ element seen by the environment. It has an edge between every pair of $\mathbb{G}$ elements that triggered a “wrong” test. Every edge is associated with the $(\text{sid}, \text{pw}, s)$ that it tests for. The simulator only makes at most a single “wrong” query for every pair; the attack graph is therefore simple and contains at most $\mathcal{O}(n^2)$ edges. Consequently, SIM_{Vand} has rate $r \leq 2$ (regardless of parameter N). To improve this bound, we must bound the edge density of $\mathcal{G}$.

Next, we define $K((\text{pw}_1, s_1), \dots, (\text{pw}_c, s_c))$ to be an $N \times c$ matrix that is a function of $(\text{pw}_1, s_1), \dots, (\text{pw}_c, s_c) \in \{0, 1\}^* \times \{0, 1\}^t$ (for any $c \in \mathbb{Z}^+$):

$$K((\text{pw}_1, s_1), \dots, (\text{pw}_c, s_c)) := \begin{bmatrix} k_1(\text{pw}_1, s_1) & k_1(\text{pw}_2, s_2) & \dots & k_1(\text{pw}_c, s_c) \\ k_2(\text{pw}_1, s_1) & k_2(\text{pw}_2, s_2) & \dots & k_2(\text{pw}_c, s_c) \\ \vdots & & \ddots & \vdots \\ k_N(\text{pw}_1, s_1) & k_N(\text{pw}_2, s_2) & \dots & k_N(\text{pw}_c, s_c) \end{bmatrix}$$

Using this notation, we take a step toward proving Theorem 2 with the following lemma.

Lemma 2. *For every integer $c \geq 3$, if the ADVTEST queries of SIM_{Vand} imply an attack graph $\mathcal{G}$ containing a cycle of length $\leq c$, then there exist distinct $(\text{pw}_1, s_1), \dots, (\text{pw}_c, s_c) \in \{0, 1\}^* \times \{0, 1\}^t$ such that $\text{rank}(K((\text{pw}_1, s_1), \dots, (\text{pw}_c, s_c))) < c$.*

Suppose that $\mathcal{G}$ contains a cycle $C = (\gamma_1, \gamma_2, \dots, \gamma_{\tilde{c}})$ of length $\tilde{c} \leq c$. Label it such that $(\gamma_1, \gamma_{\tilde{c}})$ is the first of C 's edges added to $\mathcal{G}$ (i.e. the earliest of the corresponding ADVTEST queries made by SIM_{Vand}), and consider the sid tested by that $(\gamma_1, \gamma_{\tilde{c}})$ edge. For every edge (γ, γ') in C , either $\forall_{i \in [N]} \alpha[\gamma](\text{sid}, i) = \alpha[\gamma'](\text{sid}, i)$ or $\exists_{\text{pw}, s} \forall_{i \in [N]} \alpha[\gamma](\text{sid}, i) - \alpha[\gamma'](\text{sid}, i) = k_i(\text{pw}, s) \cdot (\beta[\gamma](\text{sid}) - \beta[\gamma'](\text{sid})) \neq 0$ (by Condition 2). Define $\hat{C} = ((\hat{\gamma}_1, \hat{\gamma}'_1, \text{pw}_1, s_1), (\hat{\gamma}_2, \hat{\gamma}'_2, \text{pw}_2, s_2), \dots, (\hat{\gamma}_{\hat{c}}, \hat{\gamma}'_{\hat{c}}, \text{pw}_{\hat{c}}, s_{\hat{c}}))$ to be the subset of C 's edges which meet the latter condition (and their associated pw, s values). $(\hat{\gamma}_1, \hat{\gamma}'_1) = (\gamma_1, \gamma_{\tilde{c}})$, so $1 \leq \hat{c} \leq \tilde{c} \leq c$.

For every $i \in [N]$, the sum of the difference of $\alpha[\cdot](\text{sid}, i)$ across every edge in C must equal 0 (because C is a cycle). By definition, this difference is 0 for the edges not in $\hat{C}$. Therefore:

$$\begin{aligned} \forall_{i \in [N]} \quad 0 &= \sum_{(\hat{\gamma}, \hat{\gamma}', \text{pw}, s) \in \hat{C}} \alpha[\hat{\gamma}](\text{sid}, i) - \alpha[\hat{\gamma}'](\text{sid}, i) \\ &= \sum_{(\hat{\gamma}, \hat{\gamma}', \text{pw}, s) \in \hat{C}} k_i(\text{pw}, s) \cdot (\beta[\hat{\gamma}](\text{sid}) - \beta[\hat{\gamma}'](\text{sid})) \end{aligned}$$

Written in matrix form:

$$\begin{bmatrix} 0 \\ 0 \\ \vdots \\ 0 \end{bmatrix} = \begin{bmatrix} k_1(\text{pw}_1, s_1) & \dots & k_1(\text{pw}_{\hat{c}}, s_{\hat{c}}) \\ k_2(\text{pw}_1, s_1) & \dots & k_2(\text{pw}_{\hat{c}}, s_{\hat{c}}) \\ \vdots & \ddots & \vdots \\ k_N(\text{pw}_1, s_1) & \dots & k_N(\text{pw}_{\hat{c}}, s_{\hat{c}}) \end{bmatrix} \cdot \begin{bmatrix} \beta[\hat{\gamma}_1](\text{sid}) - \beta[\hat{\gamma}'_1](\text{sid}) \\ \beta[\hat{\gamma}_2](\text{sid}) - \beta[\hat{\gamma}'_2](\text{sid}) \\ \vdots \\ \beta[\hat{\gamma}_{\hat{c}}](\text{sid}) - \beta[\hat{\gamma}'_{\hat{c}}](\text{sid}) \end{bmatrix} \quad (3)$$

The $(\text{pw}_1, s_1), \dots, (\text{pw}_{\hat{c}}, s_{\hat{c}})$ values are not necessarily distinct, which means that the matrix above may contain duplicate columns. However, it is guaranteed that (pw_1, s_1) is distinct from all $(\text{pw}_2, s_2), \dots, (\text{pw}_{\hat{c}}, s_{\hat{c}})$. Recall that we labeled $(\gamma_1, \gamma_{\tilde{c}}) = (\hat{\gamma}_1, \hat{\gamma}'_1)$ such that it corresponds to the first ADVTEST query of cycle C and that all the queries recorded in $\mathcal{G}$ are “wrong”. Therefore, any future vertex pair whose Condition 2 implies a test for $(\text{sid}, \text{pw}_1, s_1)$ would not trigger an ADVTEST query (i.e. would not add an edge to $\mathcal{G}$) because the recorded information $X[\text{sid}, \text{pw}_1, s_1] = 0$ is sufficient for SIM_{Vand} to conclude that the $\mathbb{G}$ elements of the pair are not equal.

Define d to be the number of distinct (pw, s) in $(\text{pw}_1, s_1), \dots, (\text{pw}_{\hat{c}}, s_{\hat{c}})$. Consider the condensed set $(\text{pw}_{\rho_1}, s_{\rho_1}), \dots, (\text{pw}_{\rho_d}, s_{\rho_d})$, where $\rho_1, \dots, \rho_d$ are the first indices of each distinct (pw, s) in $\hat{C}$, and $P_1, \dots, P_d$ are the sets of all indices of each of them. $\rho_1 = 1$, $P_1 = \{1\}$, and $1 \leq d \leq \hat{c} \leq \tilde{c} \leq c$. Using this notation, we can now rewrite Equation 3 without duplicate columns:

$$\begin{bmatrix} 0 \\ 0 \\ \vdots \\ 0 \end{bmatrix} = \begin{bmatrix} k_1(\text{pw}_{\rho_1}, s_{\rho_1}) & \dots & k_1(\text{pw}_{\rho_d}, s_{\rho_d}) \\ k_2(\text{pw}_{\rho_1}, s_{\rho_1}) & \dots & k_2(\text{pw}_{\rho_d}, s_{\rho_d}) \\ \vdots & \ddots & \vdots \\ k_N(\text{pw}_{\rho_1}, s_{\rho_1}) & \dots & k_N(\text{pw}_{\rho_d}, s_{\rho_d}) \end{bmatrix} \cdot \begin{bmatrix} \beta[\hat{\gamma}_1](\text{sid}) - \beta[\hat{\gamma}'_1](\text{sid}) \\ \sum_{\rho \in P_2} \beta[\hat{\gamma}_{\rho}](\text{sid}) - \beta[\hat{\gamma}'_{\rho}](\text{sid}) \\ \vdots \\ \sum_{\rho \in P_d} \beta[\hat{\gamma}_{\rho}](\text{sid}) - \beta[\hat{\gamma}'_{\rho}](\text{sid}) \end{bmatrix} \quad (4)$$

$$\mathbf{0} = K((\text{pw}_{\rho_1}, s_{\rho_1}), \dots, (\text{pw}_{\rho_d}, s_{\rho_d})) \cdot \mathbf{B}$$

Crucially, the topmost element of vector $\mathbf{B}$ must be non-zero (by the conditions given above), and therefore $\mathbf{B} = \mathbf{0}$ is not an acceptable solution to this system of equations. In order for the system to have a non-zero solution, the nullity of matrix K must be greater than 0, i.e. the rank of K must be less than d .

We conclude the proof of Lemma 2 by observing that if the $N \times d$ matrix $K((\mathbf{pw}_1, s_1), \dots, (\mathbf{pw}_d, s_d))$ has rank $< d$, then an $N \times c$ matrix $K((\mathbf{pw}_1, s_1), \dots, (\mathbf{pw}_d, s_d), (\mathbf{pw}_{d+1}, s_{d+1}), \dots, (\mathbf{pw}_c, s_c))$ with rank $< c$ can be easily created by adding columns for any additional (distinct) $(\mathbf{pw}_{d+1}, s_{d+1}), \dots, (\mathbf{pw}_c, s_c) \in \{0, 1\}^* \times \{0, 1\}^t$.

For $c > N$, all $N \times c$ matrices have rank $< c$ (in particular, they have rank $\leq N$). Consider, however, the case of $K((\mathbf{pw}_1, s_1), \dots, (\mathbf{pw}_c, s_c))$ for $c \leq N$:

$$\begin{bmatrix} k_1(\mathbf{pw}_1, s_1) & \dots & k_1(\mathbf{pw}_c, s_c) \\ k_2(\mathbf{pw}_1, s_1) & \dots & k_2(\mathbf{pw}_c, s_c) \\ \vdots & \ddots & \vdots \\ k_N(\mathbf{pw}_1, s_1) & \dots & k_N(\mathbf{pw}_c, s_c) \end{bmatrix} = \begin{bmatrix} (H_p(\mathbf{pw}_1)|0|s_1)^1 & \dots & (H_p(\mathbf{pw}_c)|0|s_1)^1 \\ (H_p(\mathbf{pw}_1)|0|s_1)^2 & \dots & (H_p(\mathbf{pw}_c)|0|s_1)^2 \\ \vdots & \ddots & \vdots \\ (H_p(\mathbf{pw}_1)|0|s_1)^N & \dots & (H_p(\mathbf{pw}_c)|0|s_1)^N \end{bmatrix}$$

This is a Vandermonde matrix, and it has rank c iff all $(H_p(\mathbf{pw}_1)|0|s_1), \dots, (H_p(\mathbf{pw}_c)|0|s_1)$ are distinct. Since (as argued in the proof of Theorem 1) an H_p collision is negligibly likely, it is sufficient that $(\mathbf{pw}_1, s_1), \dots, (\mathbf{pw}_c, s_c)$ are distinct. We can therefore infer from Lemma 2 that the attack graph $\mathcal{G}$ of SIM_{Vand} contains no cycles of length $\leq N$.

To complete the proof of Theorem 2, we refer to two theorems from extremal graph theory (cf. [12] for an excellent survey) that relate the lack of small cycles in $\mathcal{G}$ to its edge sparsity.

Theorem 3 ([1], [12, Theorem 4.1]). *For any integer $\lambda \geq 2$, if undirected graph $\mathcal{G}$ contains q vertices and no cycles of length $\leq \lambda$, then the number of edges in $\mathcal{G}$ is less than $\frac{1}{2}q^{1+1/\lfloor \lambda/2 \rfloor} + \frac{1}{2}q$.*

Theorem 4 ([9], [12, Theorem 4.39]). *For any integer $\delta \geq 2$, if undirected graph $\mathcal{G}$ contains q vertices and no cycles of length $< 2 \log q / \log \delta$, then the number of edges in $\mathcal{G}$ is at most $\delta \cdot q$.*

Theorem 3 implies that the number of edges in $\mathcal{G}$ is $\mathcal{O}(q_{\mathbb{G}}^{1+1/\lfloor N/2 \rfloor})$, i.e. SIM_{Vand} has rate $r = 1 + \frac{1}{\lfloor N/2 \rfloor}$. Setting $\delta = 4$, Theorem 4 implies that if $N \geq \log q_{\mathbb{G}}$ then the number of edges in $\mathcal{G}$ is $\mathcal{O}(q_{\mathbb{G}})$, i.e. SIM_{Vand} has rate $r = 1$ and f_{Vand} is tight. Theorem 2 is proven.

3.2 Proposed Function f_{PoNR}

Our second proposed tight OWF is $f_{\text{PoNR}} : \{0, 1\}^* \times \{0, 1\}^* \times \{0, 1\}^t \rightarrow \mathbb{G}$. Once again we treat function inputs as tuples $(\text{sid}, \mathbf{pw}, s)$. f_{PoNR} is parameterized by group $\mathbb{G}$ of prime order p , repetition number $N \in \mathbb{Z}^+$, and hash functions $H_{\mathbb{G}} : \{0, 1\}^* \rightarrow \mathbb{G}$, $H_{\mathbb{G}} : \{0, 1\}^* \rightarrow \mathbb{G}$, $H_p : \{0, 1\}^* \rightarrow \mathbb{Z}_p^*$, and $H_x : \{0, 1\}^* \rightarrow \mathbb{Z}_p^*$.

Definition 4 (f_{PoNR}).

$$f_{\text{PoNR}}(\text{sid}, \text{pw}, s) := \prod_{i \in [N]} H_{\mathbb{G}}(\text{sid}, i)^{k_i(\text{pw}, s)} \cdot \hat{H}_{\mathbb{G}}(\text{sid}, i)^{H_p(i, \text{pw})}$$

where $k_i(\text{pw}, s) := H_p(i, \text{pw}) \cdot \prod_{\substack{j \in [t] \\ s[j]=1}} H_x(i, j)$

This function follows the structure of f_{Vand} (Definition 2), differing only in the definition of $k_i(\text{pw}, s)$ and the addition of the $\hat{H}_{\mathbb{G}}$ bases (which are needed to ensure collision-resistance and are not involved in the proof of tightness).¹³

To aid our analysis, we decompose $f_{\text{PoNR}} = f_{\text{PoNR}}^* \circ f_{\text{pre}}^{\mathcal{HP}}$ (Definition 3), where $\mathcal{HP} = (\mathbb{Z}_p^*)^N$ and $f_{\text{PoNR}}^*(\text{sid}, \text{hpw}, s) := \prod_{i \in [N]} H_{\mathbb{G}}(\text{sid}, i)^{\text{hpw}_i \cdot \prod_{j \in [t], s[j]=1} H_x(i, j)}$. $\hat{H}_{\mathbb{G}}(\text{sid}, i)^{\text{hpw}_i}$.

Lemma 3. *Function f_{PoNR}^* is collision-resistant, assuming $H_{\mathbb{G}}$, $\hat{H}_{\mathbb{G}}$, and H_x are modeled as random oracles and DL is hard in $\mathbb{G}$.*

Theorem 5. *Function f_{PoNR} realizes $\mathcal{F}_{\text{OWF}}[\{0, 1\}^* \times \{0, 1\}^t]$, assuming hash functions $H_{\mathbb{G}}$, $\hat{H}_{\mathbb{G}}$, and H_x are modeled as random oracles, hash function H_p is modeled as a non-programmable random oracle, $\mathbb{G}$ is modeled as a generic group, at most $q_H \leq 2^{(N \log p - \kappa)/2}$ queries are made to H_p , at most $q_G \leq 2^{(\log p - \kappa)/2}$ $\mathbb{G}$ elements are seen by the environment, $q_H q_G^2 \leq 2^{N \log p - \kappa}$, and $N \geq \frac{\kappa + 2t}{\log(p-1)}$.*

Theorem 6 (f_{PoNR} Tightness). *Assume that at most $q_H \leq 2^{(N \log p - \kappa)/2}$ queries are made to H_p , at most q_G $\mathbb{G}$ elements are seen by the environment, and $N \geq \frac{\kappa + 2t}{\log(p-1)}$. Take c to be the greatest integer satisfying $c - 1 + \frac{\kappa + c(t + \log q_H)}{\log(p-1) - \log(2c(t+1))} \leq N \leq 2c$.*

- If $c \geq \log q_G$, then SIM_{PoNR} has rate $r = 1$, i.e. f_{PoNR} is tight.
- If $c < \log q_G$, then SIM_{PoNR} has rate $r = 1 + \frac{1}{\lceil c/2 \rceil}$.

We defer the proofs of Lemma 3 and Theorems 5 and 6 to Appendix B. They follow the same approaches as the correspondings proofs for f_{Vand} , but they are complicated by probabilistic arguments over the randomness of H_p and H_x .

Note that for all parameters we consider reasonable (e.g. $\kappa = 128$, $t = 30$, and $\log p = 256$) any $N \geq 1$ satisfies the condition $N \geq \frac{\kappa + 2t}{\log(p-1)}$. For a taste of Theorem 6's inequality between N and c , if $\log q_H = 128$ and other parameters are as above, then this condition is approximated by $1.66c \leq N \leq 2c$.

3.3 Committed One-Way Functions

Looking ahead to our TFA-KE application, our tight OWFs will be evaluated on some (pw_{Δ}, s) at user registration time to create the server's secret

¹³ When $N = 1$, f_{PoNR} resembles the Naor-Reingold PRF [22]. We therefore name our function “product of Naor-Reingold”, or PoNR.

file $F := f(\text{sid}, \text{pw}_\Delta, s)$. The server will also store an ElGamal commitment to F , its associated decommitment, and a NIZK that the commitment is valid for some (pw_Δ, s) . Then, in each TFA-KE session, the server will send the client that commitment, that proof, and an additional proof that they are executing correctly relative to the F committed therein.

As analyzed thus far, we have no assurance that f_{Vand} and f_{PoNR} remain tight OWFs against an environment that sees an ElGamal commitment to F before the LEAKVAL event and later sees the associated decommitment upon LEAKVAL. As described in Figure 3, SIM_{Vand} does not simulate these values. We therefore define compiler $\text{CommittedOwf}[f, \text{COM}]$ that, given function f and commitment scheme COM , compiles a construction matching the syntax of $\mathcal{F}_{\text{OWF}}$.

Definition 5. $\text{CommittedOwf}[f, \text{COM}]$ is the following protocol.

- On $(\text{EVAL}, \text{sid}, x)$, $\mathcal{P}$ does: compute $F := f(\text{sid}, x)$, $(C_F, d_F) \leftarrow \text{COM}(F)$, store $(\text{sid}, F, C_F, d_F)$, and send C_F to $\mathcal{A}^*$.
- On $(\text{TEST}, \text{sid}, x^*)$, $\mathcal{P}$ does: retrieve $(\text{sid}, F, C_F, d_F)$ and compute $F^* := f(\text{sid}, x^*)$; if $F^* = F$, return “correct”; otherwise, return “wrong”.

Lemma 4. The claims of Theorems 1 and 2 are true for $\text{CommittedOwf}[f_{\text{Vand}}, \text{ElG}]$ in addition to f_{Vand} . The claims of Theorems 5 and 6 are true for $\text{CommittedOwf}[f_{\text{PoNR}}, \text{ElG}]$ in addition to f_{PoNR} .

Due to space constraints, we defer the proof of Lemma 4 to Appendix C. In brief, this proof uses the hiding property of ElGamal commitments to show that, in the GGM, all algorithms have negligible probability of using the $\mathbb{G}$ elements of these commitments in a way that disrupts the simulation strategy of SIM_{Vand} .

4 The TFA-KE Functionality

We define TFA-KE in the setting of Universal Composability (UC) via an ideal functionality $\mathcal{F}_{\text{TFA-KE}}$ shown in Figure 4. A real-world TFA-KE scheme is a tuple of efficient algorithms $(\text{Init}, \text{AuthCodeGen}, \text{Auth}_C, \text{Auth}_S)$, which correspond to, respectively, password file initialization by server $\mathcal{S}$, generation of a one-time authentication code by device $\mathcal{D}$, and client $\mathcal{C}$ and server $\mathcal{S}$ on-line authentication. In the UC formalization, these algorithms implement commands STOREFILE , READCODE , CSESSION , and SSESSION , which the environment can issue to parties $\mathcal{S}$, $\mathcal{D}$, $\mathcal{C}$, and $\mathcal{S}$, respectively.¹⁴

The starting point for $\mathcal{F}_{\text{TFA-KE}}$ is the well-established $\mathcal{F}_{\text{aPAKE}}$ UC functionality of [13], which we now briefly recall. $\mathcal{F}_{\text{aPAKE}}$ extends UC PAKE to the asymmetric setting, where the server stores a password file (e.g. a password hash) rather than a plaintext password. Compromise of this file is modeled by COMPROMISEFILE . That interface is observable by the environment, which ensures that it can only be used by the ideal adversary at the time of server compromise by the real-world adversary. An offline attack against a compromised file is modeled by

¹⁴ We further discuss TFA-KE syntax and intuitive security properties in Appendix D.

The functionality internally keeps a random function $R^{sid} : \mathbb{N} \rightarrow \{0, 1\}^\ell$ for every sid .

File Storage

- On $(STOREFILE, sid, pw_\Delta)$ from S , if this is the first $STOREFILE$ message, record $\langle FILE, sid, S, pw_\Delta \rangle$, mark it **uncompromised**, and send $(STOREFILE, sid, S)$ to $\mathcal{A}^*$.

Server Leakage

- On $(COMPROMISEFILE, sid)$ from $\mathcal{A}^*$, if there is no record $\langle FILE, sid, S, pw_\Delta \rangle$, return “no password file” to $\mathcal{A}^*$. Otherwise, if the record is marked **uncompromised**, mark it **compromised**; regardless,
 - If there is a record $\langle OFFLINE, pw_\Delta, T^*, R^{sid}(T^*) \rangle$, send $(pw_\Delta, T^*, R^{sid}(T^*))$ to $\mathcal{A}^*$.
 - Otherwise return “password file stolen” to $\mathcal{A}^*$.
- On $(EVALCODEDIFF, sid, T_1, T_2)$ from $\mathcal{A}^*$, if there is a record $\langle FILE, sid, S, pw_\Delta \rangle$ marked **compromised**, send $R^{sid}(T_2) - R^{sid}(T_1)$ to $\mathcal{A}^*$.
- On $(OFFLINETEST, sid, pw^*, T^*, code^*)$ from $\mathcal{A}^*$, do:
 - If there is a record $\langle FILE, sid, S, pw_\Delta \rangle$ marked **compromised**, do: if $pw^* = pw_\Delta$ and $code^* = R^{sid}(T^*)$, return “correct guess” to $\mathcal{A}^*$; otherwise return “wrong guess.”
 - Otherwise record $\langle OFFLINE, pw^*, T^*, code^* \rangle$.

Password Authentication

- On $(READCODE, sid, T)$ from D , send $R^{sid}(T)$ to D .
- On $(CSESSION, sid, ssid, S, pw, code)$ from C , send $(CSESSION, sid, ssid, C, S)$ to $\mathcal{A}^*$. If this is the first $CSESSION$ query for $ssid$, record $\langle ssid, C, S, pw, code \rangle$ and mark it **fresh**.
- On $(SSESSION, sid, ssid, C, T)$ from S , retrieve $\langle FILE, sid, S, pw_\Delta \rangle$, and send $(SSESSION, sid, ssid, S, C)$ to $\mathcal{A}^*$. If this is the first $SSESSION$ query for $ssid$, record $\langle ssid, S, C, pw_\Delta, R^{sid}(T) \rangle$ and mark it **fresh**.

Active Session Attacks

- On $(TEST, sid, ssid, P, pw^*, code^*)$ from $\mathcal{A}^*$, if there is a record $\langle ssid, P, P', pw, code \rangle$ marked **fresh**, do: if $pw^* = pw$ and $code^* = code$, mark it **compromised** and return “correct guess” to $\mathcal{A}^*$; otherwise mark it **interrupted** and return “wrong guess” to $\mathcal{A}^*$.
- On $(IMPERSONATE, sid, ssid, T, \Delta)$ from $\mathcal{A}^*$, if there is a record $\langle ssid, C, S, pw, code \rangle$ marked **fresh**, do: if there is a record $\langle FILE, sid, S, pw_\Delta \rangle$ marked **compromised**, $pw = pw_\Delta$ and $code = R^{sid}(T) + \Delta$, mark $\langle ssid, C, S, pw, code \rangle$ **compromised** and return “correct guess” to $\mathcal{A}^*$; otherwise mark it **interrupted** and return “wrong guess” to $\mathcal{A}^*$.

Key Generation and Authentication

- On $(NEWKEY, sid, ssid, P, SK^*)$ from $\mathcal{A}^*$, if there is a record $\langle ssid, P, P', pw, code \rangle$ not marked **completed**, do:
 - If the record is marked **compromised** set $SK := SK^*$.
 - If the record is marked **fresh**, a $(sid, ssid, SK')$ tuple was sent to P' , and at that time there was a record $\langle ssid, P', P, pw, code \rangle$ marked **fresh**, set $SK := SK'$.
 - Otherwise choose $SK \leftarrow_{\$} \{0, 1\}^\ell$.
 Finally, mark $\langle ssid, P, P', pw, code \rangle$ **completed** and send $(sid, ssid, SK)$ to P .

Fig. 4. Functionality $\mathcal{F}_{TFA-KE}$, with additions relative to $\mathcal{F}_{aPAKE}$ in shadowed text

the OFFLINETEST interface, by which an adversary who has compromised the server can test a single `pw` guess against the stolen file. Honest clients and servers perform online sessions via the CSESSION and SSESSION commands. An online attack against an honest session is modeled by the TESTPWD interface, to which the adversary must supply a single `pw` guess. After server compromise, the adversary can also use the IMPERSONATE interface, which models an online attacker’s ability to act like an honest server by using a stolen file. For both TESTPWD and IMPERSONATE, a failed online attack results in the attacked session being marked *interrupted*, which guarantees that only a single `pw` guess per attacked session is possible.

$\mathcal{F}_{\text{TFA-KE}}$ augments $\mathcal{F}_{\text{aPAKE}}$ for the two-factor setting, wherein there is a new party D that generates a random t -bit `code` for every epoch $T \in \mathbb{N}$. These `codes` are modeled by the functionality’s internal function $\text{R}^{\text{sid}} : \mathbb{N} \rightarrow \{0, 1\}^t$, which is perfectly random for every `sid`. Most of the changes from $\mathcal{F}_{\text{aPAKE}}$ to $\mathcal{F}_{\text{TFA-KE}}$ follow naturally:

1. Device D, on message (READCODE, `sid`, T), outputs $\text{R}^{\text{sid}}(T)$.
2. Client C supplies a `code` to CSESSION, and it is included in the resulting session record.
3. Server S supplies an epoch T to SSESSION, and the resulting session record includes $\text{R}^{\text{sid}}(T)$.
4. In NEWKEY, a client session and server session output in the same SK only if they agree on `pw` *and* `code`.

Notice that, while servers always run on a correct $\text{R}^{\text{sid}}(T)$, clients are “fallible” in a way that is controlled by the environment. In particular, real $\text{R}^{\text{sid}}(T)$ values generated by D do not have to be passed correctly to CSESSION. This modeling choice captures the usual UC (a)PAKE notion of security even in case of arbitrary mistyping. Furthermore, every `code` outputted by D is visible to the environment, and a protocol realizing $\mathcal{F}_{\text{TFA-KE}}$ must be secure against environments that may or may not leak this information to the adversary. Leaking a particular epoch’s $\text{R}^{\text{sid}}(T)$ corresponds to a lunchtime attack against D, and leaking every $\text{R}^{\text{sid}}(T)$ corresponds to full compromise of D. Again, this modeling choice is typical for UC (a)PAKE, where all parties’ `pw` inputs are visible to the environment and therefore potentially to the adversary.

Our notation treats the `code` space $\{0, 1\}^t$ as an abelian group, and we use $+$ and $-$ to denote addition and subtraction in that group. Looking ahead, our two TFA-KE protocols instantiate this group differently. The first (Section 5.3) casts $\{0, 1\}^t$ elements as integers in $\mathbb{Z}_{2^t}$, with $a + b$ and $a - b$ standing for addition and subtraction modulo 2^t . The second (Section 5.4) casts $\{0, 1\}^t$ elements as bitstrings, with both $a + b$ and $a - b$ standing for bitwise XOR.

Online security in $\mathcal{F}_{\text{TFA-KE}}$ is strengthened as compared to $\mathcal{F}_{\text{aPAKE}}$. In particular, a successful TESTPWD attack requires a correct guess of `code` in addition to `pw`. As long as the adversary has not learned $\text{R}^{\text{sid}}(T)$, this reduces the probability of success by a factor of 2^t . Since R^{sid} is a random function, leakage of $\text{R}^{\text{sid}}(T')$ for $T' \neq T$ (i.e. a lunchtime attack) reveals no information about $\text{R}^{\text{sid}}(T)$. An online IMPERSONATE attacker must, like an honest server, supply an epoch T .

We relax the security notion by also allowing the attacker to supply a `code` offset Δ . In effect, this enables an attacker to successfully IMPERSONATE against a client who has mistyped their `code`, if the attacker can guess the difference Δ between the correct and mistyped values. We leave it as an open problem to realize $\mathcal{F}_{\text{TFA-KE}}$ without this relaxation.

Offline security of $\mathcal{F}_{\text{TFA-KE}}$ is also strengthened over $\mathcal{F}_{\text{aPAKE}}$. Inevitably, an adversary who has compromised the server’s file can locally execute the client and server algorithms to test arbitrary inputs. We therefore include (T^*, code^*) in addition to pw^* in the input to OFFLINETESTPWD. A test is successful only if the `pw` guess is correct *and* $\text{code}^* = \text{R}^{\text{sid}}(T^*)$. As long as the attacker has not learned $\text{R}^{\text{sid}}(T)$ for any T , their offline dictionary attack is slowed down by a factor of 2^t . As in $\mathcal{F}_{\text{aPAKE}}$, offline attacks can be precomputed, in which case their success or failure is learned upon COMPROMISEFILE.

Finally, we make a second relaxation in the form of the EVALCODEDIFF interface, by which an attacker can, after server compromise, learn the difference between $\text{R}^{\text{sid}}(T)$ values across any two epochs. Notice that EVALCODEDIFF does not weaken offline security, since the attacker can just as well perform OFFLINETESTPWD queries for one epoch as for another. EVALCODEDIFF does however have a slight impact on online security: for an attacker who has compromised the server, learning any one epoch’s $\text{R}^{\text{sid}}(T)$ is sufficient to predict the honest client’s `code` for every epoch. From then forward, the attacker can always supply correct `code` values to TESTPWD. We leave it as an open problem to realize $\mathcal{F}_{\text{TFA-KE}}$ without this relaxation.

5 TFA-KE Protocols

In this section we show our TFA-KE protocols. We begin in Section 5.1 by defining $\mathcal{F}_{\text{CKEM}}$, a flexible building block that can be parameterized for different functions. Then, in Section 5.2, we present TFA_{gen} , a generic construction of a TFA-KE scheme from any tight OWF given a corresponding realization of $\mathcal{F}_{\text{CKEM}}$. Finally, Sections 5.3 and 5.4 contain the efficient realizations of $\mathcal{F}_{\text{CKEM}}$ needed to instantiate TFA_{gen} for our two tight OWF proposals f_{Vand} and f_{PoNR} .

5.1 Conditional KEM

Figure 5 is $\mathcal{F}_{\text{CKEM}}$, an ideal functionality for a Conditional Key Encapsulation Mechanism (CKEM). This functionality is parameterized by a circuit $\mathcal{C} : X \times Y \rightarrow \{0, 1\}$, which defines the condition that must be satisfied for key transfer to succeed. In anticipation of our TFA-KE application, we label our CKEM parties as server S , who runs on input $y \in Y$, and client C , who runs on input $x \in X$.¹⁵ When S and C interact, S first outputs a random key $\text{SK} \leftarrow_{\$} \{0, 1\}^{\kappa}$, and C then outputs key SK' . If $\mathcal{C}(x, y) = 1$, then $\text{SK}' := \text{SK}$; otherwise, $\text{SK}' \leftarrow_{\$} \{0, 1\}^{\kappa}$.

$\mathcal{F}_{\text{CKEM}}$ is designed to be realizable over insecure channels, so network adversary $\mathcal{A}^*$ has full control over the communication of S and C and can assume the

¹⁵ In usual KEM parlance, these parties might instead be labeled *sender* and *receiver*.

Functionality $\mathcal{F}_{\text{CKEM}}[\mathcal{C}, \mathcal{C}_{\text{Adv}}, \mathcal{L}]$ is parameterized by a circuit $\mathcal{C} : X \times Y \rightarrow \{0, 1\}$ for some X, Y , an accompanying ‘downgraded’ circuit $\mathcal{C}_{\text{Adv}} : X' \times Y \rightarrow \{0, 1\}$ for some X' , and leakage functions $\mathcal{L} = (\mathcal{L}_C, \mathcal{L}_S)$. Functionality interacts with honest parties (denoted C and S depending on the protocol role they play) and an adversary $\mathcal{A}^*$. The functionality is also parameterized by *session identifier*, sid , which we omit from the syntax below for visual clarity.

- On $(\text{SEVAL}, \text{ssid}, y)$ from honest party S for $y \in Y$: If there is no prior record $(\text{SEVAL}, \text{ssid}, \dots)$ then save $(\text{SEVAL}, \text{ssid}, S, y)$ marked **fresh** and send $(\text{SEVAL}, \text{ssid}, S, \mathcal{L}_S(y))$ to $\mathcal{A}^*$.
- On $(\text{CEVAL}, \text{ssid}, x)$ from honest party C for $x \in X$: If there is no prior record $(\text{CEVAL}, \text{ssid}, \dots)$ then save $(\text{CEVAL}, \text{ssid}, C, x)$ marked **fresh** and send $(\text{CEVAL}, \text{ssid}, C, \mathcal{L}_C(x))$ to $\mathcal{A}^*$.
- On $(\text{SOUT}, \text{ssid})$ from $\mathcal{A}^*$: If there are records $(\text{CEVAL}, \text{ssid}, C, x)$ and $(\text{SEVAL}, \text{ssid}, S, y)$ for some C, S, x, y , both marked **fresh**, then pick $\text{SK} \leftarrow_{\$} \{0, 1\}^\kappa$, mark the second record **completed**(SK), and output $(\text{SOUT}, \text{ssid}, \text{SK})$ to S .
- On $(\text{COUT}, \text{ssid})$ from $\mathcal{A}^*$: If there are records $(\text{CEVAL}, \text{ssid}, C, x)$ marked **fresh** and $(\text{SEVAL}, \text{ssid}, S, y)$ marked **completed**(SK) for some C, S, x, y, SK , then mark the first record **completed** and if $\mathcal{C}(x, y) = 1$ then output $(\text{COUT}, \text{ssid}, \text{SK})$ to C , else pick $\text{SK}' \leftarrow \{0, 1\}^\kappa$ and output $(\text{COUT}, \text{ssid}, \text{SK}')$ to C .
- On $(\text{ADVSEVAL}, \text{ssid}, y^*, \text{SK}^*)$ from $\mathcal{A}^*$ for $y^* \in Y \cup \{\perp\}$ and $\text{SK}^* \in \{0, 1\}^\kappa$: If there is a record $(\text{CEVAL}, \text{ssid}, C, x)$ marked **fresh** for some C, x , then mark this record **used** and if $y^* \neq \perp$ and $\mathcal{C}(x, y^*) = 1$ then output $(\text{COUT}, \text{ssid}, \text{SK}^*)$ to C , else pick $\text{SK}' \leftarrow \{0, 1\}^\kappa$ and output $(\text{COUT}, \text{ssid}, \text{SK}')$ to C .
- On $(\text{ADVCEVAL}, \text{ssid}, x^*)$ from $\mathcal{A}^*$ for $x^* \in X' \cup \{\perp\}$: If there is a record $(\text{SEVAL}, \text{ssid}, S, y)$ marked **fresh** for some S, y , then pick $\text{SK} \leftarrow_{\$} \{0, 1\}^\kappa$, mark this record **used**, output $(\text{SOUT}, \text{ssid}, \text{SK})$ to S , and if $x^* \neq \perp$ and $\mathcal{C}_{\text{Adv}}(x^*, y) = 1$ then output $(\text{COUT}, \text{ssid}, \text{SK})$ to $\mathcal{A}^*$ else output $(\text{COUT}, \text{ssid}, \perp)$ to $\mathcal{A}^*$.

Fig. 5. $\mathcal{F}_{\text{CKEM}}[\mathcal{C}, \mathcal{C}_{\text{Adv}}, \mathcal{L}]$: Conditional KEM over insecure channels

role of either party in interaction with the other. We relax $\mathcal{F}_{\text{CKEM}}$ with parameter $\mathcal{C}_{\text{Adv}} : X^* \times Y \rightarrow \{0, 1\}$, which is a “downgraded” condition circuit. $\mathcal{A}^*$, when acting as a dishonest client, learns SK if they supply $x^* \in X^*$ s.t. $\mathcal{C}_{\text{Adv}}(x^*, y) = 1$. $\mathcal{F}_{\text{CKEM}}$ is also parameterized by leakage functions $\mathcal{L} = (\mathcal{L}_C, \mathcal{L}_S)$ s.t. $\mathcal{L}_C(x)$ and $\mathcal{L}_S(y)$ are allowed to be leaked by a CKEM protocol to the network adversary.

5.2 Generic TFA-KE using Conditional KEM

Figure 6 is TFA_{gen} , a generic TFA-KE protocol from any tight one-way function f . More precisely, we rely on a commitment scheme COM and a compiler CommittedOwf , see Section 3.3, which transforms f into a function that in addition to computing $F = f(x)$ outputs a commitment C_F to F . We require that this “commitment-amended function” also realizes the tight OWF model $\mathcal{F}_{\text{OWF}}$. For concreteness we will assume that $\text{CommittedOwf}[f, \text{COM}]$ realizes $\mathcal{F}_{\text{OWF}}$ in the *generic group model* (GGM), as indeed is the case for our $\mathcal{F}_{\text{OWF}}$ proposals in

Section 3. Moreover, we require that OWF f can be decomposed as $f = f^* \circ f_{\text{pre}}^{\mathcal{HP}}$ (Definition 3) such that f^* is a collision-resistant function. We specify this decomposition so that TFA_{gen} can use NIZKs for knowledge of f^* preimage (i.e. hpw, s) instead of f preimage (i.e. pw, s). Thus we avoid proving knowledge of hash function H_p preimages, which would be highly inefficient. Additionally, TFA_{gen} relies on random oracles, NIZKs, and a realization of CKEM functionality $\mathcal{F}_{\text{CKEM}}$ parametrized by a circuit $\text{SSEM}[f^*, \text{COM}]$ that we describe below.

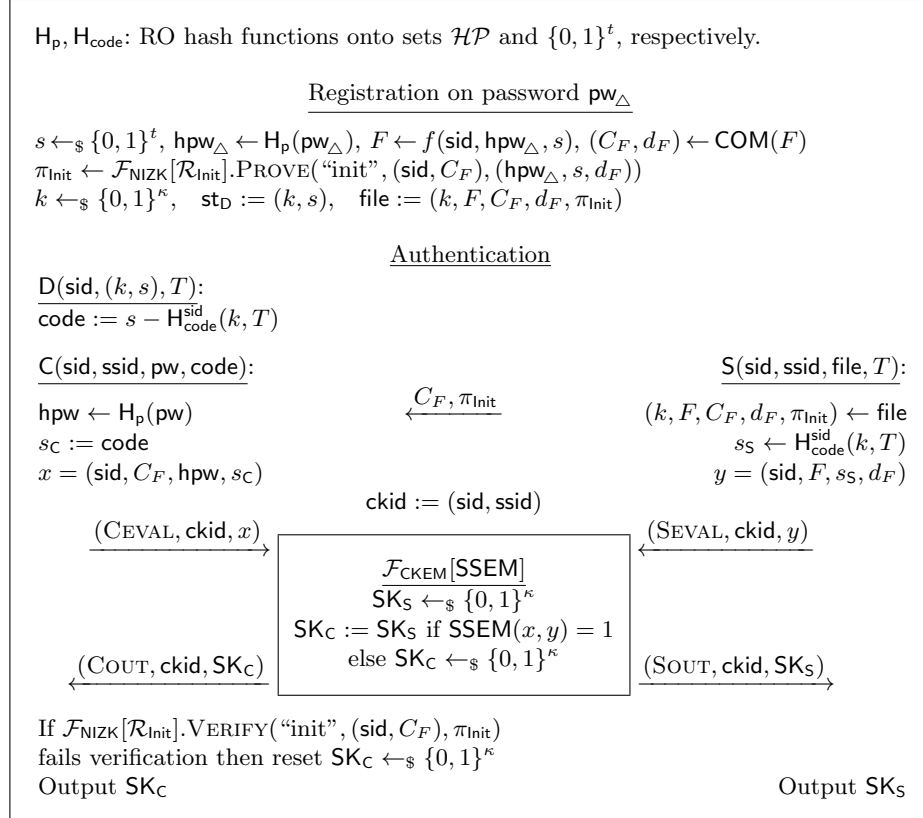


Fig. 6. TFA_{gen} : TFA-KE for a “commitment-amended” OWF function $f = f^* \circ f_{\text{pre}}^{\mathcal{HP}}$

In the initialization, $\text{Init}(1^\kappa, t, \text{sid}, \text{pw}_\Delta)$ samples a key $k \leftarrow \{0, 1\}^\kappa$ and a t -bit *initialization entropy* value s , evaluates the tight one-way function f on the hashed password and the initialization entropy, i.e. computes $F = f^*(\text{sid}, \text{hpw}_\Delta, s)$ where $\text{hpw}_\Delta = H_p(\text{pw}_\Delta)$ and H_p is an RO hash onto space $\mathcal{HP}$, commits to F as $(C_F, d_F) \leftarrow \text{COM}(F)$, and sets the device state as $\text{st}_D = (k, s)$ and the server password file as $\text{file} = (k, F, C_F, d_F, \pi_{\text{Init}})$ where π_{Init} is a NIZK for relation $\mathcal{R}_{\text{Init}}$

defined as follows:

$$\begin{aligned} \mathcal{R}_{\text{Init}} = \{ (\mathbb{x}, \mathbb{w}) = ((\text{sid}, C_F), ((\text{hpw}_\Delta, s, d_F))) \text{ s.t.} \\ C_F = \text{COM}(F; d_F) \text{ for } F = f^*(\text{sid}, \text{hpw}_\Delta, s) \} \end{aligned}$$

Given the setup above, the code generation $\text{AuthCodeGen}((k, s), T)$ is an “ s -shifted PRF”, $R_{(k, s)}^{\text{sid}}(T) = s - H_{\text{code}}^{\text{sid}}(k, T)$ where $H_{\text{code}}^{\text{sid}}$ is an RO hash onto $\{0, 1\}^t$.¹⁶ The authentication protocol (we omit sub-session identifiers from this overview), executed between the client C on (pw, code) and the server S on (file, T) proceeds as follows. C sets $\text{hpw} := H_p(\text{pw})$ and $s_C := \text{code}$, and S parses $\text{file} = (k, F, C_F, d_F, \pi_{\text{Init}})$ and sets $s_S := H_{\text{code}}^{\text{sid}}(k, T)$. Note that if C uses a device code for epoch T then $s_C = s - H_{\text{code}}^{\text{sid}}(k, T)$ and s_S form a *secret-sharing* of the initialization entropy s , namely $s = s_C + s_S$. Server S then sends C_F together with NIZK π_{Init} to C , and the two parties run a CKEM protocol on respective inputs $x = (\text{sid}, C_F, \text{hpw}, s_C)$ and $y = (\text{sid}, F, s_S, d_F)$, parametrized by circuit $\mathcal{C} = \text{SSEM}[f^*, \text{COM}]$ and leakage functions $\mathcal{L}_{\text{SSEM}} = (\mathcal{L}_C, \mathcal{L}_S)$ defined as follows:

$$\begin{aligned} \text{SSEM}[f^*, \text{COM}]((\text{sid}_C, C_F, \text{hpw}, s_C), (\text{sid}_S, F, s_S, d_F)) = \\ (\text{sid}_C \equiv \text{sid}_S \wedge F \equiv f^*(\text{sid}_C, \text{hpw}, s_C + s_S) \wedge C_F \equiv \text{COM}(F; d_F)) \\ \mathcal{L}_C(\text{sid}_C, C_F, \text{hpw}, s_C) = (\text{sid}_C, C_F) \\ \mathcal{L}_S(\text{sid}_S, F, s_S, d_F) = (\text{sid}_S, \text{COM}(F; d_F)) \end{aligned}$$

We call the above circuit SSEM for *Secret-Shared Evaluation Match*. Recall, see Section 5.1, that a CKEM functionality parametrized by condition $\mathcal{C}$ that takes inputs x and y from resp. the client C and the server S , outputs a random key SK_S to S , and if $\mathcal{C}(x, y) = 1$ then it outputs the same key $\text{SK}_C = \text{SK}_S$ to C , otherwise it samples SK_C as an independent random value. Consequently, if $\mathcal{C}$ is defined as above then $\text{SK}_C = \text{SK}_S$ if and only if (1) they use the same sid , (2) C contributes hpw, s_C s.t. $f^*(\text{sid}, \text{hpw}, s_C + s_S)$ evaluates to the server-held value $F = f^*(\text{sid}, \text{hpw}_\Delta, s)$, and (3) C contributes C_F which commits to F .

Recall also, see Section 5.1, that our notion of CKEM allows a malicious client to learn the server’s output key if a ‘downgraded’ condition $\mathcal{C}_{\text{Adv}}$ holds on (x^*, y) where x^* is an input of a malicious client. In our case it suffices to define $\mathcal{C}_{\text{Adv}} = \text{SSEM}_{\text{Adv}}[f^*]$ as follows:

$$\begin{aligned} \text{SSEM}_{\text{Adv}}[f^*](\text{sid}_C, \text{hpw}, s_C, (\text{sid}_S, F, s_S, d_F)) = \\ (\text{sid}_C \equiv \text{sid}_S \wedge F \equiv f^*(\text{sid}_C, \text{hpw}, s_C + s_S)) \end{aligned}$$

Note that the leakage functions $\mathcal{L}_C, \mathcal{L}_S$ imply that the computation might reveal C ’s inputs sid_C and C_F and S ’s inputs sid_S and $C_F = \text{COM}(F; d_F)$. Since all these values are public this leakage is innocuous in the context of our protocol. Finally, C verifies that π_{Init} is a valid proof for relation $\mathcal{R}_{\text{Init}}$ on statement (sid, C_F) , and if it does not hold then C resets its output SK_C as a random key.

¹⁶ As explained in Section 4, we treat $\{0, 1\}^t$ as an abelian group, which can be instantiated with either modular addition or bitwise XOR as the group operation.

Theorem 7. *Protocol TFA_{gen} realizes $\mathcal{F}_{\text{TFA-KE}}$ in the $(\mathcal{F}_{\text{CKEM}}[\text{SSEM}[f^*, \text{COM}], \text{SSEM}_{\text{Adv}}[f^*]], \mathcal{F}_{\text{NIZK}}[\mathcal{R}_{\text{Init}}])$ -hybrid world and GGM, assuming $\text{CommittedOwf}[f, \text{COM}]$ tightly realizes $\mathcal{F}_{\text{OWF}}$ in the GGM, $f = f^* \circ f_{\text{pre}}^{\mathcal{HP}}$ (Definition 3), f^* is collision-resistant, COM is a hiding and binding commitment scheme, hash function H_{code} is modeled as a random oracle, hash function H_p is modeled as a non-programmable random oracle, and at most $q_H \leq 2^{(\log |\mathcal{HP}| - \kappa)/2}$ queries are made to H_p .*

Due to space constraints, we defer the proof of Theorem 7 to Appendix E.

5.3 TFA-KE from f_{Vand}

To instantiate the generic TFA_{gen} construction for OWF f_{Vand} (Section 3.1) and commitment scheme ElG , we need a realization of $\mathcal{F}_{\text{CKEM}}[\text{SSEM}[f_{\text{Vand}}^*, \text{ElG}], \text{SSEM}_{\text{Adv}}[f_{\text{Vand}}^*]]$, which we provide as protocol $\text{CKEM}_{\text{Vand}}$ in Figure 7. This protocol is based on the following application of the binomial theorem:

$$\begin{aligned} \prod_{i=1}^N H_i^{(a+b)^i} &= \prod_{i=1}^N H_i^{\sum_{j=0}^i \binom{i}{j} a^{i-j} b^j} \\ &= \prod_{i=1}^N \prod_{j=0}^i H_i^{\binom{i}{j} a^{i-j} b^j} \\ &= \prod_{j=0}^N \left(\prod_{i=\max\{j, 1\}}^N H_i^{\binom{i}{j} a^{i-j}} \right)^{b^j} \end{aligned} \quad (5)$$

Setting $H_i = H_G^{\text{sid}}(i)$, $a = (\text{hpw}|0|s_C)$, and $b = s_S$, Equation 5 suggests an efficient way for two parties to securely evaluate $f_{\text{Vand}}^*(\text{sid}, \text{hpw}, s_C + s_S)$. The client computes $g_j := \prod_{i=\max\{j, 1\}}^N H_i^{\binom{i}{j} a^{i-j}}$ for every $j \in \{0, 1, \dots, N\}$ and sends the server $(\text{pk}_C, \mathbf{G}, \pi_C)$ s.t. $\mathbf{G}$ is a vector of ElGamal encryptions to the g_j 's under public key pk_C . π_C is a NIZK for relation $\mathcal{R}_C$ s.t. a statement $(\text{sid}, \text{pk}_C, \mathbf{G})$ with witness $(\text{hpw}, s_C, \mathbf{d}_G)$ belongs to $\mathcal{R}_C$ iff $\mathbf{G}$ is correctly computed for sid , hpw , s_C , pk_C , and encryption randomness $\mathbf{d}_G$.

Receiving vector $\mathbf{G}$, the server uses the homomorphic property of ElGamal encryption to complete the computation of $F' = f_{\text{Vand}}^*(\text{sid}, \text{hpw}, s_C + s_S) = \prod_{j=0}^N (G_j)^{b^j}$. This value is computed in a ciphertext under pk_C . To realize $\mathcal{F}_{\text{CKEM}}$, the server must send the client a randomly chosen SK conditioned on whether F' equals the server's input F . To achieve this, the server samples $K \leftarrow_{\S} \mathbb{G}$, defines $\text{SK} := H_{\text{SK}}^{\text{sid}}(K)$, samples $r \leftarrow_{\S} \mathbb{Z}_p$, and computes a ciphertext V for $K' = K \cdot (F'/F)^r$. K' equals K if $F' = F$ and is random otherwise. The server sends V to the client, who decrypts K' and outputs $\text{SK}' := H_{\text{SK}}^{\text{sid}}(K')$.¹⁷

¹⁷ Note the cruciality of the $j = 0$ factor in the outer product of Equation 5. $0^j = 0$ for all j , except $0^0 = 1$. Thus, without the $j = 0$ factor, setting $b = 0$ would cause $F' = 1$ regardless of the client's input.

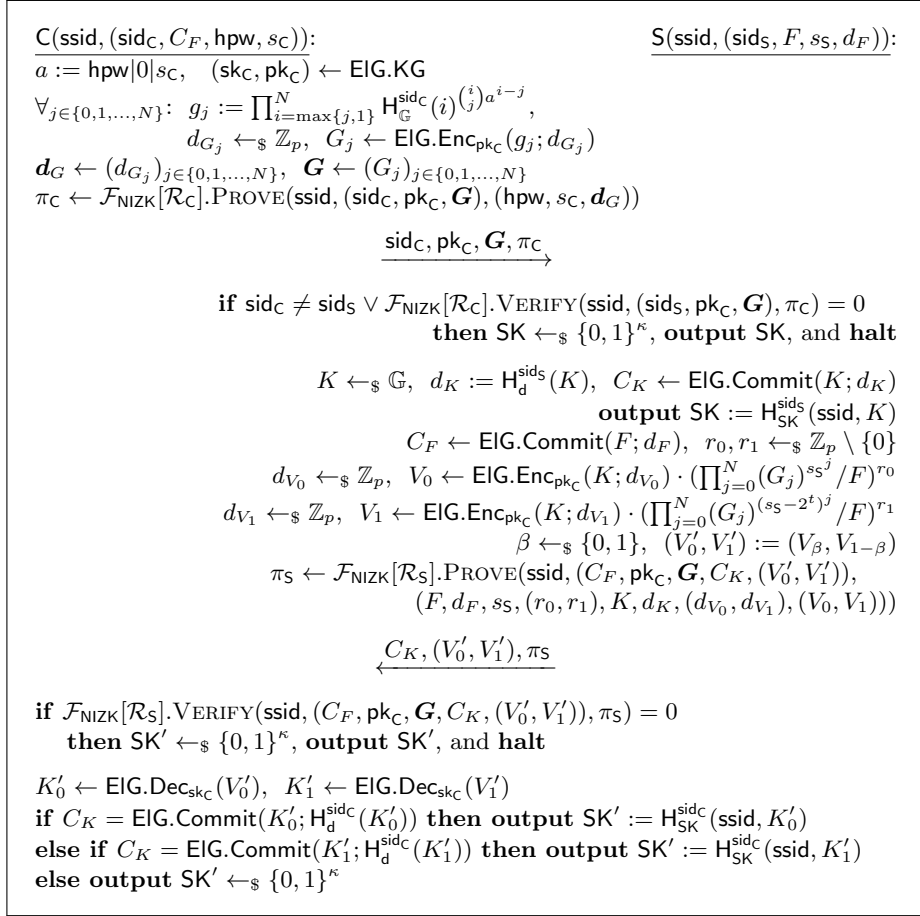


Fig. 7. CKEM_{Vand}: Conditional KEM protocol for SSEM of function f_{Vand}^*

Some details remain unexplained. First, we must compare F with $F' = f_{\text{Vand}}^*(\text{sid}, \text{hpw}, s_C + s_S \bmod 2^t)$, but our protocol adds a and b using arithmetic modulo p , not 2^t . To correct for this, the server performs the above procedure twice in parallel, once with $b = s_S$ and once with $b = s_S - 2^t$. To hide all information about s_S from the client, the server randomly flips the order of these two responses. The client now computes two possibilities for K' and must somehow decide which one to use. Therefore, the server sends the client a key confirmation $C_K \leftarrow \text{ELG.Commit}(K; H_d^{\text{sid}}(K))$. The client uses it to check which K' is correct, and outputs the corresponding $\text{SK}' := H_{\text{SK}}^{\text{sid}}(K')$. If neither K' matches C_K , the client instead outputs a random $\text{SK}' \leftarrow_{\$} \{0,1\}^\kappa$. Finally, the server must send a proof of knowledge π_S for relation $\mathcal{R}_S$, which proves their correct behavior. In particular, it is essential that the server uses the F committed in C_F and the K committed in C_K to compute both V ciphertexts. If a dishonest server were able to misbehave on one of the two V computations, they could gain information

about the relative values of s_C , s_S , and the s embedded in F by observing the success or failure of the protocol, i.e. whether or not $SK' = SK$.

Appendix F contains formal definitions of relations $\mathcal{R}_C$ and $\mathcal{R}_S$ and a discussion of how NIZKs for them are efficiently realized by standard techniques (as is a NIZK for relation $\mathcal{R}_{\text{Init}}$ of TFA_{gen} for OWF f_{Vand}).

Theorem 8. *Protocol $\text{CKEM}_{\text{Vand}}$ realizes $\mathcal{F}_{\text{CKEM}}[\text{SSEM}[f_{\text{Vand}}^*, \text{EIG}], \text{SSEM}_{\text{Adv}}[f_{\text{Vand}}^*]]$ in the $(\mathcal{F}_{\text{NIZK}}[\mathcal{R}_C], \mathcal{F}_{\text{NIZK}}[\mathcal{R}_S])$ -hybrid world, assuming hash functions H_d and H_{SK} are modeled as random oracles and DDH is hard in $\mathbb{G}$.*

Due to space constraints, we defer the proof of Theorem 8 to Appendix G.

5.4 TFA-KE from f_{PoNR}

An instantiation of protocol TFA_{gen} for OWF f_{PoNR} (Section 3.2) and commitment scheme EIG requires realization of $\mathcal{F}_{\text{CKEM}}[\text{SSEM}[f_{\text{PoNR}}^*, \text{EIG}], \text{SSEM}_{\text{Adv}}[f_{\text{PoNR}}^*]]$, which we provide as protocol $\text{CKEM}_{\text{PoNR}}$ in Figure 8. This protocol follows the same general structure as $\text{CKEM}_{\text{Vand}}$ (Section 5.3). In particular, the client on input (hpw, s_C) produces some ElGamal ciphertexts which the server on input (F, s_S) uses to compute an encryption of $F' = f_{\text{PoNR}}^*(\text{sid}, \text{hpw}, s_C \oplus s_S)$. The server then samples $K \leftarrow_{\mathbb{S}} \mathbb{G}$, defines $SK := H_{SK}^{\text{sid}}(K)$, samples $r \leftarrow_{\mathbb{S}} \mathbb{Z}_p$, and computes a ciphertext V for $K' = K \cdot (F'/F)^r$. The server sends V to the client, who decrypts K' and outputs $SK' := H_{SK}^{\text{sid}}(K')$ (as in $\text{CKEM}_{\text{Vand}}$, the client first verifies K' against key confirmation C_K).

$\text{CKEM}_{\text{PoNR}}$ differs from $\text{CKEM}_{\text{Vand}}$ in how F' is computed. Recalling that OWF f_{PoNR} resembles a product of N Naor-Reingold PRFs [22], we use the approach introduced by [11] for obliviously evaluating the Naor-Reingold PRF using oblivious transfer (OT), and we repeat the procedure in parallel for each $i \in [N]$. For each bit $s_C[j]$ of s_C , the client picks $\delta_{i,j} \leftarrow_{\mathbb{S}} \mathbb{Z}_p^*$ and acts as an OT sender on messages $(m_0, m_1) = (\delta_{i,j}, \delta_{i,j} \cdot H_x(i, j))$. If bit $s_C[j] = 1$, then the client reverses the order of the messages. The server, acting as OT receiver, uses $s_S[j]$ as their choice bit. Additionally, the client computes $g_i := H_{\mathbb{G}}^{\text{sid}}(i)^{\text{hpw}_i / \prod_{j \in [t]} \delta_{i,j}}$ and sends the server an encryption G_i of it. By exponentiating G_i to the product of their received OT messages, the server computes a ciphertext for $H_{\mathbb{G}}^{\text{sid}}(i)^{\text{hpw}_i \cdot \prod_{j \in [t], s_C[j] \oplus s_S[j] = 1} H_x(i, j)}$. The server combines these ciphertexts for all $i \in [N]$ (along with an encryption $\hat{G}$ of $\hat{g} := \prod_{i \in [N]} \hat{H}_{\mathbb{G}}^{\text{sid}}(i)^{\text{hpw}_i}$ additionally sent by the client) to create the encryption of F' .

Like $\text{CKEM}_{\text{Vand}}$, security of $\text{CKEM}_{\text{PoNR}}$ requires both parties to create NIZKs that prove their correct behavior and enable efficient extraction of their inputs. We therefore upgrade the OT building block to committed OT (COT; Appendix A). The client's proof π_C is for relation $\mathcal{R}_C$ and proves that $(G_i)_{i \in [N]}$, $\hat{G}$, and all COT inputs are correct for some hpw , s_C . The server's proof π_S is for relation $\mathcal{R}_S$ and proves that V is correctly computed for the committed F , the committed K , and the received COT outputs.

Appendix F contains formal definitions of relations $\mathcal{R}_C$ and $\mathcal{R}_S$ and a discussion of how NIZKs for them are efficiently realized by standard techniques (as is a NIZK for relation $\mathcal{R}_{\text{Init}}$ of TFA_{gen} for OWF f_{PoNR}).

$\mathbf{C}(\text{ssid}, (\text{sid}_C, C_F, \text{hpw}, s_C)):$
 $\mathbf{S}(\text{ssid}, (\text{sid}_S, F, s_S, d_F)):$

$(\text{sk}_C, \text{pk}_C) \leftarrow \text{ELG.KG}$

$\forall i \in [N]: \delta_{i,1}, \dots, \delta_{i,t} \leftarrow_{\$} \mathbb{Z}_p^*, \Delta_i := \prod_{j \in [t]} \delta_{i,j}$

$g_i := H_G^{\text{sid}_C}(i)^{\text{hpw}_i / \Delta_i}$

$d_{G_i} \leftarrow_{\$} \mathbb{Z}_p, G_i \leftarrow \text{ELG.Enc}_{\text{pk}_C}(g_i; d_{G_i})$

$\delta \leftarrow (\delta_{i,j})_{i \in [N], j \in [t]}, \mathbf{d}_G \leftarrow (d_{G_i})_{i \in [N]}, \mathbf{G} \leftarrow (G_i)_{i \in [N]}$

$\hat{g} := \prod_{i \in [N]} H_G^{\text{sid}_C}(i)^{\text{hpw}_i}, d_{\hat{G}} \leftarrow_{\$} \mathbb{Z}_p, \hat{G} := \text{ELG.Enc}_{\text{pk}_C}(\hat{g}; d_{\hat{G}})$

$\forall j \in [t]: \forall i \in [N]: (m_{i,j,\text{sc}[j]}, m_{i,j,1-\text{sc}[j]}) := (\delta_{i,j}, \delta_{i,j} \cdot H_x(i, j))$

$(C_{m_{j,0}}, d_{m_{j,0}}) \leftarrow \text{Ped}^N((m_{i,j,0})_{i \in [N]})$

$(C_{m_{j,1}}, d_{m_{j,1}}) \leftarrow \text{Ped}^N((m_{i,j,1})_{i \in [N]})$

$\mathbf{d}_m \leftarrow (d_{m_{j,0}}, d_{m_{j,1}})_{j \in [t]}, \mathbf{C}_m \leftarrow (C_{m_{j,0}}, C_{m_{j,1}})_{j \in [t]}$

$\pi_C \leftarrow \mathcal{F}_{\text{NIZK}}[\mathcal{R}_C].\text{PROVE}(\text{ssid}, (\text{sid}_C, \text{pk}_C, \mathbf{G}, \hat{G}, \mathbf{C}_m), (\text{hpw}, s_C, \mathbf{d}_G, d_{\hat{G}}, \delta, \mathbf{d}_m))$

$\xrightarrow{\text{sid}_C, \text{pk}_C, \mathbf{G}, \hat{G}, \pi_C}$

$\forall j \in [t]:$

$\xrightarrow{(m_{i,j,0})_{i \in [N]}, d_{m_{j,0}}}$
 $\xrightarrow{(m_{i,j,1})_{i \in [N]}, d_{m_{j,1}}}$

$\mathcal{F}_{\text{COT}}$

$\xleftarrow{s_S[j]}$
 $\xrightarrow{(m_{i,j})_{i \in [N]}, d_{m_j}, C_{m_{j,1-s_S[j]}}}$

$\mathbf{m}^S \leftarrow (m_{i,j})_{i \in [N], j \in [t]}, \mathbf{d}_m^S \leftarrow (d_{m_j})_{j \in [t]}$

$\forall j \in [t]: C_{m_{j,s_S[j]}} \leftarrow \text{Ped}^N((m_{i,j})_{i \in [N]}; d_{m_j})$

$\mathbf{C}_m \leftarrow (C_{m_{j,0}}, C_{m_{j,1}})_{j \in [t]}$

if $\text{sid}_C \neq \text{sid}_S \vee \mathcal{F}_{\text{NIZK}}[\mathcal{R}_C].\text{VERIFY}(\text{ssid}, (\text{sid}_S, \text{pk}_C, \mathbf{G}, \hat{G}, \mathbf{C}_m), \pi_C) = 0$

then $\text{SK} \leftarrow_{\$} \{0, 1\}^\kappa$, **output** SK, and **halt**

$K \leftarrow_{\$} \mathbb{G}, d_K := H_d^{\text{sid}_S}(K), C_K \leftarrow \text{ELG.Commit}(K; d_K)$

output SK := $H_{\text{SK}}^{\text{sid}_S}(\text{ssid}, K)$

$C_F \leftarrow \text{ELG.Commit}(F; d_F), r \leftarrow_{\$} \mathbb{Z}_p \setminus \{0\}$

$d_V \leftarrow_{\$} \mathbb{Z}_p, V \leftarrow \text{ELG.Enc}_{\text{pk}_C}(K; d_V) \cdot \left(\hat{G} \cdot \prod_{i=1}^N (G_i)^{\prod_{j \in [t]} m_{i,j} / F} \right)^r$

$\pi_S \leftarrow \mathcal{F}_{\text{NIZK}}[\mathcal{R}_S].\text{PROVE}(\text{ssid}, (C_F, \text{pk}_C, \mathbf{G}, \hat{G}, \mathbf{C}_m, C_K, V), (F, d_F, s_S, \mathbf{m}^S, \mathbf{d}_m^S, r, K, d_K, d_V))$

$\xleftarrow{C_K, V, \pi_S}$

if $\mathcal{F}_{\text{NIZK}}[\mathcal{R}_S].\text{VERIFY}(\text{ssid}, (C_F, \text{pk}_C, \mathbf{G}, \hat{G}, \mathbf{C}_m, C_K, V), \pi_S) = 0$

then $\text{SK}' \leftarrow_{\$} \{0, 1\}^\kappa$, **output** SK', and **halt**

$K' \leftarrow \text{ELG.Dec}_{\text{sk}_C}(V)$

if $C_K = \text{ELG.Commit}(K'; H_d^{\text{sid}_C}(K'))$ **then** **output** SK' := $H_{\text{SK}}^{\text{sid}_C}(\text{ssid}, K')$

else **output** SK' $\leftarrow_{\$} \{0, 1\}^\kappa$

Fig. 8. CKEM_{PoNR}: Conditional KEM protocol for SSEM of function f_{PoNR}^*

Theorem 9. *Protocol $\text{CKEM}_{\text{PoNR}}$ realizes $\mathcal{F}_{\text{CKEM}}[\text{SSEM}[f_{\text{PoNR}}^*, \text{EIG}], \text{SSEM}_{\text{Adv}}[f_{\text{PoNR}}^*]]$ in the $(\mathcal{F}_{\text{COT}}[\text{Ped}], \mathcal{F}_{\text{NIZK}}[\mathcal{R}_C], \mathcal{F}_{\text{NIZK}}[\mathcal{R}_S])$ -hybrid world, assuming hash functions H_d and H_{SK} are modeled as random oracles and DDH is hard in $\mathbb{G}$.*

Due to space constraints, we defer the proof of Theorem 9 to Appendix H.

References

1. N. Alon, S. Hoory, and N. Linial. The Moore Bound for Irregular Graphs. *Graphs and Combinatorics*, 18(1):53–57, Mar. 2002.
2. T. Attema and R. Cramer. Compressed sigma-protocol theory and practical application to plug & play secure algorithmics. In *Advances in Cryptology – CRYPTO 2020: 40th Annual International Cryptology Conference, CRYPTO 2020, Santa Barbara, CA, USA, August 17–21, 2020, Proceedings, Part III*, page 513–543, Berlin, Heidelberg, 2020. Springer-Verlag.
3. J. Bonneau. The science of guessing: Analyzing an anonymized corpus of 70 million passwords. In *Proceedings of the 2012 IEEE Symposium on Security and Privacy, SP ’12*, page 538–552, USA, 2012. IEEE Computer Society.
4. T. Bradley, S. Jarecki, and J. Xu. Strong asymmetric PAKE based on trapdoor CKEM. In A. Boldyreva and D. Micciancio, editors, *Advances in Cryptology – CRYPTO 2019*, pages 798–825, Cham, 2019. Springer International Publishing.
5. C. Cachin and J. Camenisch. Optimistic Fair Secure Computation. In M. Bellare, editor, *Advances in Cryptology – CRYPTO 2000*, pages 93–111, Berlin, Heidelberg, 2000. Springer.
6. J. Camenisch and M. Stadler. Proof systems for general statements about discrete logarithms. Technical Report No. 260, March 1997. <ftp://ftp.inf.ethz.ch/pub/publications/tech-reports/>.
7. Y.-H. Chen and Y. Lindell. Optimizing and Implementing Fischlin’s Transform for UC-Secure Zero Knowledge. *IACR Communications in Cryptology*, 1(2), July 2024.
8. C. Crepeau. Verifiable Disclosure of Secrets and Applications. In J.-J. Quisquater and J. Vandewalle, editors, *Advances in Cryptology – EUROCRYPT ’89*, pages 150–154, Berlin, Heidelberg, 1989. Springer.
9. P. Erdős and H. Sachs. Reguläre Graphen gegebener Taillenweite mit minimaler Knotenzahl. *Wissenschaftliche Zeitschrift der Martin-Luther-Universität Halle-Wittenberg*, Math.-Nat. XII(3):251–258, Mar. 1963.
10. M. Fischlin. Communication-Efficient Non-interactive Proofs of Knowledge with Online Extractors. In V. Shoup, editor, *Advances in Cryptology – CRYPTO 2005*, pages 152–168, Berlin, Heidelberg, 2005. Springer.
11. M. J. Freedman, Y. Ishai, B. Pinkas, and O. Reingold. Keyword Search and Oblivious Pseudorandom Functions. In J. Kilian, editor, *Theory of Cryptography*, pages 303–324, Berlin, Heidelberg, 2005. Springer.
12. Z. Füredi and M. Simonovits. The History of Degenerate (Bipartite) Extremal Graph Problems. In L. Lovász, I. Z. Ruzsa, and V. T. Sós, editors, *Erdős Centennial*, pages 169–264. Springer, Berlin, Heidelberg, 2013.
13. C. Gentry, P. MacKenzie, and Z. Ramzan. A Method for Making Password-Based Key Exchange Resilient to Server Compromise. In C. Dwork, editor, *Advances in Cryptology – CRYPTO 2006*, pages 142–159, Berlin, Heidelberg, 2006. Springer.

14. O. Goldreich, S. Micali, and A. Wigderson. How to play any mental game or A completeness theorem for protocols with honest majority. In A. Aho, editor, *19th ACM STOC*, pages 218–229. ACM Press, May 1987.
15. Y. Gu, S. Jarecki, P. Kedzior, P. Nazarian, and J. Xu. Threshold pake with security against compromise of all servers. In K.-M. Chung and Y. Sasaki, editors, *Advances in Cryptology – ASIACRYPT 2024*, pages 66–100, Singapore, 2025. Springer Nature Singapore.
16. S. Jarecki, H. Krawczyk, and J. Xu. OPAQUE: An asymmetric PAKE protocol secure against pre-computation attacks. In J. B. Nielsen and V. Rijmen, editors, *EUROCRYPT 2018, Part III*, volume 10822 of *LNCS*, pages 456–486. Springer, Cham, Apr. / May 2018.
17. S. Jarecki and V. Shmatikov. Efficient Two-Party Secure Computation on Committed Inputs. In M. Naor, editor, *Advances in Cryptology - EUROCRYPT 2007*, pages 97–114, Berlin, Heidelberg, 2007. Springer.
18. S. Kim, H. Lee, and J. H. Seo. Efficient zero-knowledge arguments in discrete logarithm setting: Sublogarithmic proof or sublinear verifier. In S. Agrawal and D. Lin, editors, *Advances in Cryptology – ASIACRYPT 2022*, pages 403–433, Cham, 2022. Springer Nature Switzerland.
19. P. D. MacKenzie, T. Shrimpton, and M. Jakobsson. Threshold password-authenticated key exchange. In M. Yung, editor, *CRYPTO 2002*, volume 2442 of *LNCS*, pages 385–400. Springer, Berlin, Heidelberg, Aug. 2002.
20. I. McQuoid, M. Rosulek, and J. Xu. How to obfuscate MPC inputs. In E. Kiltz and V. Vaikuntanathan, editors, *Theory of Cryptography*, pages 151–180, Cham, 2022. Springer Nature Switzerland.
21. I. McQuoid and J. Xu. An efficient strong asymmetric PAKE compiler instantiable from group actions. In *Advances in Cryptology – ASIACRYPT 2023: 29th International Conference on the Theory and Application of Cryptology and Information Security, Guangzhou, China, December 4–8, 2023, Proceedings, Part VIII*, page 176–207, Berlin, Heidelberg, 2023. Springer-Verlag.
22. M. Naor and O. Reingold. Number-theoretic constructions of efficient pseudo-random functions. *J. ACM*, 51(2):231–262, Mar. 2004.
23. R. Radwan and S. Zejnilovic. Password reuse is rampant: nearly half of observed user logins are compromised, 2025. <https://blog.cloudflare.com/password-reuse-rampant-half-user-logins-compromised/>.
24. J. T. Schwartz. Fast probabilistic algorithms for verification of polynomial identities. *J. ACM*, 27(4):701–717, Oct. 1980.
25. M. Shirvanian, S. Jarecki, N. Saxena, and N. Nathan. Two-factor authentication resilient to server compromise using mix-bandwidth devices. In *21st Annual Network and Distributed System Security Symposium, NDSS 2014, San Diego, California, USA, February 23-26, 2014*. The Internet Society, 2014.
26. V. Shoup. Lower bounds for discrete logarithms and related problems. In W. Fumy, editor, *Advances in Cryptology — EUROCRYPT ’97*, pages 256–266, Berlin, Heidelberg, 1997. Springer Berlin Heidelberg.
27. M. Stadler. Publicly verifiable secret sharing. In U. Maurer, editor, *Advances in Cryptology — EUROCRYPT ’96*, pages 190–199, Berlin, Heidelberg, 1996. Springer Berlin Heidelberg.
28. A. C.-C. Yao. Protocols for secure computations (extended abstract). In *23rd FOCS*, pages 160–164. IEEE Computer Society Press, Nov. 1982.

A Additional Preliminaries

In this section we include standard definitions deferred from Section 2 due to space constraints.

Hardness Assumptions. We recall the Discrete Logarithm and Decisional Diffie-Hellman assumptions.

Definition 6 (Discrete Logarithm Problem). Let $\mathbb{G}$ be a cyclic group of prime order p with generator g . The Discrete Logarithm problem (DL) on $\mathbb{G}$ is, given (g, g^ξ) where $\xi \leftarrow_{\$} \mathbb{Z}_p$, to find ξ . We say that DL is hard in $\mathbb{G}$ if all algorithms running in polynomial time have no better than negligible probability of solving the DL problem.

Definition 7 (Decisional Diffie-Hellman Problem). Let $\mathbb{G}$ be a cyclic group of prime order p with generator g . The Decisional Diffie-Hellman problem (DDH) on $\mathbb{G}$ is to distinguish Diffie-Hellman tuples from random tuples. In particular, the DDH advantage of an algorithm $\mathcal{A}$ is defined as:

$$\left| \Pr_{a,b \leftarrow_{\$} \mathbb{Z}_p} [\mathcal{A}(g, g^a, g^b, g^{ab}) = 1] - \Pr_{a,b,c \leftarrow_{\$} \mathbb{Z}_p} [\mathcal{A}(g, g^a, g^b, g^c) = 1] \right|$$

We say that DDH is hard in $\mathbb{G}$ if all algorithms running in polynomial time have no better than negligible DDH advantage.

The hardness of DDH in $\mathbb{G}$ implies the hardness of DL in $\mathbb{G}$.

Commitment Hiding and Binding. As mentioned in Section 2, a secure commitment scheme needs to be hiding and binding, which we formally define below. Both definitions consider an adversary whose implicit input is a security parameter, which is also an implicit input of algorithm COM and an index that defines message, decommitment, and commitment spaces $\mathcal{M}$, $\mathcal{D}$, and $\mathcal{C}$. As a consequence, the adversarial advantage expressions defined below are also functions of the security parameter.

Definition 8 (Commitment Hiding). Let $\text{COM} : \mathcal{M} \times \mathcal{D} \rightarrow \mathcal{C}$ be a commitment scheme. Suppose $\mathcal{A}$ is an algorithm that first outputs two messages $m_0, m_1 \in \mathcal{M}$. The advantage of $\mathcal{A}$ against the hiding property of COM is defined as:

$$\text{Adv}_{\text{COM}}^{\text{hid}}[\mathcal{A}] = \left| \Pr_{(C,d) \leftarrow \text{COM}(m_0)} [\mathcal{A}(C) = 1] - \Pr_{(C,d) \leftarrow \text{COM}(m_1)} [\mathcal{A}(C) = 1] \right|$$

We say that COM is hiding if $\text{Adv}_{\text{COM}}^{\text{hid}}[\mathcal{A}]$ is negligible for all efficient $\mathcal{A}$.

Definition 9 (Commitment Binding). Let $\text{COM} : \mathcal{M} \times \mathcal{D} \rightarrow \mathcal{C}$ be a commitment scheme. The advantage of algorithm $\mathcal{A}$ against the binding property of COM , denoted $\text{Adv}_{\text{COM}}^{\text{bind}}[\mathcal{A}]$, is defined as the probability that $\mathcal{A}$ finds $m_0, m_1 \in \mathcal{M}$ and $d_0, d_1 \in \mathcal{D}$ such that $m_0 \neq m_1$ and $\text{COM}(m_0; d_0) = \text{COM}(m_1; d_1)$. We say that COM is binding if $\text{Adv}_{\text{COM}}^{\text{bind}}[\mathcal{A}]$ is negligible for all efficient $\mathcal{A}$.

ElGamal Commitments. We define ElGamal commitment scheme ElG.Commit with message space $\mathcal{M} = \mathbb{G}$, randomness space $\mathcal{D} = \mathbb{Z}_p^2$, and commitment space $\mathcal{C} = \mathbb{G}^3$, where $\mathbb{G}$ is a group of prime order p with generator g . The commitment algorithm is:

$$\text{ElG.Commit}(m; d_1, d_2) := (g^{d_1}, g^{d_2}, m \cdot g^{d_1 d_2})$$

ElG.Commit is perfectly binding and computationally hiding under the DDH assumption. Where the meaning is clear from context, we sometimes abbreviate ElG.Commit as simply ElG .

Pedersen Commitments. We define (“compact”) Pedersen commitment scheme Ped^N with message space $\mathcal{M} = \mathbb{Z}_p^N$, randomness space $\mathcal{D} = \mathbb{Z}_p$, and commitment space $\mathcal{C} = \mathbb{G}$, where $\mathbb{G}$ is a group of prime order p with generators $g, h_1, \dots, h_N$. Security is based on the hardness of computing a DL between bases $g, h_1, \dots, h_N$, which can be assumed if, for example, they are provided by a common reference string. Our protocols all use the random oracle model, so $h_1, \dots, h_N$ could be outputs of a random oracle. The commitment algorithm is:

$$\text{Ped}^N(m_1, \dots, m_N; d) := g^d \cdot (h_1)^{m_1} \cdot \dots \cdot (h_N)^{m_N}$$

Ped^N is perfectly hiding and computationally binding under the DL assumption.

Committed Oblivious Transfer. Figure 9 is $\mathcal{F}_{\text{COT}}$, an ideal functionality for committed oblivious transfer (COT) with commitment scheme COM . COT, introduced in [8], is a strengthened variant of standard oblivious transfer that allows parties to prove correct behavior relative to committed values. In particular, the receiver learns commitments to both sender messages but only learns the corresponding decommitment to one of them.

The functionality interacts with set of honest parties $\mathcal{P}$ and ideal adversary $\mathcal{A}^*$. The functionality is also parameterized by a global *session identifier*, sid , which we omit from the syntax below for visual clarity.

- On $(\text{SEND}, \text{ssid}_S, m_0, d_0, m_1, d_1)$ from $S \in \mathcal{P} \cup \{\mathcal{A}^*\}$ for $m_0, m_1 \in \mathcal{M}$ and $d_0, d_1 \in \mathcal{D}$: If there is no prior record $(\text{SEND}, \text{ssid}_S, \dots)$ then save $(\text{SEND}, \text{ssid}_S, S, m_0, d_0, m_1, d_1)$ marked **fresh** and send $(\text{SEND}, \text{ssid}_S, S, \text{COM}(m_0; d_0), \text{COM}(m_1; d_1))$ to $\mathcal{A}^*$.
- On $(\text{RCV}, \text{ssid}_R, \sigma)$ from $R \in \mathcal{P} \cup \{\mathcal{A}^*\}$ for $\sigma \in \{0, 1\}$: If there is no prior record $(\text{RCV}, \text{ssid}_R, \dots)$ then save $(\text{RCV}, \text{ssid}_R, R, \sigma)$ marked **fresh** and send $(\text{RCV}, \text{ssid}_R, R)$ to $\mathcal{A}^*$.
- On $(\text{COMPLETE}, \text{ssid}_S, \text{ssid}_R)$ from $\mathcal{A}^*$: If there are records $(\text{SEND}, \text{ssid}_S, S, m_0, d_0, m_1, d_1)$ and $(\text{RCV}, \text{ssid}_R, R, \sigma)$, both marked **fresh**, then mark them both **completed**, and output $(\text{COMPLETE}, \text{ssid}_R, m_\sigma, d_\sigma, \text{COM}(m_{1-\sigma}; d_{1-\sigma}))$ to R .

Fig. 9. $\mathcal{F}_{\text{COT}}[\text{COM}]$: Committed oblivious transfer functionality

One of our two TFA-KE protocols ($\text{CKEM}_{\text{PoNR}}$, see Section 5.4) utilizes a COT protocol for commitment scheme Ped^N . There are two COT protocols [5,17] in the literature that seem suitable. The latter is proven UC-secure, but for a model that differs somewhat from our $\mathcal{F}_{\text{COT}}$ in that it does not consider insecure channels and it does not expose commitments and decommitments for use by higher-level applications. We believe that both COT protocols [5,17] could be proven UC-secure in our model with only small adjustments, but a detailed re-analysis of these protocols is outside the scope of our work.

B Proof of f_{PoNR} One-Wayness and Tightness

In this section we prove Lemma 3, Theorem 5, and Theorem 6, which state, respectively, that f_{PoNR}^* is collision-resistant, f_{PoNR} realizes $\mathcal{F}_{\text{OWF}}$, and f_{PoNR} is tight.

B.1 Proof of Lemma 3

Proof. To prove Lemma 3, first suppose there exists an algorithm $\mathcal{A}$ that outputs $(\text{hpw}, s) \neq (\text{hpw}', s')$ and sid s.t. $f_{\text{PoNR}}^*(\text{sid}, \text{hpw}, s) = f_{\text{PoNR}}^*(\text{sid}, \text{hpw}', s')$ and $\exists_{i \in [N]} \text{hpw}_i \cdot \prod_{j \in [t], s[j]=1} H_x(i, j) \neq \text{hpw}'_i \cdot \prod_{j \in [t], s'[j]=1} H_x(i, j) \vee \text{hpw}_i \neq \text{hpw}'_i$. As in the proof of Lemma 1, there is a straightforward reduction from $\mathcal{A}$ that solves DL in $\mathbb{G}$.

Here we see the usefulness of the $\hat{H}_{\mathbb{G}}$ bases, as they ensure that collision is only possible for $\text{hpw} = \text{hpw}'$ and $s \neq s'$ s.t. $\forall_{i \in [N]} \prod_{j \in [t], s[j]=1} H_x(i, j) = \prod_{j \in [t], s'[j]=1} H_x(i, j)$. We will show that such $s, s' \in \{0, 1\}^t$ exist with negligible

probability over the randomness of H_x . We abbreviate $H_x(i, j)$ by $x_{i,j}$.

$$\begin{aligned}
& \Pr_{\substack{x_{1,1}, \dots, x_{1,t} \leftarrow \mathbb{Z}_p^* \\ \dots \\ x_{N,1}, \dots, x_{N,t} \leftarrow \mathbb{Z}_p^*}} \left[\bigcup_{\substack{s, s' \in \{0,1\}^t \\ s \neq s'}} \bigcap_{i \in [N]} \prod_{\substack{j \in [t] \\ s[j]=1}} x_{i,j} = \prod_{\substack{j \in [t] \\ s'[j]=1}} x_{i,j} \right] \\
& \leq \sum_{\substack{s, s' \in \{0,1\}^t \\ s \neq s'}} \Pr_{\substack{x_{1,1}, \dots, x_{1,t} \leftarrow \mathbb{Z}_p^* \\ \dots \\ x_{N,1}, \dots, x_{N,t} \leftarrow \mathbb{Z}_p^*}} \left[\bigcap_{i \in [N]} \prod_{\substack{j \in [t] \\ s[j]=1}} x_{i,j} = \prod_{\substack{j \in [t] \\ s'[j]=1}} x_{i,j} \right] \\
& = \sum_{\substack{s, s' \in \{0,1\}^t \\ s \neq s'}} \prod_{i \in [N]} \Pr_{x_{i,1}, \dots, x_{i,t} \leftarrow \mathbb{Z}_p^*} \left[\prod_{\substack{j \in [t] \\ s[j]=1}} x_{i,j} = \prod_{\substack{j \in [t] \\ s'[j]=1}} x_{i,j} \right] \\
& = \sum_{\substack{s, s' \in \{0,1\}^t \\ s \neq s'}} \left(\Pr_{x_{1,1}, \dots, x_{1,t} \leftarrow \mathbb{Z}_p^*} \left[\prod_{\substack{j \in [t] \\ s[j]=1}} x_{1,j} = \prod_{\substack{j \in [t] \\ s'[j]=1}} x_{1,j} \right] \right)^N \\
& = \sum_{\substack{s, s' \in \{0,1\}^t \\ s \neq s'}} \left(\frac{1}{p-1} \right)^N \\
& < \frac{4^t}{(p-1)^N} \tag{6}
\end{aligned}$$

At the fourth line of the above, observe that, since $s \neq s'$, there must be at least one $x_{1,j}$ variable that appears on only one side of the equation. That variable can be isolated to show that it has exactly one possible value in $\mathbb{Z}_p^*$ that satisfies the equation.

To ensure that collisions are negligibly probable, N must be sufficiently large. In particular, by upper-bounding Expression 6 with $2^{-\kappa}$, we obtain the bound $N \geq \frac{\kappa+2t}{\log(p-1)}$ and complete the proof of Lemma 3. Note that for all parameters we consider reasonable (e.g. $\kappa = 128$, $t = 30$, and $\log p = 256$) any $N \geq 1$ suffices.

B.2 Proof of Theorem 5

Proof. To prove the security of f_{PoNR} we provide in Figure 10 the simulator SIM_{PoNR} , which operates in the GGM and ROM. SIM_{PoNR} follows the same simulation strategy as f_{Vand} 's SIM_{Vand} . They differ only in that SIM_{PoNR} has the additional random oracles $\hat{H}_{\mathbb{G}}$ and H_x to program, and that the $\hat{H}_{\mathbb{G}}$ bases participate in SIM_{PoNR} 's internal $\mathbb{G}$ element descriptions. The two simulators' most important difference is in their underlying definitions of $k_i(\text{pw}, s)$.

Like SIM_{Vand} , SIM_{PoNR} perfectly emulates the real world except in the case of a discrete logarithm break, an H_p premonition, or a OWF collision. The first event is negligibly probable in the GGM by the same argument as in the proof of Theorem 1. The second event is also negligibly probable under the assumption that $q_{H\mathbb{G}}^2 \leq 2^{N \log p - \kappa}$. By Lemma 3, in the case that no DL break occurs, the

Whenever a new $\mathbb{G}$ element is sent to the environment, the simulator picks a unique random string to represent it (except when this behavior is overridden by the code below). The simulator maintains internal descriptions for these $\mathbb{G}$ elements in the form of three tables of maps α , $\hat{\alpha}$, and β . $\alpha[\gamma]$ and $\hat{\alpha}[\gamma]$ are maps $(\text{sid}, i) \mapsto \mathbb{Z}_p$ and $\beta[\gamma]$ is a map $\text{sid} \mapsto \mathbb{Z}_p$ such that $\gamma \in \mathbb{G}$ is described as:

$$\gamma = \prod_{\text{sid}, i} H_{\mathbb{G}}(\text{sid}, i)^{\alpha[\gamma](\text{sid}, i)} \cdot \hat{H}_{\mathbb{G}}(\text{sid}, i)^{\hat{\alpha}[\gamma](\text{sid}, i)} \cdot \prod_{\text{sid}} (y_{\text{sid}})^{\beta[\gamma](\text{sid})} \quad (7)$$

The simulator also maintains a table $X : (\text{sid}, \text{pw}, s) \mapsto \{0, 1\}$ of possible OWF inversions that are known to be either correct or wrong.

- On fresh query $H_{\mathbb{G}}(\text{sid}, i)$, define new $\gamma \in \mathbb{G}$ and return it. $\alpha[\gamma](\text{sid}, i) := 1$.
- On fresh query $\hat{H}_{\mathbb{G}}(\text{sid}, i)$, define new $\mathbb{G}$ element γ and return it. $\hat{\alpha}[\gamma](\text{sid}, i) := 1$.
- Sample $H_p(i, \text{pw})$ uniformly at random for every $(i, \text{pw}) \in [N] \times \{0, 1\}^*$.
- Sample $H_x(i, j)$ uniformly at random for every $(i, j) \in [N] \times [t]$.
- On $(\text{LEAKVAL}, \text{sid})$, define new $y_{\text{sid}} \in \mathbb{G}$ and send it to $\mathcal{A}^*$. $\beta[y_{\text{sid}}](\text{sid}) := 1$.
- On group operation $\gamma_1 \cdot \gamma_2$, define new $\mathbb{G}$ element γ_3 and return it. $\alpha[\gamma_3] := \alpha[\gamma_1] + \alpha[\gamma_2]$, $\hat{\alpha}[\gamma_3] := \hat{\alpha}[\gamma_1] + \hat{\alpha}[\gamma_2]$, and $\beta[\gamma_3] := \beta[\gamma_1] + \beta[\gamma_2]$.

Whenever a new $\mathbb{G}$ element γ is defined (by $H_{\mathbb{G}}$ or $\hat{H}_{\mathbb{G}}$ query, LEAKVAL event, or group operation), the simulator checks whether its internal description is identical to any previous γ' ; if so, the same unique string is used as its representation. If not, the simulator proceeds to check whether the internal description of γ is *conditionally* equivalent to any previous γ' , conditioned on possible OWF inversions:

$$\begin{aligned} & \forall_{\text{sid} \text{ s.t. } \beta[\gamma](\text{sid}) = \beta[\gamma'](\text{sid})} \\ & \quad \forall_{i \in [N]} \quad \alpha[\gamma](\text{sid}, i) = \alpha[\gamma'](\text{sid}, i) \wedge \hat{\alpha}[\gamma](\text{sid}, i) = \hat{\alpha}[\gamma'](\text{sid}, i) \\ & \quad \wedge \\ & \quad \forall_{\text{sid} \text{ s.t. } \beta[\gamma](\text{sid}) \neq \beta[\gamma'](\text{sid})} \quad \exists_{\text{pw}, s} \\ & \quad \quad \forall_{i \in [N]} \quad \alpha[\gamma](\text{sid}, i) - \alpha[\gamma'](\text{sid}, i) = k_i(\text{pw}, s) \cdot (\beta[\gamma](\text{sid}) - \beta[\gamma'](\text{sid})) \\ & \quad \quad \wedge \\ & \quad \quad \hat{\alpha}[\gamma](\text{sid}, i) - \hat{\alpha}[\gamma'](\text{sid}, i) = H_p(i, \text{pw}) \cdot (\beta[\gamma](\text{sid}) - \beta[\gamma'](\text{sid})) \end{aligned} \quad (8)$$

If Condition 8 is true with some $(\text{sid}_1, \text{pw}_1, s_1), (\text{sid}_2, \text{pw}_2, s_2), \dots, (\text{sid}_n, \text{pw}_n, s_n)$ satisfying the second clause, then consider each of those as a possible OWF inversion. If all these inversions are correct, then $\gamma = \gamma'$; otherwise, $\gamma \neq \gamma'$. If there is insufficient information in X to determine whether $\gamma = \gamma'$, perform the relevant $(\text{ADVTEST}, \text{sid}, (\text{pw}, s))$ queries until receiving one “wrong” or receiving all “correct”. Record responses in X to avoid redundant queries in the future.

Fig. 10. SIM_{PoNR} : Simulator for f_{PoNR} , with differences from SIM_{Vand} in shadowed text

third event (i.e. $f_{\text{PoNR}} = f_{\text{PoNR}}^* \circ f_{\text{pre}}$ collision) is only possible in the case of f_{pre} collision. Under the assumption that at most $2^{(N \log p - \kappa)/2}$ H_p queries are made, the probability of such a collision is $< 2^{-\kappa}$ by a simple birthday bound.

B.3 Proof of Theorem 6

Proof. We follow the same proof structure as in the proof of Theorem 2, recalling that f_{Vand} and f_{PoNR} (and SIM_{Vand} and SIM_{PoNR}) differ only in their definitions of $k_i(\text{pw}, s)$ and by the presence of the $\hat{H}_{\mathbb{G}}$ bases. The proof of Lemma 2 does not depend upon the definition of $k_i(\text{pw}, s)$, nor is it affected by the presence of the $\hat{H}_{\mathbb{G}}$ bases; therefore its statement also applies to SIM_{PoNR} .

Lemma 5. *For every integer $c \geq 3$, if the ADVTEST queries of SIM_{PoNR} imply an attack graph $\mathcal{G}$ containing a cycle of length $\leq c$, then there exist distinct $(\text{pw}_1, s_1), \dots, (\text{pw}_c, s_c) \in \{0, 1\}^* \times \{0, 1\}^t$ such that $\text{rank}(K((\text{pw}_1, s_1), \dots, (\text{pw}_c, s_c))) < c$.*

In the case of f_{Vand} , the Vandermonde structure of matrix K made it easy to observe that it could not have rank $< c$ if $c \leq N$. For f_{PoNR} , K is not a Vandermonde matrix, but we will still argue that it is unlikely for a K of low rank to exist.

Lemma 6. *If $c - 1 + \frac{\kappa + c(t + \log q_H)}{\log(p-1) - \log(2c(t+1))} \leq N \leq 2c$, then the probability that there exist distinct $(\text{pw}_1, s_1), \dots, (\text{pw}_c, s_c) \in \{0, 1\}^t$ such that $\text{rank}(K((\text{pw}_1, s_1), \dots, (\text{pw}_c, s_c))) < c$ is at most $2^{-\kappa}$.*

Fix any distinct $(\text{pw}_1, s_1), \dots, (\text{pw}_c, s_c)$. For all $1 \leq i \leq N$, define K_i to be the submatrix of $K((\text{pw}_1, s_1), \dots, (\text{pw}_c, s_c))$ consisting of its first i rows. $0 \leq \text{rank}(K_i) \leq \min\{i, c\}$. Denote by LC_i the event that the i th row of K is some linear combination of its first $i-1$ rows. If $\text{rank}(K_i) = \nu$, then either $\text{rank}(K_{i-1}) = \nu - 1$ and LC_i does not occur, or $\text{rank}(K_{i-1}) = \nu$ and LC_i does occur. We therefore have the following recurrence relation for $1 \leq i \leq N$ and $0 \leq \nu \leq \min\{i, c-1\}$:

$$\begin{aligned} \Pr[\text{rank}(K_i) = \nu] &\leq \Pr[\text{rank}(K_{i-1}) = \nu - 1] \\ &\quad + \Pr[\text{rank}(K_{i-1}) = \nu] \cdot \Pr[LC_i \mid \text{rank}(K_{i-1}) = \nu] \end{aligned} \tag{9}$$

Conditioned on the event that $\text{rank}(K_{i-1}) = \nu < c$, we can define the square matrix S_i which consists of a basis of the row space of K_{i-1} augmented with the i th row of K , all truncated to just the first $\nu + 1$ columns. For event LC_i to occur, it is necessary (though not sufficient) that $\det(S_i) = 0$. We can therefore bound the probability of event LC_i . We abbreviate $H_p(i, \text{pw}_\ell)$ as $h_{pw_{i,\ell}}$, and we abbreviate $H_x(i, j)$ as $x_{i,j}$. All of the following probabilities are conditioned on

the event $\text{rank}(K_{i-1}) = \nu$, which for brevity we do not repeat on every line.

$$\begin{aligned}
& \Pr_{\substack{hpw_{1,1}, \dots, hpw_{1,c}, \\ x_{1,1}, \dots, x_{1,t} \leftarrow \mathbb{Z}_p^* \\ \dots \\ hpw_{i,1}, \dots, hpw_{i,c}, \\ x_{i,1}, \dots, x_{i,t} \leftarrow \mathbb{Z}_p^*}} [LC_i \mid \text{rank}(K_{i-1}) = \nu] \\
& \leq \Pr_{\substack{hpw_{1,1}, \dots, hpw_{1,c}, \\ x_{1,1}, \dots, x_{1,t} \leftarrow \mathbb{Z}_p^* \\ \dots \\ hpw_{i,1}, \dots, hpw_{i,c}, \\ x_{i,1}, \dots, x_{i,t} \leftarrow \mathbb{Z}_p^*}} [0 = \det(S_i)] \\
& = \Pr_{\substack{hpw_{1,1}, \dots, hpw_{1,c}, \\ x_{1,1}, \dots, x_{1,t} \leftarrow \mathbb{Z}_p^* \\ \dots \\ hpw_{i,1}, \dots, hpw_{i,c}, \\ x_{i,1}, \dots, x_{i,t} \leftarrow \mathbb{Z}_p^*}} \left[0 = \sum_{\ell \in [\nu+1]} C_{\nu+1, \ell}(S_i) \cdot k_i(\text{pw}_\ell, s_\ell) \right] \\
& = \Pr_{\substack{hpw_{1,1}, \dots, hpw_{1,c}, \\ x_{1,1}, \dots, x_{1,t} \leftarrow \mathbb{Z}_p^* \\ \dots \\ hpw_{i,1}, \dots, hpw_{i,c}, \\ x_{i,1}, \dots, x_{i,t} \leftarrow \mathbb{Z}_p^*}} \left[0 = \sum_{\ell \in [\nu+1]} C_{\nu+1, \ell}(S_i) \cdot hpw_{i, \ell} \cdot \prod_{\substack{j \in [t] \\ s_\ell[j]=1}} x_{i, j} \right] \\
& \leq \frac{t+1}{p-1}
\end{aligned}$$

At line 3 of the above, we use Laplace's formula for the determinant, denoting by $C_{\nu+1, \ell}(S_i)$ the $(\nu+1, \ell)$ cofactor of S_i . Observe that the variables $hpw_{i, \ell}, x_{i,1}, \dots, x_{i,t}$ do not participate in $C_{\nu+1, \ell}(S_i)$. We can therefore treat the equation in line 4 as some fixed polynomial evaluated on randomly sampled values for those $t+1$ variables. At the last line, we invoke the Schwartz-Zippel lemma [24], which states that a point randomly sampled over $\mathbb{F}' \subseteq \mathbb{F}$ on a non-zero polynomial of degree $\leq t+1$ over finite field $\mathbb{F}$ is zero with probability $\leq \frac{t+1}{|\mathbb{F}'|}$. The first ν rows of S_i are by definition linearly independent, which ensures that the cofactors are all non-zero. The fact that $(\text{pw}_1, s_1), \dots, (\text{pw}_c, s_c)$ are distinct therefore guarantees that the polynomial in question is non-zero.

Returning with this bound to Inequality 9, we have:

$$\Pr[\text{rank}(K_i) = \nu] \leq \Pr[\text{rank}(K_{i-1}) = \nu - 1] + \Pr[\text{rank}(K_{i-1}) = \nu] \cdot \frac{t+1}{p-1} \quad (10)$$

Along with the base case $\Pr[\text{rank}(K_0) = 0] = 1$, Inequality 10 is a slight generalization of Pascal's recurrence relation for binomial coefficients. It has the following solution (for all $0 \leq i \leq N$ and $0 \leq \nu \leq \min\{i, c-1\}$):

$$\Pr[\text{rank}(K_i) = \nu] \leq \binom{i}{\nu} \cdot \left(\frac{t+1}{p-1} \right)^{i-\nu} \quad (11)$$

Setting $i = N$, we can finally show that $K (= K_N)$ is unlikely to be of low rank.

$$\begin{aligned}
\Pr[\text{rank}(K) < c] &\leq \sum_{\nu=0}^{c-1} \binom{N}{\nu} \cdot \left(\frac{t+1}{p-1}\right)^{N-\nu} \\
&\leq \binom{N}{c} \cdot \left(\frac{t+1}{p-1}\right)^{N-c} \cdot \sum_{\nu=0}^{c-1} \binom{c}{\nu} \cdot \left(\frac{t+1}{p-1}\right)^{c-\nu} \\
&< N^{N-c} \cdot \left(\frac{t+1}{p-1}\right)^{N-c} \cdot \left[2 \cdot c \cdot \frac{t+1}{p-1}\right] \\
&= \frac{2c(t+1)}{p-1} \cdot \left(\frac{N(t+1)}{p-1}\right)^{N-c}
\end{aligned}$$

Even for $c = N$ (the largest c in consideration), the above probability is negligible. However, this bound only applies to some fixed $(\mathbf{pw}_1, s_1), \dots, (\mathbf{pw}_c, s_c)$. We use a union bound to show that the event is negligibly probable for any $(\mathbf{pw}_1, s_1), \dots, (\mathbf{pw}_c, s_c)$.

$$\begin{aligned}
&\Pr \left[\bigcup_{\substack{(\mathbf{pw}_1, s_1), \dots, (\mathbf{pw}_c, s_c) \in \{0,1\}^* \times \{0,1\}^t \\ \text{all distinct}}} \text{rank}(K((\mathbf{pw}_1, s_1), \dots, (\mathbf{pw}_c, s_c))) < c \right] \\
&\leq \sum_{\substack{(\mathbf{pw}_1, s_1), \dots, (\mathbf{pw}_c, s_c) \in \{0,1\}^* \times \{0,1\}^t \\ \text{all distinct}}} \Pr[\text{rank}(K((\mathbf{pw}_1, s_1), \dots, (\mathbf{pw}_c, s_c))) < c] \\
&< (q_H \cdot 2^t)^c \cdot \frac{2c(t+1)}{p-1} \cdot \left(\frac{N(t+1)}{p-1}\right)^{N-c} \tag{12}
\end{aligned}$$

To ensure this probability is negligible, we must set parameter N somewhat greater than c . Assuming as in the statement of Lemma 6 that $N \leq 2c$ (which we will see to be sufficient for reasonable parameters), and upper-bounding Expression 12 with $2^{-\kappa}$, we reach the result needed to complete the proof of Lemma 6.

$$N \geq c - 1 + \frac{\kappa + c(t + \log q_H)}{\log(p-1) - \log(2c(t+1))}$$

For example, if $\kappa = 128$, $t = 30$, $\log p = 256$, and $\log q_H = 128$, then this condition is approximated by $N \geq 1.66c$. Then, to ensure that the attack graph $\mathcal{G}$ contains no cycles of length c or less, one could set $N \approx 1.66c$.

To complete the proof of Theorem 6 we return to Theorems 3 and 4 of extremal graph theory, taking c to be the greatest integer satisfying $c - 1 + \frac{\kappa + c(t + \log q_H)}{\log(p-1) - \log(2c(t+1))} \leq N \leq 2c$. Theorem 3 implies that the number of edges in $\mathcal{G}$ is $\mathcal{O}(q_G^{1+1/\lfloor c/2 \rfloor})$, i.e. SIM_{PoNR} has rate $r = 1 + \frac{1}{\lfloor c/2 \rfloor}$. Setting $\delta = 4$, Theorem 4 implies that if $c \geq \log q_G$ then the number of edges in $\mathcal{G}$ is $\mathcal{O}(q_G)$, i.e. SIM_{PoNR} has rate $r = 1$ and f_{PoNR} is tight. Theorem 6 is proven.

For example, if $\log q_G = 128$ and other parameters are as above, then f_{PoNR} is tight for $N = 211$.

C Proof of Committed f_{Vand} and f_{PoNR} One-Wayness and Tightness

In this section we prove Lemma 4, i.e. that $\text{CommittedOwf}[f_{\text{Vand}}, \text{ElG}]$ and $\text{CommittedOwf}[f_{\text{PoNR}}, \text{ElG}]$ have the same one-wayness and tightness properties as f_{Vand} and f_{PoNR} , respectively.

Proof. We write the proof for f_{Vand} . Exactly the same argument applies to f_{PoNR} .

SIM_{Vand} can be modified to simulate $\text{CommittedOwf}[f_{\text{Vand}}, \text{ElG}]$. As defined in Appendix A, ElGamal commitment $\text{ElG}(m) := (g^{d_1}, g^{d_2}, m \cdot g^{d_1 d_2})$ where $d_1, d_2 \leftarrow_{\$} \mathbb{Z}_p$. On $(\text{EVAL}, \text{sid}, \text{P})$ from $\mathcal{F}_{\text{OWF}}$, SIM_{Vand} defines three new $\mathbb{G}$ elements $C_{\text{sid}} = (c_{\text{sid},1}, c_{\text{sid},2}, c_{\text{sid},3})$ and sends them to $\mathcal{A}^*$. SIM_{Vand} must incorporate $\mathbb{G}$ generator g and all the $c_{\text{sid},1}$, $c_{\text{sid},2}$, and $c_{\text{sid},3}$ bases in its internal descriptions of $\mathbb{G}$ elements. Condition 2 is augmented so that γ and γ' only constitute a OWF inversion test if the exponents of all of these bases are equal in their descriptions.

On event $(\text{LEAKVAL}, \text{sid})$, SIM_{Vand} samples $d_{\text{sid}} = (d_{\text{sid},1}, d_{\text{sid},2}) \leftarrow_{\$} \mathbb{Z}_p^2$ and sends it to $\mathcal{A}^*$ along with y_{sid} . The simulator internally defines $c_{\text{sid},1} = g^{d_{\text{sid},1}}$, $c_{\text{sid},2} = g^{d_{\text{sid},2}}$, $c_{\text{sid},3} = y_{\text{sid}} \cdot g^{d_{\text{sid},1} d_{\text{sid},2}}$ and updates its descriptions of all $\mathbb{G}$ elements accordingly. $c_{\text{sid},1}$, $c_{\text{sid},2}$, and $c_{\text{sid},3}$ are excluded from all future $\mathbb{G}$ element descriptions in favor of g and y_{sid} .

These changes to SIM_{Vand} introduce a new possibility of deviation from the real world, if SIM_{Vand} programs some $\gamma, \gamma' \in \mathbb{G}$ as if they are unequal and then a later choice of d_{sid} reveals that $\gamma = \gamma'$. Intuitively, an environment that causes such an event must be computing the discrete logarithm of $c_{\text{sid},1}$ or $c_{\text{sid},2}$. Accordingly, the classical GGM proof of Shoup [26] can be adapted to show that the event is negligibly probable. In short, this proof is based on the observation that any γ, γ' pair's probability of having been misprogrammed as unequal is at most $1/p$ over the random choice of d_{sid} . If at most $q_{\mathbb{G}}$ $\mathbb{G}$ elements are seen by the environment, then the total probability of a misprogramming is $< q_{\mathbb{G}}^2/p$.

More than one d_{sid} is sampled by SIM_{Vand} , which we could handle by simply multiplying the above bound by the number of $(\text{LEAKVAL}, \text{sid})$ events. This, however, is unnecessary. If a γ, γ' pair's equality is dependent on the choice of more than one d_{sid} , then their misprogramming is not determined until the last of those d_{sid} 's is sampled. In other words, every γ, γ' pair only contributes a $1/p$ probability of misprogramming once, and the standard $q_{\mathbb{G}}^2/p$ bound holds.

Thus, $\text{CommittedOwf}[f_{\text{Vand}}, \text{ElG}]$ realizes $\mathcal{F}_{\text{OWF}}$ with the same tightness as f_{Vand} .

D Syntax of a TFA-KE Scheme

In this section we further explain the TFA-KE functionality $\mathcal{F}_{\text{TFA-KE}}$ of Section 4 by mapping its syntax to a tuple of algorithms and discussing their intuitive properties.

A TFA-KE scheme is a tuple of algorithms $(\text{Init}, \text{AuthCodeGen}, \text{Auth}_C, \text{Auth}_S)$, to be executed by three parties denoted server S , user's client machine C , and

user's personal device D . The TFA-KE scheme is parametrized by integer t which sets the length of the one-time authentication codes produced by the device D .

Since we will model TFA-KE in the UC setting, the TFA-KE procedures below take additional identifiers as inputs. First, all procedures except for `AuthCodeGen` take as an additional input a *session identifier* sid , which denotes a unique identifier of some user's account on some server, the account to which this user authenticates via the TFA-KE scheme. Secondly, the `AuthC` and `AuthS` algorithms, which model respectively the client-side and the server-side online authentication algorithms, take as an additional input a *subsession identifier* ssid , which denotes a unique string identifying this online authentication instance, e.g. a binder to a TLS session on which the client and the server interact. Since the role of these identifiers is explained above, we mark these inputs in gray and omit them from the discussion below.

- `Init( $1^\kappa, t, \text{sid}, \text{pw}$ )`: The initialization algorithm takes the security parameter κ , code length parameter t , and a password pw , and outputs a device state st_D and a server state file.
- `AuthCodeGen( $\text{st}_D, T$ )`: The device algorithm takes as inputs the device state st_D and an epoch counter T , and outputs a one-time authenticator $\text{code} \in \{0, 1\}^t$. We assume that `AuthCodeGen` is a deterministic algorithm, and we denote the function it computes as $R_{\text{st}_D} : \mathbb{N} \rightarrow \{0, 1\}^t$.
- `AuthC( $\text{sid}, \text{ssid}, \text{pw}, \text{code}$ )` The client algorithm takes as inputs password pw , and code generated by `AuthCodeGen`. The algorithm outputs a key SK_C after an interaction with the server algorithm instance `AuthS`.
- `AuthS( $\text{sid}, \text{ssid}, \text{file}, T$ )`: The server algorithm takes as inputs the server state file and an epoch counter T . The algorithm outputs a key SK_S after an interaction with the client algorithm instance `AuthC`.

The scheme is correct if for every κ, t, pw, T , if $(\text{st}_D, \text{file}) \leftarrow \text{Init}(1^\kappa, t, \text{pw})$, $\text{code} \leftarrow \text{AuthCodeGen}(\text{st}_D, T)$, and algorithms `AuthC( $\text{pw}, \text{code}$ )` and `AuthS( $\text{file}, T$ )` interact without adversarial interference, then $\text{SK}_C = \text{SK}_S$.

Intuitively, a TFA-KE scheme is *online secure* if one can distinguish C 's or S 's outputs SK_C and SK_S from random only by authenticating under the correct password pw and the correct t -bit code $\text{code} = R_{\text{st}_D}(T)$ for epoch T assumed by the attacked party. To neutralize adversarial lunch-time attacks on the device, an authentication code revealed in epoch T , i.e. $\text{code} = R_{\text{st}_D}(T)$, should leak no information on $\text{code}' = R_{\text{st}_D}(T')$ for any $T' \neq T$, which implies that R_{st_D} must be a pseudorandom function. A TFA-KE scheme is *offline secure* if, given a user whose password pw is chosen at random from dictionary D , an attacker who compromises server S 's private state file but does not receive an authenticator code for any epoch T , can learn pw only via a brute-force search which takes time $\Theta(|D| * 2^t)$. Moreover, an attacker who learns both S 's state file and an authenticator one-time code for any epoch T , can recover pw only via a brute-force search in time $\Theta(|D|)$.

E Proof of TFA_{gen} Security

In this section we prove Theorem 7, i.e. that protocol TFA_{gen} realizes $\mathcal{F}_{\text{TFA-KE}}$.

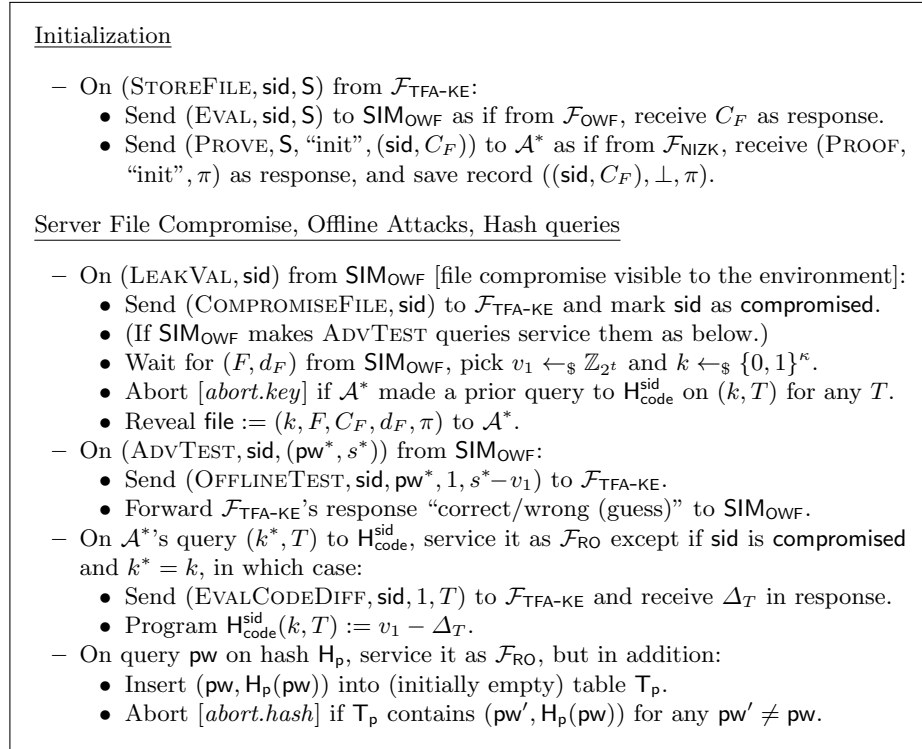


Fig. 11. $\text{SIM}_{\text{TFA-gen}}$: Simulator for protocol TFA_{gen} , part 1: Initialization, Server Corruption, Offline Attacks, Hash query processing

Proof. The simulator SIM that exhibits the claim that protocol TFA_{gen} realizes functionality $\mathcal{F}_{\text{TFA-KE}}$ is described in Figures 11 and 12. To handle the compromise of the server file, which reveals the tight OWF function value $F = f(\text{sid}, \text{hpw}_\Delta, s)$, and offline dictionary attacks that search for the preimage (hpw_Δ, s) given this value, simulator SIM uses as a sub-procedure a simulator SIM_{OWF} which exhibits the assumption that function $\text{CommittedOwf}[f, \text{COM}]$ (tightly) realizes the OWF functionality $\mathcal{F}_{\text{OWF}}$. Recall that an honest party that evaluates $\text{CommittedOwf}[f, \text{COM}]$ on argument x must output a commitment C_F to $F = f(x)$ to the environment. Consequently, simulator SIM_{OWF} must also produce such commitment whenever a real-world party evaluates this function. Since in our context this happens when S initializes the password file, hence SIM engages SIM_{OWF} also in the password file initialization.

Authentication

[Below sending command (RKEY, sid, ssid, P) to $\mathcal{F}_{\text{TFA-KE}}$ is a short-cut for sending commands (TEST, sid, ssid, P, $\perp$, $\perp$) and (NEWKEY, sid, ssid, P, $\perp$) to $\mathcal{F}_{\text{TFA-KE}}$.]

- On (CSESSION, sid, ssid, C, S) from $\mathcal{F}_{\text{TFA-KE}}$, service client session $\mathcal{C}^{\text{ssid}}$:
 1. Receive (C'_F, π') from $\mathcal{A}^*$ on behalf of $\mathcal{C}^{\text{ssid}}$.
 2. Set $\text{ckid} := \text{sid} \mid \text{ssid}$ and mark session $\mathcal{C}^{\text{ckid}}$ as fresh.
 3. Send (CEVAL, ckid, C, (sid, C'_F)) as if from $\mathcal{F}_{\text{CKEM}}$ to $\mathcal{A}^*$.
 4. If there is no record $((\text{sid}, C'_F), \cdot, \pi')$ then send (VERIFY, C, ckid, (sid, C'_F), π') as if from $\mathcal{F}_{\text{OWF}}$ to $\mathcal{A}^*$, receive (WITNESS, ckid, w) from $\mathcal{A}^*$ and:
 - (a) If $w = (\text{hpw}, s, d_{F'})$ s.t. $C'_F = \text{COM}(f^*(\text{sid}, \text{hpw}, s); d_{F'})$ then save record $((\text{sid}, C'_F), (\text{hpw}, s, d_{F'}), \pi')$;
 - (b) else skip steps 5,6 below and regardless of $\mathcal{A}^*$'s response on $\mathcal{F}_{\text{CKEM}}$ interface mark $\mathcal{C}^{\text{ssid}}$ used and send (RKEY, sid, ssid, C) to $\mathcal{F}_{\text{TFA-KE}}$.
 5. If $\mathcal{A}^*$ sends (COUT, ckid) as if to $\mathcal{F}_{\text{CKEM}}$ and $\exists$ session $\mathcal{S}^{\text{ckid}}$ marked completed then mark $\mathcal{C}^{\text{ckid}}$ used and:
 - (a) If $C'_F \neq C_F$ send (RKEY, sid, ssid, C) to $\mathcal{F}_{\text{TFA-KE}}$;
 - (b) else send (NEWKEY, sid, ssid, C, $\perp$) to $\mathcal{F}_{\text{TFA-KE}}$.
 6. If $\mathcal{A}^*$ sends (ADVSEVAL, ckid, y, SK*) as if to $\mathcal{F}_{\text{CKEM}}$ then mark $\mathcal{C}^{\text{ckid}}$ used, parse $y = (\text{sid}', F', s_S, d_{F'})$ (if $y = \perp$ then set all fields to $\perp$), and:
 - (a) If $(\text{sid}', C'_F) \neq (\text{sid}, \text{COM}(F'; d_{F'}))$ send (RKEY, sid, ssid, C) to $\mathcal{F}_{\text{TFA-KE}}$;
 - (b) elseif $C'_F = C_F$ and sid is marked compromised send (IMPERSONATE, sid, ssid, 1, $v_1 - s_S$), (NEWKEY, sid, ssid, C, SK*) to $\mathcal{F}_{\text{TFA-KE}}$;
 - (c) elseif $C'_F = C_F$ but sid is not compromised then Abort [*abort.cmt*];
 - (d) else retrieve record $((\text{sid}, C'_F), w, \pi')$, parse $w = (\text{hpw}, s, d_{F'})$, and:
 - i. If $\exists \text{pw}$ s.t. $(\text{pw}, \text{hpw}) \in \mathcal{T}_p$ then send (TEST, sid, ssid, C, pw, $s - s_S$), (NEWKEY, sid, ssid, C, SK*) to $\mathcal{F}_{\text{TFA-KE}}$;
 - ii. else send (RKEY, sid, ssid, C) to $\mathcal{F}_{\text{TFA-KE}}$.
- On (SSESSION, sid, ssid, S, C) from $\mathcal{F}_{\text{TFA-KE}}$, service server session $\mathcal{S}^{\text{ssid}}$:
 1. Send (C_F, π) to $\mathcal{A}^*$, set $\text{ckid} := \text{sid} \mid \text{ssid}$, mark session $\mathcal{S}^{\text{ckid}}$ as fresh.
 2. Send (SEVAL, ckid, C, (sid, C_F)) as if from $\mathcal{F}_{\text{CKEM}}$ to $\mathcal{A}^*$.
 3. If $\mathcal{A}^*$ sends (SOUT, ckid) as if to $\mathcal{F}_{\text{CKEM}}$ and $\exists$ session $\mathcal{C}^{\text{ckid}}$ marked fresh then mark $\mathcal{S}^{\text{ckid}}$ completed and send (NEWKEY, sid, ssid, S, $\perp$) to $\mathcal{F}_{\text{TFA-KE}}$.
 4. If $\mathcal{A}^*$ sends (ADVCEVAL, ckid, x) as if to $\mathcal{F}_{\text{CKEM}}$ then mark $\mathcal{S}^{\text{ckid}}$ used, parse $x = (\text{sid}', \text{hpw}, s_C)$ (if $x = \perp$ then set all fields to $\perp$), and:
 - (a) If $\text{sid}' \neq \text{sid}$ or $(\cdot, \text{hpw}) \notin \mathcal{T}_p$ then send (RKEY, sid, ssid, C) to $\mathcal{F}_{\text{TFA-KE}}$ and (COUT, ssid, $\perp$) as if from $\mathcal{F}_{\text{CKEM}}$ to $\mathcal{A}^*$;
 - (b) else let $(\text{pw}, \text{hpw}) \in \mathcal{T}_p$, send (TEST, sid, ssid, S, pw, s_C) to $\mathcal{F}_{\text{TFA-KE}}$, and:
 - i. if $\mathcal{F}_{\text{TFA-KE}}$ responds “correct guess” then pick $\text{SK} \leftarrow_{\$} \{0, 1\}^{\kappa}$ and send (NEWKEY, sid, ssid, S, SK) to $\mathcal{F}_{\text{TFA-KE}}$ and (COUT, ssid, SK) as if from $\mathcal{F}_{\text{CKEM}}$ to $\mathcal{A}^*$;
 - ii. else send (NEWKEY, sid, ssid, S, $\perp$) to $\mathcal{F}_{\text{TFA-KE}}$ and (COUT, ssid, $\perp$) as if from $\mathcal{F}_{\text{CKEM}}$ to $\mathcal{A}^*$.

Fig. 12. $\text{SIM}_{\text{TFA-gen}}$: Simulator for protocol TFA_{gen} , part 2: Authentication

Game hopping proof. Below we show a sequence of games that bridges between the environment’s view of an interaction with the real-world protocol TFA_{gen} and the environment’s view of an interaction with the UC TFA functionality $\mathcal{F}_{\text{TFA-KE}}$ which internally interacts with the simulator SIM shown in Figures 11 and 12. To simplify notation we treat the real-world adversary as a “dummy” sub-procedure of the environment $\mathcal{Z}$, hence we assume that $\mathcal{Z}$ interacts directly with all security games. We denote the probability that $\mathcal{Z}$ outputs 1 in the interaction with the real-world game or the simulated game as resp. $\text{Real}_{\mathcal{Z}}[\text{TFA}_{\text{gen}}]$ and $\text{Ideal}_{\mathcal{Z}}[\mathcal{F}_{\text{TFA-KE}}, \text{SIM}]$. We denote the probability that $\mathcal{Z}$ outputs 1 in an interaction with security game G_i as $\text{Exp}_{\mathcal{Z}}[G_i] = \Pr[1 \leftarrow \mathcal{Z}^{G_i}(1^\kappa)]$, and we define $\Delta(G_i, G_j) = |\Pr[\text{Exp}_{\mathcal{Z}}[G_i]] - \Pr[\text{Exp}_{\mathcal{Z}}[G_j]]|$. The sequence of games below starts with game G_0 s.t. $\text{Exp}_{\mathcal{Z}}[G_0] = \text{Real}_{\mathcal{Z}}[\text{TFA}_{\text{gen}}]$ and ends with game G_{13} s.t. $\text{Exp}_{\mathcal{Z}}[G_{13}] = \text{Ideal}_{\mathcal{Z}}[\mathcal{F}_{\text{TFA-KE}}, \text{SIM}]$, and the theorem follows if we show that $\Delta(G_0, G_{13})$ is a negligible function of the security parameter.

Throughout the proof we fix environment $\mathcal{Z}$ and we use q_{sid} for the number of password account initializations $\mathcal{Z}$ creates (via `STOREFILE` calls to some S parties), q_{ses} for the total number of C and S authentication sessions $\mathcal{Z}$ invokes, and q_{H} for the number of $\mathcal{Z}$ ’s hash oracle calls. Moreover, at some steps we reduce $\mathcal{Z}$ ’s distinguishing advantage to breaking the binding property of commitment COM , or the hiding property of COM , or the collision resistance of function f^* , or to distinguishing between the real-world evaluation of function f and its *simulation* as a realization of $\mathcal{F}_{\text{OWF}}$ via simulator SIM_{OWF} . The reductions which convert $\mathcal{Z}$ ’s distinguishing advantage to an attack against these properties are only a small overlay over $\mathcal{Z}$, so for notational convenience we denote the adversarial advantage of these reductions against these notions as if the corresponding reduction is equated with $\mathcal{Z}$. We thus denote these reduction advantages as resp. $\text{Adv}_{\text{COM}}^{\text{bind}}[\mathcal{Z}]$, $\text{Adv}_{\text{COM}}^{\text{hid}}[\mathcal{Z}]$, $\text{Adv}_{f^*}^{\text{crh}}[\mathcal{Z}]$, and $\text{Adv}_{f, \text{COM}}^{\mathcal{F}_{\text{OWF}}}[\mathcal{Z}, \text{SIM}_{\text{OWF}}]$, all of them functions of the security parameter. (The last advantage stands for a distinguishing advantage of $\mathcal{Z}$ between a real-world view of an honest party evaluating the one-way function $\text{CommittedOwf}[f, \text{COM}]$ and the view created by the simulator SIM_{OWF} interacting with the ideal one-way function model $\mathcal{F}_{\text{OWF}}$.)

Game G_0 (real world with inscribed helper functionalities): We start with a game G_0 which models the real-world execution $\text{Real}_{\mathcal{Z}}[\text{TFA}_{\text{gen}}]$, but with inscribed functionalities $\mathcal{F}_{\text{RO}}$, $\mathcal{F}_{\text{NIZK}}$, and $\mathcal{F}_{\text{CKEM}}$. The result is a game, shown in Figure 13, where $\mathcal{Z}$ can access oracles Init^{sid} , $\text{O}_{\text{compr}}^{\text{sid}}$, $\text{D}_{\text{code}}^{\text{sid}}$, $\text{H}^{\text{[sid]}}$, C_1^{ckid} , C_2^{ckid} , and S^{ckid} , corresponding resp. to an initialization call `STOREFILE` to S , a password file compromise call `COMPROMISEFILE` to S , a code retrieval from the device via call `READCODE` to D , calls to hash functions H_{code} , H_{p} , and calls to client and server oracles where C_1^{ckid} , C_2^{ckid} , S^{ckid} model resp. the first and second round of C ’s code and the single round of S . We index these oracles by $\text{ckid} = (\text{sid}, \text{ssid})$ which defines all the relevant identifiers on C and S sessions. The last oracle, O_{group} , models $\mathcal{Z}$ ’s operations in a group which is modeled as a generic group in the proof that $\text{CommittedOwf}[f, \text{COM}]$ realizes $\mathcal{F}_{\text{OWF}}$.

The real-world experiment in Figure 13 makes some notational simplifications. When $\mathcal{Z}$ calls Oracles S^{ckid} , C_1^{ckid} , C_2^{ckid} , which model S and C sessions, it

$\text{Init}^{\text{sid}}(\text{pw}_\Delta[+\pi])$, i.e. $(\text{STOREFILE}, \text{sid}, \text{pw}_\Delta)$ to $\mathcal{S}$ and $(\text{PROOF}, \text{"init"}, \pi)$ to $\mathcal{F}_{\text{NIZK}}$:

1. $s \leftarrow_{\$} \mathbb{Z}_{2^t}, k \leftarrow_{\$} \{0, 1\}^\kappa, \text{hpw}_\Delta \leftarrow H_p(\text{pw}_\Delta), F \leftarrow f^*(\text{sid}, \text{hpw}_\Delta, s)$
2. $(C_F, d_F) \leftarrow \text{COM}(F)$, return C_F to $\mathcal{Z}$ [$\mathcal{Z}$ learns C_F from $\mathcal{F}_{\text{NIZK}}$ interface]
3. receive π from $\mathcal{Z}$ and save $((\text{sid}, C_F), \perp, \pi)$

$\text{O}_{\text{compr}}^{\text{sid}}$, i.e. $(\text{COMPROMISEFILE}, \text{sid})$ to $\mathcal{S}$:

- return (k, F, d_F) [values C_F, π are already known]

$H_{\text{name}}^{\text{sid}}(x)$ for $\text{name} \in \{\text{code}, p\}$, i.e. query to hash functions $H_{\text{code}}^{\text{sid}}$ and H_p :
 (same procedure is followed whenever H_{code} or H_p is called by another oracle)

1. if $H_{\text{name}}^{\text{sid}}(x) \neq \perp$ then return $H_{\text{name}}^{\text{sid}}(x)$ [initially $H_{\text{name}}^{\text{sid}}(x) = \perp$ for all x]
2. else sample y from $H_{\text{name}}^{\text{sid}}$ range and set (and return) $H_{\text{name}}^{\text{sid}}(x) := y$

$\text{D}_{\text{code}}^{\text{sid}}(T)$, i.e. query $(\text{READCODE}, \text{sid}, T)$ to $\mathcal{D}$:

- return $\text{code} = s - H_{\text{code}}^{\text{sid}}(k, T)$

$\text{C}_1^{\text{ckid}}(\text{pw}_C, \text{code}, C'_F, \pi')$ for $\text{ckid} = (\text{sid}, \text{ssid})$:
 i.e. command $(\text{CSESSION}, \dots, \text{pw}_C, \text{code})$ to $\mathcal{C}$ and a network message (C'_F, π') to $\mathcal{C}$

- set $\text{hpw}_C = H_p(\text{pw}_C)$, mark C^{ckid} as fresh, save $(\text{hpw}_C, \text{code}, C'_F, \pi')$

$\text{S}^{\text{ckid}}(b, T, x)$ for $\text{ckid} = (\text{sid}, \text{ssid})$:
 i.e. $(\text{SSSESSION}, \dots, T)$ and $(\text{SOUT}, \text{ckid})$ [$b = 0$] or $(\text{ADVCEVAL}, \text{ckid}, x)$ [$b = 1$]:

1. if $b = 0$ and C^{ckid} is not marked fresh then ignore this query
2. if $b = 0$: set $\text{SK}_S \leftarrow \{0, 1\}^\kappa$, mark S^{ckid} as **completed** (T, SK_S) , return SK_S
3. if $b = 1$: set $\text{SK}_S \leftarrow \{0, 1\}^\kappa$, mark S^{ckid} as **used**, and if $x = (\text{sid}, \text{hpw}, s_C)$ s.t.
 $F = f(\text{sid}, \text{hpw}, s_C + H_{\text{code}}^{\text{sid}}(k, T))$ then return $(\text{SK}_S, \text{SK}_S)$ else return $(\perp, \text{SK}_S)$

$\text{C}_2^{\text{ckid}}(b, y, \text{SK}^*, w)$, i.e. $(\text{COUT}, \text{ckid})$ [$b = 0$] or $(\text{ADVSEVAL}(\text{ckid}, y, \text{SK}^*))$ [$b = 1$]
 followed by $(\text{WITNESS}, w)$ to $\mathcal{F}_{\text{NIZK}}$:

1. if $b = 0$ and S^{ckid} is marked **used** then ignore this query else mark C^{ckid} as **used**
2. if $\exists$ no record $((\text{sid}, C'_F), \cdot, \pi')$ and $\mathcal{Z}$ sends $(\text{WITNESS}, w)$ s.t.
 $w = (\text{hpw}', s', d_F')$ and $C'_F = \text{COM}(f^*(\text{sid}, \text{hpw}', s'); d_F')$ then save
 $((\text{sid}, C'_F), w, \pi')$
3. if $\exists$ (still) no record $((\text{sid}, C'_F), \cdot, \pi')$ return $\text{SK}_C \leftarrow_{\$} \{0, 1\}^\kappa$
4. elseif $b = 0$ and S^{ckid} is marked **completed** (T, SK_S) :
 - (a) if $C'_F = C_F$ and $F = f^*(\text{sid}, \text{hpw}_C, \text{code} + H_{\text{code}}^{\text{sid}}(k, T))$ return $\text{SK}_C := \text{SK}_S$
 - (b) else return $\text{SK}_C \leftarrow_{\$} \{0, 1\}^\kappa$
5. elseif $b = 1$ then:
 - (a) if $y = (\text{sid}, F', s_S, d_F')$ s.t. $C'_F = \text{COM}(F'; d_F')$ and $F' = f^*(\text{sid}, \text{hpw}_C, \text{code} + s_S)$ and return $\text{SK}_C = \text{SK}^*$
 - (b) else return $\text{SK}_C \leftarrow_{\$} \{0, 1\}^\kappa$

$\text{O}_{\text{group}}(\cdot)$, i.e. adversary's group operations:

- adversarial group operations are handled by a generic group interface

Fig. 13. Game $\text{Real}_{\mathcal{Z}}[\text{TFA}_{\text{gen}}]$: the real world with inscribed code of $\mathcal{F}_{\text{RO}}, \mathcal{F}_{\text{NIZK}}, \mathcal{F}_{\text{CKEM}}$

receives some messages which we omit because they return information which is already known to $\mathcal{Z}$: (1) we omit message (C_F, π) from $\mathbf{S}$ which should be triggered by a call to $\mathbf{S}^{\text{ckid}}$, because both values are already known to $\mathcal{Z}$ from an interaction with Init^{sid} , and for the same reason we combine $\mathcal{Z}$'s call $(\text{SSession}, \dots)$ to $\mathbf{S}$ with calls $(\text{Sout}, \text{ckid})$ or $(\text{ADVCEVAL}, \text{ckid}, \dots)$ to $\mathcal{F}_{\text{CKEM}}$, since the first one leaks no new information to $\mathcal{Z}$; (2) we omit messages $(\text{CEVAL}, \text{ckid}, \mathbf{C}, (\text{sid}, C'_F))$ and $(\text{VERIFY}, \mathbf{C}, \text{ckid}, (\text{sid}, C'_F), \pi')$ which the adversary would see as messages from resp. $\mathcal{F}_{\text{CKEM}}$ and $\mathcal{F}_{\text{NIZK}}$ in response to a call to $\mathbf{C}_2^{\text{ckid}}$, because all values these messages carry are specified in $\mathcal{Z}$'s call to $\mathbf{C}_1^{\text{ckid}}$. Finally, we note that oracle $\mathbf{S}^{\text{ckid}}$ in Figure 13 return a pair of values, e.g. $(\text{SK}_S, \text{SK}_S)$ or $(\perp, \text{SK}_S)$, in response to a call $(\text{ADVCEVAL}, \text{ckid}, \dots)$ to $\mathcal{F}_{\text{CKEM}}$, because the first element stands for a value SK returned by the $\mathcal{F}_{\text{CKEM}}$ interface in message $(\text{Cout}, \text{ssid}, \text{SK})$, which can be either $\text{SK} = \text{SK}_S$ or $\text{SK} = \perp$, while the second element stands for the $\mathbf{S}$'s output which is passed to the environment. We conclude that:

$$\text{Exp}_{\mathcal{Z}}[\mathbf{G}_0] = \text{Real}_{\mathcal{Z}}[\text{TFA}_{\text{gen}}] \quad (13)$$

Game $\mathbf{G}_1$ (move defining k to the first query to $\mathbf{H}_{\text{code}}^{\text{sid}}$): We modify the game so that key k is not chosen in the call to Init^{sid} but in the first query to $\mathbf{H}_{\text{code}}^{\text{sid}}$, and then Init^{sid} re-uses this key k . This change will be useful in the next game change but it purely notational, and therefore:

$$\Delta_{\mathcal{Z}}(\mathbf{G}_0, \mathbf{G}_1) = 0 \quad (14)$$

Game $\mathbf{G}_2$ (program $\mathbf{H}_{\text{code}}^{\text{sid}}$ using a random function $\mathbf{R}^{\text{sid}}$): We modify the game so that whenever $\mathbf{H}_{\text{code}}^{\text{sid}}(k, \cdot)$ is called (for k output by the initialization), either directly via $\mathcal{Z}$'s call to $\mathbf{H}_{\text{code}}^{\text{sid}}$ or indirectly by oracles $\mathbf{D}_{\text{code}}^{\text{sid}}$, $\mathbf{S}^{\text{ckid}}$, and $\mathbf{C}_2^{\text{ckid}}$, on an argument for which this value is undefined, the game programs its value using a random function $\mathbf{R}^{\text{sid}} : \{0, 1\}^* \rightarrow \{0, 1\}^t$. Specifically, we set $\mathbf{H}_{\text{code}}^{\text{sid}}(k, T) = s - \mathbf{R}^{\text{sid}}(T)$ for all T , where k is the unique PRF key chosen for this sid by Init^{sid} . (Note that by the rules of $\mathbf{G}_1$ oracle $\mathbf{H}_{\text{code}}^{\text{sid}}$ can act differently on input $(k, \cdot)$ for k which will be output by the initialization even before $\mathcal{Z}$ calls Init^{sid} .) Since both $\mathbf{H}_{\text{code}}^{\text{sid}}(k, \cdot)$ and $\mathbf{R}^{\text{sid}}(\cdot)$ are random functions on the same range, this is an internal game re-write which makes no difference to an entity that interacts with it, hence:

$$\Delta_{\mathcal{Z}}(\mathbf{G}_1, \mathbf{G}_2) = 0 \quad (15)$$

Game $\mathbf{G}_3$ (using function $\mathbf{R}^{\text{sid}}$ instead of $\mathbf{H}_{\text{code}}^{\text{sid}}$, defining s indirectly): We make the following modifications to $\mathbf{G}_2$:

1. Oracle Init^{sid} doesn't choose s , and instead chooses $v_1 \leftarrow_{\$} \mathbb{Z}_{2^t}$, and sets $F \leftarrow f^*(\text{sid}, \text{hpw}_{\Delta}, v_1 + \mathbf{R}^{\text{sid}}(1))$, which implicitly defines s as $s = v_1 + \mathbf{R}^{\text{sid}}(1)$.
2. Oracle $\mathbf{H}_{\text{code}}^{\text{sid}}$ on $x = (k, T)$ for any T , returns $(v_1 + \mathbf{R}^{\text{sid}}(1)) - \mathbf{R}^{\text{sid}}(T)$.
3. Oracle $\mathbf{D}_{\text{code}}^{\text{sid}}(T)$ returns $\mathbf{R}^{\text{sid}}(T)$.

4. Oracle S^{ckid} for $b = 1$ changes condition $F = f^*(\text{sid}, \text{hpw}, s_C + H_{\text{code}}^{\text{sid}}(k, T))$ to condition $F = f^*(\text{sid}, \text{hpw}, (v_1 + R^{\text{sid}}(1)) + (s_C - R^{\text{sid}}(T)))$.
5. Oracle C_2^{ckid} for $b = 0$ changes condition $F = f^*(\text{sid}, \text{hpw}_C, \text{code} + H_{\text{code}}^{\text{sid}}(k, T))$ to condition $F = f^*(\text{sid}, \text{hpw}_C, (v_1 + R^{\text{sid}}(1)) + (\text{code} - R^{\text{sid}}(T)))$

The first change is only a renaming of variables: Instead of choosing s in $\mathbb{Z}_{2^t}$ directly, we sample random v_1 in $\mathbb{Z}_{2^t}$ and define s as $s = v_1 + R^{\text{sid}}(1)$. The second change follows because $H_{\text{code}}^{\text{sid}}(k, T) = s - R^{\text{sid}}(T) = (v_1 + R^{\text{sid}}(1)) - R^{\text{sid}}(T)$. The third change follows because if $H_{\text{code}}^{\text{sid}}(k, T) = s - R^{\text{sid}}(T)$ then $D_{\text{code}}^{\text{sid}}(T) = s - H_{\text{code}}^{\text{sid}}(k, T) = s - (s - R^{\text{sid}}(T)) = R^{\text{sid}}(T)$. The fourth change follows because if $H_{\text{code}}^{\text{sid}}(k, T) = s - R^{\text{sid}}(T)$ and $s = v_1 + R^{\text{sid}}(1)$ then $s_C + H_{\text{code}}^{\text{sid}}(k, T) = s_C + (s - R^{\text{sid}}(T)) = (v_1 + R^{\text{sid}}(1)) + (s_C - R^{\text{sid}}(T))$. Replacing s_C with code in these equations also shows that the fifth change is also only a re-write. Since all these changes are only notational we conclude:

$$\Delta_{\mathcal{Z}}(\mathbf{G}_2, \mathbf{G}_3) = 0 \quad (16)$$

Game $\mathbf{G}_4$ (postpone choice of k to file compromise): We modify the game so that (1) key k is not picked by the first call to $H_{\text{code}}^{\text{sid}}$ but at $O_{\text{compr}}^{\text{sid}}$, (2) oracle $H_{\text{code}}^{\text{sid}}$ does not program $H_{\text{code}}^{\text{sid}}$ outputs on $x = (k, \cdot)$ before password file compromise (because k is not yet defined then), but performs this programming on $(k, \cdot)$ queries after $O_{\text{compr}}^{\text{sid}}$ picks and reveals k , and (3) we introduce an abort, denoted $[\text{abort.key}]$ to $O_{\text{compr}}^{\text{sid}}$ if it picks $k \leftarrow_{\$} \{0, 1\}^{\kappa}$ s.t. $\mathcal{Z}$ made a prior query of the form $x = (k, \cdot)$, i.e. if $H_{\text{code}}^{\text{sid}}(k, T) \neq \perp$ for any T .

Game $\mathbf{G}_3$ and $\mathbf{G}_4$ are identical conditioned on the event that $\mathcal{Z}$ does not query $O_{\text{compr}}^{\text{sid}}$ on an argument of the form $x = (k, \cdot)$ before $\mathcal{Z}$ queries $O_{\text{compr}}^{\text{sid}}$. Indeed, in $\mathbf{G}_3$ the only oracle which uses k apart of $O_{\text{compr}}^{\text{sid}}$ is $H_{\text{code}}^{\text{sid}}$, so if $\mathcal{Z}$ doesn't make a query of the form $(k, \cdot)$ to $H_{\text{code}}^{\text{sid}}$ before it queries $O_{\text{compr}}^{\text{sid}}$ then the two games are identical. Since k is randomly sampled in $\{0, 1\}^{\kappa}$, the probability that $\mathcal{Z}$ does make such query to $H_{\text{code}}^{\text{sid}}$ before a query to $O_{\text{compr}}^{\text{sid}}$ is bounded by $q_H/2^{\kappa}$. Since the full bound is that this does not happen for any sid , we conclude:

$$\Delta_{\mathcal{Z}}(\mathbf{G}_3, \mathbf{G}_4) \leq (q_{\text{sid}} \cdot q_H)/2^{\kappa} \quad (17)$$

Game $\mathbf{G}_5$ (abort on attack on $\mathbf{C}$ for $C'_F = C_F$ but $F' \neq F$): We modify the game to introduce an abort, denoted $[\text{abort.cmt1}]$, in the processing of oracle C_2^{ckid} in case $b = 1$, i.e. an active attack on $\mathbf{C}$'s CKEM session, if it holds that $C'_F = C_F$, and $y = (\text{sid}, F', s_S, d_{F'})$ s.t. $C'_F = \text{COM}(F'; d_{F'})$, but $F' \neq F$. The difference between $\mathbf{G}_4$ and $\mathbf{G}_5$ is upper-bounded by the probability that the above condition holds. If we denote the event that this happens on the i -th call to C_2^{ckid} by E_i , then $\Pr_{\mathcal{Z}}[E_i]$ is upper-bounded by $\text{Adv}_{\text{bind}}^{\text{COM}}[\mathcal{Z}]$, because an environment $\mathcal{Z}$ which causes event E_i allows for an immediate reduction against the binding property of commitment COM . Since the difference between $\mathbf{G}_4$ and $\mathbf{G}_5$ is upper-bounded by $\Pr[\cup_{i=1}^{q_{\text{ses}}} E_i]$ we conclude:

$$\Delta_{\mathcal{Z}}(\mathbf{G}_4, \mathbf{G}_5) \leq q_{\text{ses}} \cdot \text{Adv}_{\text{COM}}^{\text{bind}}[\mathcal{Z}] \quad (18)$$

Game G_6 (abort on attack on C for $C'_F = C_F$ before file compromise): We modify the game to introduce an abort, denoted $[abort.cmt2]$, in the processing of oracle C_2^{ckid} in the same case $b = 1$ as above, if it holds that $C'_F = C_F$, and $y = (sid, F', s_S, d_{F'})$ s.t. $C'_F = COM(F'; d_{F'})$, and the file compromise oracle O_{compr}^{sid} has not been queried yet.

Note that the above conditions imply that on some call to C_2^{ckid} , $\mathcal{Z}$ provides $(F', d_{F'})$ s.t. $C_F = COM(F'; d_{F'})$. Moreover, by G_5 this introduces a difference only in the case that $F' = F$. Let's denote the event that this happens on i -th call to C_2^{ckid} by E_i , and let $\epsilon = \Pr_{\mathcal{Z}}[E_i]$. Extending $\mathcal{Z}$ by the code of $Init^{sid}$ which computes F , note that $\mathcal{Z}$ receives $C_F \leftarrow_{\$} COM(F)$, and with probability ϵ creates $d_{F'}$ s.t. $C_F = COM(F; d_{F'})$. Such $\mathcal{Z}$ implies a reduction against the hiding property of this commitment: The reduction runs $\mathcal{Z}$ and outputs two messages, $m_0 = F$ and $m_1 \neq F$, gets a challenge $C^* \leftarrow_{\$} COM(m_b)$, and sends it to $\mathcal{Z}$ in response to $Init^{sid}$, and if $\mathcal{Z}$ outputs d on the i -th C_2^{ckid} session s.t. $C^* = COM(F; d)$ the reduction outputs 1. Since for $b = 0$ this probability is ϵ , and for $b = 1$ the probability that $C^* = COM(F; d)$ is upper-bounded by $Adv_{COM}^{bind}[\mathcal{Z}]$, we conclude that $\epsilon \leq Adv_{COM}^{hid}[\mathcal{Z}] + Adv_{COM}^{bind}[\mathcal{Z}]$. Since the difference between G_5 and G_6 is upper-bounded by $\Pr[\cup_{i=1}^{q_{ses}} E_i]$ we conclude:

$$\Delta_{\mathcal{Z}}(G_5, G_6) \leq q_{ses} \cdot (Adv_{COM}^{hid}[\mathcal{Z}] + Adv_{COM}^{bind}[\mathcal{Z}]) \quad (19)$$

Game G_7 (using f^* inputs in equality checks): We modify the game so that whenever S^{ckid} or C_2^{ckid} performs a check of the form $Y \equiv f^*(x')$ for some Y for which the game knows x s.t. $Y = f^*(x)$, we replace each such check by the check that $x' = x$. Specifically:

1. In S^{ckid} , in case $b = 1$, game G_7 replaces the check if $F = f^*(sid, hpw, (v_1 + R^{sid}(1)) + (s_C - R^{sid}(T)))$ with a check if $hpw_{\Delta} = hpw$ and $s_C = R^{sid}(T)$.
2. In C_2^{ckid} , in case $b = 0$, game G_7 replaces the check if $F = f^*(sid, hpw_C, (v_1 + R^{sid}(1)) + (code - R^{sid}(T)))$ with a check if $hpw_{\Delta} = hpw_C$ and $code = R^{sid}(T)$.
3. In C_2^{ckid} , in case $b = 1$ and $C'_F = C_F$, and there is no abort, which by G_5, G_6 implies that $F' = F$ and the password file is compromised, game G_7 replaces the check if $F = f^*(sid, hpw_C, code + s_S)$ with a check if $hpw_{\Delta} = hpw_C$ and $code + s_S = v_1 + R^{sid}(1)$.
4. In C_2^{ckid} , in case $b = 1$ and $C'_F \neq C_F$, game G_7 replaces the check if $F' = f^*(sid, hpw_C, code + s_S)$ with a check if $hpw' = hpw_C$ and $s' = code + s_S$, for hpw', s' retrieved from the witness $w = (sid, hpw', s')$.

The first three changes are based on the fact that in G_6 $F = f^*(sid, hpw_{\Delta}, v_1 + R(1))$, and the last change is based on the fact that if G_6 admits w sent by $\mathcal{Z}$ in a query to C_2^{ckid} oracle then $F' = f^*(sid, hpw', s')$.

The only divergence G_7 creates from G_6 is in the case where some condition of the form $F = f^*(x')$ holds for $F = f^*(x)$, but it does not hold that $x = x'$. However, if $\mathcal{Z}$ ever triggers such case then it also breaks the collision-resistance property of function f^* for the given sid . Since G_6 and G_7 are identical unless $\mathcal{Z}$ finds a collision in f^* , we conclude:

$$\Delta_{\mathcal{Z}}(G_6, G_7) \leq Adv_{f^*}^{crh}[\mathcal{Z}] \quad (20)$$

Game G_8 (abort on H_p collision): We modify the game by introducing an abort in H_p processing, denoted $[abort.hash]$, which is triggered when there occurs a collision in hash H_p , i.e. if for a new argument pw and a fresh sampled value $H_p(pw)$ there exists a prior query $pw' \neq pw$ s.t. $H_p(pw) = H_p(pw')$. Since H_p outputs are sampled in range $\mathcal{HP}$, by the birthday bound we have:

$$\Delta_Z(G_7, G_8) \leq (q_H)^2 / |\mathcal{HP}| \quad (21)$$

Game G_9 (using H_p inputs in tests): We modify the game as follows:

1. In C_2^{kid} , if $b = 0$ or if ($b = 1$, $C'_F = C_F$, and the password file is compromised), we replace the check $hpw_\Delta = hpw_C$ (see G_7 , case 2 and 3) with the check that $pw_\Delta = pw_C$.
2. In C_2^{kid} , if $b = 1$ and $C'_F \neq C_F$, we replace the check $hpw' = hpw_C$ (see G_7 , case 4), with the check that $\mathcal{Z}$ queried H_p on pw_C and $hpw' = H_p(pw_C)$.
3. Oracle C_1^{kid} skips computing hpw_C and saves pw_C instead.
4. Before server compromise, oracle S^{kid} replaces check that $hpw = hpw_\Delta$ (see G_7 , case 1), with the check that $\mathcal{Z}$ queried H_p on pw_Δ and $hpw = H_p(pw_\Delta)$.

By the changes in G_8 , the first modification does not change the external behavior of the game, because password hash outputs can match only if the inputs match, and otherwise game G_8 aborts. Regarding the second modification, if the environment does not compute hpw_C as $H_p(pw_C)$ then the only information the environment sees about hpw_C is via the $\mathcal{F}_{CKEM}$ interface, which allows only for testing whether a provided guess is equal to hpw_C . Therefore, distinguishing this modification requires guessing hpw_C on some session without computing it as $H_p(pw_C)$, which happens with probability at most $q_{ses} / |\mathcal{HP}|$. The third modification is not externally observable once the first two modifications take place. Finally, until password file compromise the only information about hpw_Δ which the environment sees without computing $H_p(pw_\Delta)$ is in commitment C_F , and via the $\mathcal{F}_{CKEM}$ interface which allows for testing any guessed value. By commitment hiding, if the probability that the environment guesses hpw_Δ given the commitment C_F to a value which is a function of hpw_Δ is ϵ , then the same guess probability without this commitment must be at least $\epsilon - \text{Adv}_{COM}^{hid}[\mathcal{Z}]$, but since the latter probability is upper-bounded by $q_{ses} / |\mathcal{HP}|$ it follows that $\epsilon \leq q_{ses} / |\mathcal{HP}| + \text{Adv}_{COM}^{hid}[\mathcal{Z}]$. The total distinguishing difference is therefore:

$$\Delta_Z(G_8, G_9) \leq 2 \cdot q_H / |\mathcal{HP}| + \text{Adv}_{COM}^{hid}[\mathcal{Z}] \quad (22)$$

Game G_{10} (using OWF simulator): Game G_9 looks very similar to the ideal-world game except for the following aspects:

1. At initialization query, Init^{sid} picks random v_1 and evaluates $\text{CommittedOwf}[f, COM]$ on input $x = (sid, pw_\Delta, v_1 + R^{sid}(1))$ as $F = f(x)$, which involves computing $hpw_\Delta = H_p(pw_\Delta)$, and $(C_F, d_F) \leftarrow_{\$} COM(F)$, and reveals C_F .
2. On file compromise query, O_{compr}^{sid} reveals F, d_F created in the initialization.

$\text{Init}^{\text{sid}}(\text{pw}_\Delta[+\pi])$, i.e. ($\text{STOREFILE}, \text{sid}, \text{pw}_\Delta$) to $\mathcal{S}$ and ($\text{PROOF}, \text{"init"}, \pi$) to $\mathcal{F}_{\text{NIZK}}$:

1. set C_F to output of SIM_{OWF} on $(\text{EVAL}, \text{sid}, \mathcal{S})$
2. return C_F to $\mathcal{Z}$ [$\mathcal{Z}$ learns C_F from $\mathcal{F}_{\text{NIZK}}$ interface]
3. receive π from $\mathcal{Z}$ and save $((\text{sid}, C_F), \perp, \pi)$

$\text{O}_{\text{compr}}^{\text{sid}}$, i.e. ($\text{COMPROMISEFILE}, \text{sid}$) to $\mathcal{S}$:

1. issue permission to SIM_{OWF} to output ($\text{LEAKVAL}, \text{sid}$)
2. (service SIM_{OWF} queries generated by O_{group} oracle as below)
3. wait for SIM_{OWF} to output (F, d_F)
4. set $v_1 \leftarrow_{\mathcal{S}} \mathbb{Z}_{2^t}, k \leftarrow_{\mathcal{S}} \{0, 1\}^\kappa$, mark sid as **compromised**
5. abort [abort.key] if $\text{H}_{\text{code}}^{\text{sid}}(k, T)$ has been queried before for any T
6. return (k, F, d_F) [values C_F, π are already known]

$\text{H}_{\text{name}}^{\text{sid}}(x)$ for $\text{name} \in \{\text{code}, \text{p}\}$, i.e. query to hash functions $\text{H}_{\text{code}}^{\text{sid}}$ and H_p :

1. if $\text{H}_{\text{name}}^{\text{sid}}(x) \neq \perp$ then return $\text{H}_{\text{name}}^{\text{sid}}(x)$ [initially $\text{H}_{\text{name}}^{\text{sid}}(x) = \perp$ for all x]
2. elseif $\text{name} = \text{code}$, sid is **compromised**, and $x = (k, T)$ for some T , then set (and return) $\text{H}_{\text{code}}^{\text{sid}}(k, T) := v_1 + \text{R}^{\text{sid}}(1) - \text{R}^{\text{sid}}(T)$. [R^{sid} is a random function]
3. else sample y from $\text{H}_{\text{name}}^{\text{sid}}$ range and set (and return) $\text{H}_{\text{name}}^{\text{sid}}(x) := y$, but if $\text{name} = \text{p}$ and H_p encounters a collision then abort [abort.hash]

$\text{D}_{\text{code}}^{\text{sid}}(T)$, i.e. query ($\text{READCODE}, \text{sid}, T$) to $\mathcal{D}$:

- return $\text{R}^{\text{sid}}(T)$

$\text{C}_1^{\text{ckid}}(\text{pw}_C, \text{code}, C'_F, \pi')$ for $\text{ckid} = (\text{sid}, \text{ssid})$:

i.e. command ($\text{CSESSION}, \dots, \text{pw}_C, \text{code}$) to $\mathcal{C}$ and a network message (C'_F, π') to $\mathcal{C}$

- save $(\text{pw}_C, \text{code}, C'_F, \pi')$, mark C^{ckid} as **fresh**

$\text{S}^{\text{ckid}}(b, T, x)$ for $\text{ckid} = (\text{sid}, \text{ssid})$:

i.e. ($\text{SSSESSION}, \dots, T$) and (SOUT, ckid) [$b = 0$] or ($\text{ADVCEVAL}, \text{ckid}, x$) [$b = 1$]:

1. if $b = 0$ and C^{ckid} is not marked **fresh** then ignore this query
2. if $b = 0$: set $\text{SK}_S \leftarrow \{0, 1\}^\kappa$, mark S^{ckid} as **completed**(T, SK_S), return SK_S
3. if $b = 1$: set $\text{SK}_S \leftarrow \{0, 1\}^\kappa$, mark S^{ckid} as **used**, and if $x = (\text{sid}, \text{hpw}, s_C)$ s.t. $\text{hpw} = \text{H}_p(\text{pw}_\Delta)$ and $s_C = \text{R}^{\text{sid}}(T)$ then return $(\text{SK}_S, \text{SK}_S)$ else return $(\perp, \text{SK}_S)$

$\text{C}_2^{\text{ckid}}(b, y, \text{SK}^*, \text{w})$, i.e. (COUT, ckid) [$b = 0$] or ($\text{ADVSEVAL}(\text{ckid}, y, \text{SK}^*)$) [$b = 1$]

followed by ($\text{WITNESS}, \text{w}$) to $\mathcal{F}_{\text{NIZK}}$:

1. if $b = 0$ and S^{ckid} is marked **used** then ignore this query else mark C^{ckid} as **used**
2. if $\exists$ no record $((\text{sid}, C'_F), \cdot, \pi')$ and $\mathcal{Z}$ sends ($\text{WITNESS}, \text{w}$) s.t. $\text{w} = (\text{hpw}, s, d_F')$ and $C'_F = \text{COM}(f^*(\text{sid}, \text{hpw}, s); d_F')$ then save $((\text{sid}, C'_F), \text{w}, \pi')$
3. if $\exists$ (still) no record $((\text{sid}, C'_F), \cdot, \pi')$ return $\text{SK}_C \leftarrow_{\mathcal{S}} \{0, 1\}^\kappa$
4. elseif $b = 0$ and S^{ckid} is marked **completed**(T, SK_S):
 - (a) if $C'_F = C_F$ and $(\text{pw}_C, \text{code}) = (\text{pw}_\Delta, \text{R}^{\text{sid}}(T))$ return $\text{SK}_C := \text{SK}_S$
 - (b) else return $\text{SK}_C \leftarrow_{\mathcal{S}} \{0, 1\}^\kappa$
5. elseif $b = 1$ then:
 - (a) if $y \neq (\text{sid}, F', s_S, d_F')$ s.t. $C'_F = \text{COM}(F'; d_F')$ return $\text{SK}_C \leftarrow_{\mathcal{S}} \{0, 1\}^\kappa$
 - (b) elseif $C'_F = C_F$ then:
 - i. if sid is not **compromised** then abort [abort.cmt]
 - ii. elseif $(\text{pw}_C, \text{code}) = (\text{pw}_\Delta, \text{R}^{\text{sid}}(1) + v_1 - s_S)$ then return $\text{SK}_C := \text{SK}^*$
 - iii. else return $\text{SK}_C \leftarrow_{\mathcal{S}} \{0, 1\}^\kappa$
 - (c) else retrieve $((\text{sid}, C'_F), \text{w}, \pi')$ s.t. $\text{w} = (\text{hpw}', s', d_F')$, and:
 - i. if $\text{hpw}' = \text{H}_p(\text{pw}_C)$ and $\text{code} = s' - s_S$ return $\text{SK}_C := \text{SK}^*$
 - ii. else return $\text{SK}_C \leftarrow_{\mathcal{S}} \{0, 1\}^\kappa$

48

$\text{O}_{\text{group}}(\cdot)$, i.e. adversary's group operations:

- SIM_{OWF} monitors adversarial group operations
- after $\mathcal{Z}$'s query to $\text{O}_{\text{compr}}^{\text{sid}}$, which prompts SIM_{OWF} to issue ($\text{LEAKVAL}, \text{sid}$), for every query ($\text{ADVTEST}, \text{sid}, \text{pw}^*, s^*$) which SIM_{OWF} makes (as if to $\mathcal{F}_{\text{OWF}}$), it receives “correct” if $(\text{pw}^*, s^* - v_1) = (\text{pw}_\Delta, \text{R}^{\text{sid}}(1))$ and “wrong” otherwise.

Fig. 14. Experiment $\text{Ideal}_{\mathcal{Z}}[\mathcal{F}_{\text{TFA-KE}}, \text{SIM}]$: the ideal world implied by $\mathcal{F}_{\text{TFA-KE}}$ and SIM

3. Server oracle S^{ckid} , after file compromise, in response to an active attack, i.e. in the case $b = 1$, outputs (SK_S, SK_S) rather than $(SK_\perp, SK_S)$ if $\mathcal{Z}$'s input $x = (\text{sid}, \text{hpw}, s_C)$ satisfies that $\text{hpw} = \text{hpw}_\Delta$ and $s_C = R^{\text{sid}}(1)$.
4. Group operation oracle O_{group} is handled by a generic group interface.

The ideal-world game, shown in Figure 14, matches the view of G_9 except for the aspects listed above, which it handles as follows, using simulator SIM_{OWF} implied by the assumption that function $\text{CommittedOwf}[f, \text{COM}]$ realizes OWF functionality $\mathcal{F}_{\text{OWF}}$:

1. At initialization query, Init^{sid} runs the simulator SIM_{OWF} on query $(\text{EVAL}, \text{sid}, S)$, and receives C_F output by SIM_{OWF} .
2. On file compromise query, $O_{\text{compr}}^{\text{sid}}$ acts as the environment to SIM_{OWF} and issues to it a file compromise permission, which triggers SIM_{OWF} to output $(\text{LEAKVAL}, \text{sid})$. When in response SIM_{OWF} outputs F, d_F , the game passes these to the environment and picks random v_1 . Importantly, in-between sending $(\text{LEAKVAL}, \text{sid})$ and F, d_F simulator SIM_{OWF} might issue multiple $(\text{ADVTEST}, \text{sid}, \dots)$ queries which the game services as described in item 4 below.
3. Server oracle S^{ckid} , in the same case as item 3, replaces check $\text{hpw} = \text{hpw}_\Delta$ with a check that $\mathcal{Z}$ computed H_p on pw_Δ and $\text{hpw} = H_p(\text{pw}_\Delta)$.
4. Group operation oracle is handled by SIM_{OWF} who monitors adversarial group operations, and after $\mathcal{Z}$'s query to $O_{\text{compr}}^{\text{sid}}$ which prompts SIM_{OWF} to issue $(\text{LEAKVAL}, \text{sid})$, for every query $(\text{ADVTEST}, \text{sid}, \text{pw}^*, s^*)$ which SIM_{OWF} makes, the game sends to SIM_{OWF} a response “correct” if $(\text{pw}^*, s^* - v_1) = (\text{pw}_\Delta, R^{\text{sid}}(1))$ and “wrong” otherwise.

$$\Delta_{\mathcal{Z}}(G_9, G_{10}) \leq \text{Adv}_{f, \text{COM}}^{\mathcal{F}_{\text{OWF}}}[\mathcal{Z}, \text{SIM}_{\text{OWF}}] \quad (23)$$

Game G_{11} (postpone choice of v_1 to file compromise): We modify the game so v_1 is sampled not at the initialization call to Init^{sid} but on the file compromise call to $O_{\text{compr}}^{\text{sid}}$. Since G_{10} doesn't use v_1 until file compromise we have:

$$\Delta_{\mathcal{Z}}(G_{10}, G_{11}) = 0 \quad (24)$$

Game G_{12} (use H_p pre-image in server oracle test): We modify the game so S^{ckid} oracle in the active attack case, i.e. $b = 1$, after server compromise, replaces check $\text{hpw} = \text{hpw}_\Delta$ with a check that $\mathcal{Z}$ computed H_p on pw_Δ and $\text{hpw} = H_p(\text{pw}_\Delta)$. Since without this query H_p is random to $\mathcal{Z}$ and, crucially, SIM_{OWF} works in a non-programmable RO model, and in particular has a read-only access to H_p queries and hence has no information about hpw_Δ if $\mathcal{Z}$ doesn't perform this query, this modification is indistinguishable unless $\mathcal{Z}$ guesses H_p on some query to S^{ckid} , leading to the following:

$$\Delta_{\mathcal{Z}}(G_{11}, G_{12}) \leq q_{\text{ses}}/|\mathcal{HP}| \quad (25)$$

Game G_{13} (ideal world): Game G_{13} just renders game G_{12} in a Figure 14, i.e.

$$\Delta_Z(G_{12}, G_{13}) = 0 \quad (26)$$

We argue that game G_{13} also renders the ideal-world interaction created by $\mathcal{F}_{\text{TFA-KE}}$ and our simulator SIM , and we explain this for each oracle call below:

- Init^{sid} models a $(\text{STOREFILE}, \text{sid}, S)$ call to S . In the ideal-world execution, upon receiving $(\text{STOREFILE}, \text{sid}, S)$ from $\mathcal{F}_{\text{TFA-KE}}$, SIM queries SIM_{OWF} to obtain the same C_F as Init^{sid} , and sends it via the $\mathcal{F}_{\text{NIZK}}$ interface, through which $\mathcal{Z}$ learns C_F . After receiving π , the tuple $((\text{sid}, C_F), \perp, \pi)$ is recorded.
- $\text{O}_{\text{compr}}^{\text{sid}}$ models a COMPROMISEFILE call to S . In the ideal-world execution, upon receiving $(\text{LEAKVAL}, \text{sid})$ from SIM_{OWF} , SIM sends $(\text{COMPROMISEFILE}, \text{sid})$ to $\mathcal{F}_{\text{TFA-KE}}$, marks sid as **compromised**, selects the same v_1 and k as $\text{O}_{\text{compr}}^{\text{sid}}$, and aborts with $[\text{abort.key}]$ if $\text{H}_{\text{code}}^{\text{sid}}$ was previously queried for some T . Then, SIM reveals (k, F, C_F, d_F, π) to $\mathcal{Z}$. Note that C_F and π are omitted by $\text{O}_{\text{compr}}^{\text{sid}}$ since $\mathcal{Z}$ already learns them through interaction with Init^{sid} .
- $\text{H}_{\text{name}}^{[\text{sid}]}$ models calls to the hash functions H_{code} and H_p . In the ideal-world execution, upon a query (k, T) to $\text{H}_{\text{code}}^{\text{sid}}$, SIM services it as $\mathcal{F}_{\text{RO}}$, except when sid is **compromised** and $k = k$, in which case it programs $\text{H}_{\text{code}}^{\text{sid}}(k, T) := v_1 - \Delta_T$, where Δ_T is $\mathcal{F}_{\text{TFA-KE}}$'s response to $(\text{EVALCODEDIFF}, \text{sid}, 1, T)$. By definition, $\Delta_T = \text{R}^{\text{sid}}(T) - \text{R}^{\text{sid}}(1)$, where R^{sid} is a random function maintained by $\mathcal{F}_{\text{TFA-KE}}$. Thus, $\text{H}_{\text{code}}^{\text{sid}}(k, T)$ is programmed as $v_1 - (\text{R}^{\text{sid}}(T) - \text{R}^{\text{sid}}(1)) = v_1 + \text{R}^{\text{sid}}(1) - \text{R}^{\text{sid}}(T)$, which is the value returned by $\text{H}_{\text{name}}^{[\text{sid}]}(x)$. For a query pw to H_p , SIM services it as $\mathcal{F}_{\text{RO}}$ with bookkeeping, except in the case of a collision, in which case SIM aborts with $[\text{abort.hash}]$, as in $\text{H}_{\text{name}}^{[\text{sid}]}(x)$.
- $\text{D}_{\text{code}}^{\text{sid}}$ models code retrieval via a READCODE call to D . In the ideal-world execution, upon receiving $(\text{READCODE}, \text{sid}, T)$ from D , $\mathcal{F}_{\text{TFA-KE}}$ sends $\text{R}^{\text{sid}}(T)$ to D , which is exactly what $\text{D}_{\text{code}}^{\text{sid}}(T)$ returns.
- C_1^{ckid} models the first round of the C 's code. In the ideal-world execution, upon receiving $(\text{CSESSION}, \text{sid}, \text{ssid}, S, \text{pw}, \text{code})$, $\mathcal{F}_{\text{TFA-KE}}$ records $(\text{ssid}, C, S, \text{pw}, \text{code})$ if it is the first CSESSION query for ssid . Then, upon receiving $(\text{CSESSION}, \text{sid}, \text{ssid}, C, S)$ from $\mathcal{F}_{\text{TFA-KE}}$, SIM obtains (C'_F, π') and marks session C^{ckid} as **fresh**. Since nothing new is revealed to $\mathcal{Z}$, C_1^{ckid} simply stores $(\text{pw}_C, \text{code}, C'_F, \pi')$ and marks C^{ckid} as **fresh**.
- S^{ckid} models the S 's code. In the ideal-world execution, upon receiving $(\text{SSESSION}, \text{sid}, \text{ssid}, S, C)$ from $\mathcal{F}_{\text{TFA-KE}}$, SIM sends (C_F, π) to $\mathcal{A}^*$, marks session S^{ckid} as **fresh**, and sends $(\text{SEVAL}, \text{ckid}, C, (\text{sid}, C_F))$ as if from $\mathcal{F}_{\text{CKEM}}$ to $\mathcal{A}^*$. These values are omitted since $\mathcal{Z}$ already knows C_F and π . If $\mathcal{A}^*$ then sends $(\text{SOUT}, \text{ckid})$ to $\mathcal{F}_{\text{CKEM}}$, SIM marks S^{ckid} as **completed** and sends $(\text{NEWKEY}, \text{sid}, \text{ssid}, S, \perp)$ to $\mathcal{F}_{\text{TFA-KE}}$, which in turn sends a uniformly random SK_S to S . This corresponds to the $b = 0$ branch in S^{ckid} .
If instead $\mathcal{A}^*$ sends $(\text{ADVCEVAL}, \text{ckid}, x)$ to $\mathcal{F}_{\text{CKEM}}$, SIM first checks whether $\text{sid}' \neq \text{sid}$ or $(\cdot, \text{hpw}) \notin \text{T}_p$. If so, the test fails immediately. Otherwise, it queries $(\text{TEST}, \text{sid}, \text{ssid}, S, \text{pw}, s_C)$ at $\mathcal{F}_{\text{TFA-KE}}$ and fails the test if it is a "wrong guess". In that case, SIM sends $(\text{NEWKEY}, \text{sid}, \text{ssid}, S, \perp)$ to $\mathcal{F}_{\text{TFA-KE}}$

and $(\text{COUT}, \text{ssid}, \perp)$ to $\mathcal{A}^*$ via $\mathcal{F}_{\text{CKEM}}$. Thus, $\mathcal{Z}$ learns $(\perp, \text{SK}_S)$, where $\perp$ is returned by $\mathcal{F}_{\text{CKEM}}$ and SK_S is the server's output.

The only non-failing case is when the **TEST** query to $\mathcal{F}_{\text{TFA-KE}}$ succeeds, which occurs when (pw, s_C) matches the record stored by $\mathcal{F}_{\text{TFA-KE}}$. This means $\text{hpw} = H_p(\text{pw}_\Delta)$ and $s_C = R^{\text{sid}}(T)$, which is the same condition checked in S^{ckid} . In this case, **SIM** sends $(\text{NEWKEY}, \text{sid}, \text{ssid}, S, \text{SK})$ to $\mathcal{F}_{\text{TFA-KE}}$ and $(\text{COUT}, \text{ssid}, \text{SK})$ to $\mathcal{A}^*$ via $\mathcal{F}_{\text{CKEM}}$, so $\mathcal{Z}$ learns $(\text{SK}_S, \text{SK}_S)$.

- C_2^{ckid} models the second round of the C 's code. It first checks the witness. When $b = 0$, it models the honest path where $(\text{COUT}, \text{ckid})$ is called, and $\text{SK}_C := \text{SK}_S$ is returned iff $C'_F = C_F$, $(\text{pw}_C, \text{code}) = (\text{pw}_\Delta, R^{\text{sid}}(T))$, and S^{ckid} is marked **completed** (T, SK_S) . Otherwise, SK_C is uniformly random.

When $b = 1$, it models the adversarial path where $(\text{ADVSEVAL}(\text{ckid}, y, \text{SK}^*))$ is called. In the ideal-world execution, **SIM** also first checks the witness $C'_F = \text{COM}(f^*(\text{sid}, \text{hpw}, s); d_F')$, exactly as in C_2^{ckid} . If the check fails, **SIM** short-circuits to the end and sends $(\text{RKEY}, \text{sid}, \text{ssid}, C)$ to $\mathcal{F}_{\text{TFA-KE}}$, which yields a uniformly random SK_C .

After verifying that S^{ckid} is marked **completed**, it checks whether $C'_F = C_F$. If not, it sends $(\text{RKEY}, \text{sid}, \text{ssid}, C)$ to $\mathcal{F}_{\text{TFA-KE}}$. Otherwise, it sends $(\text{NEWKEY}, \text{sid}, \text{ssid}, C, \perp)$ to $\mathcal{F}_{\text{TFA-KE}}$, mirroring C_2^{ckid} .

Upon receiving $(\text{ADVSEVAL}, \text{ckid}, y, \text{SK}^*)$, after parsing y , it checks if $(\text{sid}', C'_F) \neq (\text{sid}, \text{COM}(f^*(\text{sid}, \text{hpw}, s); d_F'))$. If so, it sends $(\text{RKEY}, \text{sid}, \text{ssid}, C)$ to $\mathcal{F}_{\text{TFA-KE}}$, yielding a uniformly random SK_C , just like C_2^{ckid} .

Otherwise, it checks whether $C'_F = C_F$. If not, and sid is not **compromised**, it aborts with $[\text{abort.cmt}]$, as in C_2^{ckid} . If sid is **compromised**, **SIM** sends $(\text{IMPERSONATE}, \text{sid}, \text{ssid}, 1, v_1 - s_S)$ and $(\text{NEWKEY}, \text{sid}, \text{ssid}, C, \text{SK}^*)$ to $\mathcal{F}_{\text{TFA-KE}}$. The **IMPERSONATE** call checks whether $\text{pw}_C = \text{pw}_\Delta$ and $\text{code} = R^{\text{sid}}(1) + v_1 - s_S$. If true, the subsequent **NEWKEY** call yields $\text{SK}_C := \text{SK}^*$; otherwise, it yields a uniformly random SK_C .

If $C'_F \neq C_F$, **SIM** searches T_p for a pw such that $(\text{pw}, \text{hpw}) \in T_p$, which corresponds to having $\text{hpw}' = H_p(\text{pw}_C)$ in C_2^{ckid} . If it finds such a pw , it sends $(\text{TEST}, \text{sid}, \text{ssid}, C, \text{pw}, s - s_S)$ and $(\text{NEWKEY}, \text{sid}, \text{ssid}, C, \text{SK}^*)$ to $\mathcal{F}_{\text{TFA-KE}}$. The **TEST** call returns "correct guess" and marks the record as **compromised** when $\text{code} = s' - s_S$. In that case, the subsequent **NEWKEY** yields $\text{SK}_C := \text{SK}^*$. Otherwise, it yields a uniformly random SK_C , just like in C_2^{ckid} .

- O_{group} models $\mathcal{Z}$'s group operations. In the ideal-world execution, upon receiving $(\text{ADVTEST}, \text{sid}, (\text{pw}, s'))$ from SIM_{OWF} , **SIM** simply sends $(\text{OFFLINETEST}, \text{sid}, \text{pw}, 1, s' - !v_1)$ to $\mathcal{F}_{\text{TFA-KE}}$ and forwards the result back to SIM_{OWF} . Here, the **OFFLINETEST** query corresponds to checking whether $(\text{pw}, s' - v_1) = (\text{pw}_\Delta, R^{\text{sid}}(1))$, which is exactly what O_{group} does.

Summarizing the above explanations we conclude:

$$\text{Exp}_{\mathcal{Z}}[\text{G}_{13}] = \text{Ideal}_{\mathcal{Z}}[\mathcal{F}_{\text{TFA-KE}}, \text{SIM}] \quad (27)$$

Summing up all the above bounds we conclude that for an efficient $\mathcal{Z}$ its real-vs-ideal distinguishing advantage, $|\text{Real}_{\mathcal{Z}}[\text{TFA}_{\text{gen}}] - \text{Ideal}_{\mathcal{Z}}[\mathcal{F}_{\text{TFA-KE}}, \text{SIM}]|$, is negligible under the stated assumptions, which proves the theorem.

F Non-Interactive Zero-Knowledge Proofs

In this section we discuss the NIZKs used by our protocols TFA_{gen} , $\text{CKEM}_{\text{Vand}}$, and $\text{CKEM}_{\text{PoNR}}$. We define universally composable NIZKs by ideal functionality $\mathcal{F}_{\text{NIZK}}$ (Figure 1), which is parameterized by a relation between statements and witnesses.

F.1 $\text{CKEM}_{\text{Vand}}$ and TFA_{gen} with f_{Vand}

Protocol $\text{CKEM}_{\text{Vand}}$ (Section 5.3) uses realizations of $\mathcal{F}_{\text{NIZK}}$ for two relations: $\mathcal{R}_{\text{C}}$ and $\mathcal{R}_{\text{S}}$. As discussed in Section 5.3, the former enforces the honest behavior of clients and the latter enforces the honest behavior of servers. We define both relations formally here.

$\mathcal{R}_{\text{C}}$ is the set $\{(\mathbf{x}, \mathbf{w})\}$ of statement-witness pairs, where statement $\mathbf{x} = (\text{sid}, \text{pk}_{\text{C}}, \mathbf{G}) \in \{0, 1\}^* \times \mathbb{G} \times (\mathbb{G}^2)^{N+1}$, witness $\mathbf{w} = (\text{hpw}, s_{\text{C}}, \mathbf{d}_{\text{G}}) \in \{0, 1\}^{\lfloor \log p \rfloor - t - 1} \times \{0, 1\}^t \times \mathbb{Z}_p^{N+1}$, and $\mathbf{x}$ and $\mathbf{w}$ satisfy:

$$\forall_{j=0}^N : G_j = (g^{d_{G_j}}, \text{pk}_{\text{C}}^{d_{G_j}} \cdot \prod_{i=\max\{j,1\}}^N H_{\text{G}}^{\text{sid}}(i)^{\binom{i}{j} (\text{hpw}|0|s_{\text{C}})^{i-j}})$$

$\mathcal{R}_{\text{S}}$ is the set $\{(\mathbf{x}, \mathbf{w})\}$ of statement-witness pairs, where statement $\mathbf{x} = (C_F, \text{pk}_{\text{C}}, \mathbf{G}, C_K, (V'_0, V'_1)) \in \mathbb{G}^3 \times \mathbb{G} \times (\mathbb{G}^2)^{N+1} \times \mathbb{G}^3 \times (\mathbb{G}^2)^2$, witness $\mathbf{w} = (F, d_F, s_{\text{S}}, (r_0, r_1), K, d_K, (d_{V_0}, d_{V_1}), (V_0, V_1)) \in \mathbb{G} \times \mathbb{Z}_p^2 \times \{0, 1\}^t \times \mathbb{Z}_p^2 \times \mathbb{G} \times \mathbb{Z}_p^2 \times \mathbb{Z}_p^2 \times (\mathbb{G}^2)^2$, and $\mathbf{x}$ and $\mathbf{w}$ satisfy:

$$\begin{aligned} C_F &= (g^{d_{F,0}}, g^{d_{F,1}}, F \cdot g^{d_{F,0}d_{F,1}}) \wedge C_K = (g^{d_{K,0}}, g^{d_{K,1}}, K \cdot g^{d_{K,0}d_{K,1}}) \wedge r_0, r_1 \neq 0 \\ V_0 &= (g^{d_{V_0}} \cdot (\prod_{j=0}^N (G_{j,0})^{s_{\text{S}}^j})^{r_0}, \text{pk}_{\text{C}}^{d_{V_0}} \cdot K \cdot (\prod_{j=0}^N (G_{j,1})^{s_{\text{S}}^j} / F)^{r_0}) \\ V_1 &= (g^{d_{V_1}} \cdot (\prod_{j=0}^N (G_{j,0})^{(s_{\text{S}} - 2^t)^j})^{r_1}, \text{pk}_{\text{C}}^{d_{V_1}} \cdot K \cdot (\prod_{j=0}^N (G_{j,1})^{(s_{\text{S}} - 2^t)^j} / F)^{r_1}) \\ (V'_0, V'_1) &= (V_0, V_1) \vee (V'_0, V'_1) = (V_1, V_0) \end{aligned}$$

In addition to our realization of $\mathcal{F}_{\text{CKEM}}$ for f_{Vand} , instantiating the generic TFA_{gen} construction for OWF f_{Vand} also requires a realization of $\mathcal{F}_{\text{NIZK}}[\mathcal{R}_{\text{Init}}]$ for OWF f_{Vand} and commitment scheme ElG . $\mathcal{R}_{\text{Init}}$ is defined in Section 5.2, but for clarity we repeat it here for this particular OWF and commitment scheme.

$\mathcal{R}_{\text{Init}}$ is the set $\{(\mathbf{x}, \mathbf{w})\}$ of statement-witness pairs, where statement $\mathbf{x} = (\text{sid}, C_F) \in \{0, 1\}^* \times \mathbb{G}^3$, witness $\mathbf{w} = (\text{hpw}_{\Delta}, s, d_F) \in \{0, 1\}^{\lfloor \log p \rfloor - t - 1} \times \{0, 1\}^t \times \mathbb{Z}_p^2$, and $\mathbf{x}$ and $\mathbf{w}$ satisfy:

$$C_F = (g^{d_{F,0}}, g^{d_{F,1}}, g^{d_{F,0}d_{F,1}} \cdot \prod_{i \in [N]} H_{\text{G}}^{\text{sid}}(i)^{(\text{hpw}_{\Delta}|0|s)^i})$$

F.2 $\text{CKEM}_{\text{PoNR}}$ and TFA_{gen} with f_{PoNR}

Protocol $\text{CKEM}_{\text{PoNR}}$ (Section 5.4) uses realizations of $\mathcal{F}_{\text{NIZK}}$ for two relations: $\mathcal{R}_{\text{C}}$ and $\mathcal{R}_{\text{S}}$. As discussed in Section 5.4, the former enforces the honest behavior

of clients and the latter enforces the honest behavior of servers. We define both relations formally here.

$\mathcal{R}_C$ is the set $\{(\mathbf{x}, \mathbf{w})\}$ of statement-witness pairs, where statement $\mathbf{x} = (\text{sid}, \text{pk}_C, \mathbf{G}, \hat{G}, \mathbf{C}_m) \in \{0, 1\}^* \times \mathbb{G} \times (\mathbb{G}^2)^N \times \mathbb{G}^2 \times \mathbb{G}^{2t}$, witness $\mathbf{w} = (\text{hpw}, s_C, \mathbf{d}_G, d_{\hat{G}}, \delta, \mathbf{d}_m) \in \mathbb{Z}_p^N \times \{0, 1\}^t \times \mathbb{Z}_p^N \times \mathbb{Z}_p \times (\mathbb{Z}_p^t)^N \times \mathbb{Z}_p^{2t}$, and $\mathbf{x}$ and $\mathbf{w}$ satisfy:

$$\begin{aligned} \forall_{i \in [N]} : \quad G_i &= (g^{d_{G_i}}, \text{pk}_C^{d_{G_i}} \cdot \text{H}_{\mathbb{G}}^{\text{sid}}(i)^{\text{hpw}_i} / \prod_{j \in [t]} \delta_{i,j}) \\ \hat{G} &= (g^{d_{\hat{G}}}, \text{pk}_C^{d_{\hat{G}}} \cdot \prod_{i \in [N]} \hat{\text{H}}_{\mathbb{G}}^{\text{sid}}(i)^{\text{hpw}_i}) \\ \forall_{j \in [t]} : \quad C_{m_{j,s_C[j]}} &= g^{d_{m_{j,s_C[j]}}} \cdot \prod_{i \in [N]} h_i^{\delta_{i,j}} \\ \forall_{j \in [t]} : \quad C_{m_{j,1-s_C[j]}} &= g^{d_{m_{j,1-s_C[j]}}} \cdot \prod_{i \in [N]} h_i^{\delta_{i,j} \cdot \text{H}_{\mathbf{x}}(i,j)} \end{aligned}$$

$\mathcal{R}_S$ is the set $\{(\mathbf{x}, \mathbf{w})\}$ of statement-witness pairs, where statement $\mathbf{x} = (C_F, \text{pk}_C, \mathbf{G}, \hat{G}, \mathbf{C}_m, C_K, V) \in \mathbb{G}^3 \times \mathbb{G} \times (\mathbb{G}^2)^N \times \mathbb{G}^2 \times \mathbb{G}^{2t} \times \mathbb{G}^3 \times \mathbb{G}^2$, witness $\mathbf{w} = (F, d_F, s_S, \mathbf{m}^S, \mathbf{d}_m^S, r, K, d_K, d_V) \in \mathbb{G} \times \mathbb{Z}_p^2 \times \{0, 1\}^t \times (\mathbb{Z}_p^N)^t \times \mathbb{Z}_p^t \times \mathbb{Z}_p \times \mathbb{G} \times \mathbb{Z}_p^2 \times \mathbb{Z}_p$, and $\mathbf{x}$ and $\mathbf{w}$ satisfy:

$$\begin{aligned} C_F &= (g^{d_{F,0}}, g^{d_{F,1}}, F \cdot g^{d_{F,0}d_{F,1}}) \wedge C_K = (g^{d_{K,0}}, g^{d_{K,1}}, K \cdot g^{d_{K,0}d_{K,1}}) \wedge r \neq 0 \\ \forall_{j \in [t]} : \quad C_{m_{j,s_S[j]}} &= g^{d_{m_{j,s_S[j]}}} \cdot \prod_{i \in [N]} h_i^{m_{i,j}} \\ V &= (g^{d_V} \cdot (\hat{G}_0 \cdot \prod_{i \in [N]} (G_{i,0})^{\prod_{j \in [t]} m_{i,j}})^r, \text{pk}_C^{d_V} \cdot K \cdot (\hat{G}_1 \cdot \prod_{i \in [N]} (G_{i,1})^{\prod_{j \in [t]} m_{i,j}} / F)^r) \end{aligned}$$

In addition to our realization of $\mathcal{F}_{\text{CKEM}}$ for f_{PoNR} , instantiating the generic TFA_{gen} construction for OWF f_{PoNR} also requires a realization of $\mathcal{F}_{\text{NIZK}}[\mathcal{R}_{\text{Init}}]$ for OWF f_{PoNR} and commitment scheme ElG . $\mathcal{R}_{\text{Init}}$ is defined in Section 5.2, but for clarity we repeat it here for this particular OWF and commitment scheme.

$\mathcal{R}_{\text{Init}}$ is the set $\{(\mathbf{x}, \mathbf{w})\}$ of statement-witness pairs, where statement $\mathbf{x} = (\text{sid}, C_F) \in \{0, 1\}^* \times \mathbb{G}^3$, witness $\mathbf{w} = (\text{hpw}_{\Delta}, s, d_F) \in \mathbb{Z}_p^N \times \{0, 1\}^t \times \mathbb{Z}_p^2$, and $\mathbf{x}$ and $\mathbf{w}$ satisfy:

$$C_F = (g^{d_{F,0}}, g^{d_{F,1}}, g^{d_{F,0}d_{F,1}} \cdot \prod_{i \in [N]} \text{H}_{\mathbb{G}}^{\text{sid}}(i)^{\text{hpw}_i} \cdot \prod_{j \in [t], s[j]=1} \text{H}_{\mathbf{x}}(i,j) \cdot \hat{\text{H}}_{\mathbb{G}}^{\text{sid}}(i)^{\text{hpw}_i})$$

F.3 Efficient Realizations

Our OWFs and CKEM protocols are specifically designed so that their requisite NIZKs can be efficiently instantiated using standard techniques. In particular, every relation is a discrete-logarithm representation over public bases. A Sigma protocol for each of these relations is a straightforward extension of Schnorr's proof of discrete-logarithm knowledge, see e.g. the report by Camenisch and Stadler [6]. Sigma protocols can be compiled into universally composable NIZK proof systems using the Fischlin transform, which is practically efficient [10,7].

Some relations for f_{Vand} require proving that witnesses are representable as bitstrings of a certain length. Such statements can be expressed as discrete-logarithm representations by committing to each bit individually, proving that

each bit is either 0 or 1, and then proving that the committed witness is represented by those bits. OR-compilation of Sigma protocols, used by our other relations as well, is easily handled by a standard technique. Finally, the relations $\mathcal{R}_S$ of both $\text{CKEM}_{\text{Vand}}$ and $\text{CKEM}_{\text{PoNR}}$ include conditions that certain values are nonzero. This is easily handled by proving the existence of a multiplicative inverse.

Proof size and verifier efficiency can be significantly improved by employing Sigma-protocol compression [2].

G Proof of $\text{CKEM}_{\text{Vand}}$ Security

In this section we prove Theorem 8, i.e. that protocol $\text{CKEM}_{\text{Vand}}$ realizes $\mathcal{F}_{\text{CKEM}}[\text{SSEM}[f_{\text{Vand}}^*, \text{EIG}], \text{SSEM}_{\text{Adv}}[f_{\text{Vand}}^*]]$.

Specifically, for any efficient adversary $\mathcal{A}^*$ against protocol $\text{CKEM}_{\text{Vand}}$, there exists a simulator SIM such that no efficient environment $\mathcal{Z}$ can distinguish the view of $\mathcal{A}^*$ interacting with the real $\text{CKEM}_{\text{Vand}}$ protocol and the view of SIM interacting with the ideal functionality $\mathcal{F}_{\text{CKEM}}[\text{SSEM}[f_{\text{Vand}}^*, \text{EIG}], \text{SSEM}_{\text{Adv}}[f_{\text{Vand}}^*]]$ with advantage better than $3 \cdot \text{Adv}_{\text{DDH}} + \frac{q_C^2 + q_S^2 + q_S q_H + q_H}{p}$ where there are at most q_C client sessions, there are at most q_S server sessions, there are at most q_H queries to H_d and H_{SK} , and Adv_{DDH} is a bound on the DDH advantage of any efficient algorithm.

Proof. For any adversary $\mathcal{A}^*$, we construct simulator $\text{SIM}_{\text{CKEM-Vand}}$ as shown in Figure 15. Without loss of generality, we assume that $\mathcal{A}^*$ is a “dummy” adversary that merely passes messages to and from the environment $\mathcal{Z}$.

We now show that, for any efficient (i.e. PPT) $\mathcal{Z}$, the distinguishing advantage of $\mathcal{Z}$ between the real and simulated worlds is negligible. We denote the probability that $\mathcal{Z}$ outputs 1 in the interaction with the real world game or the simulated game as resp. $\text{Real}_{\mathcal{Z}}[\text{CKEM}_{\text{Vand}}]$ and $\text{Ideal}_{\mathcal{Z}}[\mathcal{F}_{\text{CKEM}}, \text{SIM}_{\text{CKEM-Vand}}]$. We denote the probability that $\mathcal{Z}$ outputs 1 in an interaction with security game G_i as $\text{Exp}_{\mathcal{Z}}[G_i] = \Pr[1 \leftarrow \mathcal{Z}^{G_i}(1^\kappa)]$, and we define $\Delta(G_i, G_j) = |\text{Exp}_{\mathcal{Z}}[G_i] - \text{Exp}_{\mathcal{Z}}[G_j]|$. The argument proceeds by a series of game changes, starting with game G_0 s.t. $\text{Exp}_{\mathcal{Z}}[G_0] = \text{Real}_{\mathcal{Z}}[\text{CKEM}_{\text{Vand}}]$ and ends with game G_{10} s.t. $\text{Exp}_{\mathcal{Z}}[G_{10}] = \text{Ideal}_{\mathcal{Z}}[\mathcal{F}_{\text{CKEM}}, \text{SIM}_{\text{CKEM-Vand}}]$. We must show that $\Delta(G_0, G_{10})$ is negligible.

Game G_0 (the real world): The distinguisher $\mathcal{Z}$ interacts with $\text{CKEM}_{\text{Vand}}$ (Figure 7) in the role of the honest parties and in the role of the adversary. Functionality $\mathcal{F}_{\text{NIZK}}$ (Figure 1) interacts with $\text{CKEM}_{\text{Vand}}$ and with $\mathcal{Z}$ via the adversarial interface. H_d and H_{SK} are true random oracles.

$$\text{Exp}_{\mathcal{Z}}[G_0] = \text{Real}_{\mathcal{Z}}[\text{CKEM}_{\text{Vand}}]$$

Game G_1 (statements never collide): G_1 is G_0 with the following changes:

- If two C sessions ever pick the same pk_C , the game immediately aborts.
- If two S sessions ever pick the same K , the game immediately aborts.

- Program H_d and H_{SK} outputs uniformly at random, except where special H_{SK} programming is noted below.
- On $(\text{CEVAL}, \text{ssid}_C, C, (\text{sid}_C, C_F))$ from $\mathcal{F}_{\text{CKEM}}$:
 - Sample $\text{pk}_C \leftarrow_{\mathcal{S}} \mathbb{G}$ and $\mathbf{G} \leftarrow_{\mathcal{S}} (\mathbb{G}^2)^{N+1}$.
 - Send $(\text{PROVE}, C, \text{ssid}_C, (\text{sid}_C, \text{pk}_C, \mathbf{G}))$ to $\mathcal{A}^*$ as if from $\mathcal{F}_{\text{NIZK}}[\mathcal{R}_C]$, await response $(\text{PROOF}, \text{ssid}_C, \pi_C)$, and save record $(\mathcal{R}_C, (\text{sid}_C, \text{pk}_C, \mathbf{G}), \perp, \pi_C, \text{ssid}_C)$.
 - Send $(\text{sid}_C, \text{pk}_C, \mathbf{G}, \pi_C)$ to $\mathcal{A}^*$ as if from C to S .
 - Await $(C_K, (V'_0, V'_1), \pi_S)$ from $\mathcal{A}^*$ to C .
 - Define $\mathbb{x}_S := (C_F, \text{pk}_C, \mathbf{G}, C_K, (V'_0, V'_1))$. If there exists no record $(\mathcal{R}_S, \mathbb{x}_S, \mathbb{w}_S, \pi_S, \text{ssid}_S)$ then send $(\text{VERIFY}, C, \text{ssid}_C, \mathbb{x}_S, \pi_S)$ to $\mathcal{A}^*$ as if from $\mathcal{F}_{\text{NIZK}}[\mathcal{R}_S]$, await response $(\text{WITNESS}, \text{ssid}_C, \mathbb{w}_S)$, and save record $(\mathcal{R}_S, \mathbb{x}_S, \mathbb{w}_S, \pi_S, \perp)$.
 - If $\mathbb{w}_S \neq \perp$ then parse $\mathbb{w}_S \rightarrow (F, d_F, s_S, \cdot, K, d_K, \cdot, \cdot)$, and...
 - * If $(\mathbb{x}_S, \mathbb{w}_S) \notin \mathcal{R}_S \vee H_d^{\text{sid}_C}(K) \neq d_K$: Send $(\text{ADVSEVAL}, \text{ssid}_C, \perp, 0^\kappa)$ to $\mathcal{F}_{\text{CKEM}}$.
 - * Else: Send $(\text{ADVSEVAL}, \text{ssid}_C, (\text{sid}_C, F, s_S, d_F), H_{SK}^{\text{sid}_C}(\text{ssid}_C, K))$ to $\mathcal{F}_{\text{CKEM}}$.
 - If $\mathbb{w}_S = \perp$, then...
 - * If $\text{ssid}_C \neq \text{ssid}_S$: Send $(\text{ADVSEVAL}, \text{ssid}_C, \perp, 0^\kappa)$ to $\mathcal{F}_{\text{CKEM}}$.
 - * Else: Send $(\text{COUT}, \text{ssid}_C)$ to $\mathcal{F}_{\text{CKEM}}$.
- On $(\text{SEVAL}, \text{ssids}, S, (\text{sids}, C_F))$ from $\mathcal{F}_{\text{CKEM}}$:
 - Await $(\text{sid}_C, \text{pk}_C, \mathbf{G}, \pi_C)$ from $\mathcal{A}^*$ to S .
 - Define $\mathbb{x}_C := (\text{sids}, \text{pk}_C, \mathbf{G})$. If there exists no record $(\mathcal{R}_C, \mathbb{x}_C, \mathbb{w}_C, \pi_C, \text{ssid}_C)$ then send $(\text{VERIFY}, S, \text{ssids}, \mathbb{x}_C, \pi_C)$ to $\mathcal{A}^*$ as if from $\mathcal{F}_{\text{NIZK}}[\mathcal{R}_C]$, await response $(\text{WITNESS}, \text{ssids}, \mathbb{w}_C)$, and save record $(\mathcal{R}_C, \mathbb{x}_C, \mathbb{w}_C, \pi_C, \perp)$.
 - If $\text{sid}_C \neq \text{sids} \vee (\mathbb{w}_C \neq \perp \wedge (\mathbb{x}_C, \mathbb{w}_C) \notin \mathcal{R}_C)$: Send $(\text{ADVCEVAL}, \text{ssids}, \perp)$ to $\mathcal{F}_{\text{CKEM}}$, receive response $(\text{COUT}, \text{ssids}, \perp)$, and break (i.e. don't run the following code).
 - If $\mathbb{w}_C \neq \perp$: Parse $\mathbb{w}_C \rightarrow (\text{hpw}, s_C, \cdot)$, send $(\text{ADVCEVAL}, \text{ssids}, (\text{sids}, \text{hpw}, s_C))$ to $\mathcal{F}_{\text{CKEM}}$, and receive response $(\text{COUT}, \text{ssids}, \text{SK})$.
 - If $\mathbb{w}_C = \perp$ then define $\text{SK} := \perp$, and...
 - * If $\text{ssids} \neq \text{ssid}_C$: Send $(\text{ADVCEVAL}, \text{ssids}, \perp)$ to $\mathcal{F}_{\text{CKEM}}$ and receive response $(\text{COUT}, \text{ssids}, \perp)$.
 - * Else: Send $(\text{SOUT}, \text{ssids})$ to $\mathcal{F}_{\text{CKEM}}$.
 - Sample $K'_0, K'_1 \leftarrow_{\mathcal{S}} \mathbb{G}$, and compute $V'_0 \leftarrow \text{ELG.Enc}_{\text{pk}_C}(K'_0)$ and $V'_1 \leftarrow \text{ELG.Enc}_{\text{pk}_C}(K'_1)$.
 - If $\text{SK} \neq \perp$ then sample $\beta \leftarrow_{\mathcal{S}} \{0, 1\}$, compute $C_K \leftarrow \text{ELG.Commit}(K'_\beta; H_d^{\text{ssids}}(K'_\beta))$, and program $H_{SK}^{\text{ssids}}(\text{ssids}, K'_\beta) := \text{SK}$. Else ($\text{SK} = \perp$) sample $C_K \leftarrow_{\mathcal{S}} \mathbb{G}^3$.
 - Define $\mathbb{x}_S := (C_F, \text{pk}_C, \mathbf{G}, C_K, (V'_0, V'_1))$. Send $(\text{PROVE}, S, \text{ssids}, \mathbb{x}_S)$ to $\mathcal{A}^*$ as if from $\mathcal{F}_{\text{NIZK}}[\mathcal{R}_S]$, await response $(\text{PROOF}, \text{ssids}, \pi_S)$, and save record $(\mathcal{R}_S, \mathbb{x}_S, \perp, \pi_S, \text{ssids})$.
 - Send $(C_K, (V'_0, V'_1), \pi_S)$ to $\mathcal{A}^*$ as if from S to C .

Fig. 15. $\text{SIM}_{\text{CKEM-Vand}}$: Simulator for protocol $\text{CKEM}_{\text{Vand}}$

Per the code of $\mathcal{F}_{\text{NIZK}}$, it is possible for a single statement-proof pair $(\mathbf{x}, \pi)$ to correspond to more than one witness $\mathbf{w}$ iff statement $\mathbf{x}$ is proven more than once by honest parties. This game change ensures that the $\mathbf{x}_C$ and $\mathbf{x}_S$ statements proven by honest parties never repeat. Therefore, in future games, we can retrieve the one $\mathbf{w}$ for any $(\mathbf{x}, \pi)$ without ambiguity.

Every $\mathbf{pk}_C$ (resp. K) is sampled uniformly at random from $\mathbb{G}$. Therefore, the probability of a collision is upper-bounded by q_C^2/p (resp. q_S^2/p). If no collision occurs, G_0 and G_1 are identical.

$$\Delta(G_0, G_1) \leq \frac{q_C^2 + q_S^2}{p}$$

Game G_2 (client NIZKs are simulated): G_2 is G_1 with only one change: $\mathbf{w}_C$ is omitted when C calls $\mathcal{F}_{\text{NIZK}}[\mathcal{R}_C].\text{PROVE}$. Since honest clients never attempt to prove false statements, the check for $(\mathbf{x}, \mathbf{w}) \in \mathcal{R}_C$ is simply skipped, and instead of a witness a special symbol $\mathbf{w} := \perp$ is recorded to represent that the produced proof is correct. The code of $\mathcal{F}_{\text{NIZK}}[\mathcal{R}_C].\text{VERIFY}$ is modified to always accept proofs for records with $\mathbf{w} = \perp$.

The views of $\mathcal{Z}$ in G_1 and G_2 are identical.

$$\Delta(G_1, G_2) = 0$$

Game G_3 (clients do not decrypt V'_0 and V'_1): G_3 is G_2 with the following changes to C :

- Instead of sampling $(\mathbf{sk}_C, \mathbf{pk}_C) \leftarrow \text{ElG.KG}$, only sample $\mathbf{pk}_C \leftarrow_{\$} \mathbb{G}$.
- After verifying π_S , do not decrypt V'_0 and V'_1 . Instead, parse $\mathbf{w}_S \rightarrow (F, \cdot, s_S, \cdot, K, d_K, \cdot, \cdot)$. If $H_d^{\text{sidc}}(K) = d_K \wedge F = f_{\text{Vand}}^*(\text{sid}_C, \text{hpw}, s_C + s_S)$, then output $\text{SK}' := H_{\text{SK}}^{\text{sidc}}(\text{ssid}_C, K)$. Otherwise, output $\text{SK}' \leftarrow_{\$} \{0, 1\}^\kappa$.

Notice that C 's changed code only runs if π_S verifies. $\mathcal{R}_S$ guarantees that the received V'_0 , V'_1 , and C_K are consistent with correct server behavior on $\mathbf{w}_S$, e.g. $C_K = \text{ElG.Commit}(K; d_K)$. Therefore it is not necessary for C to refer to V'_0 , V'_1 , and C_K directly. In the previous game, C will decrypt K from either V'_0 or V'_1 iff $F = f_{\text{Vand}}^*(\text{sid}_C, \text{hpw}, s_C + s_S)$, and if additionally $H_d^{\text{sidc}}(K) = d_K$ then C will use K to derive SK' . If either of those conditions does not occur, then C will output a random SK' . The behavior of C in this game is exactly the same.

The views of $\mathcal{Z}$ in G_2 and G_3 are identical.

$$\Delta(G_2, G_3) = 0$$

Game G_4 (G is random): G_4 is G_3 with only one change: instead of computing $(g_j)_{j \in \{0, 1, \dots, N\}}$ and $G = (G_j)_{j \in \{0, 1, \dots, N\}}$ as in $\text{CKEM}_{\text{Vand}}$, C simply samples $G \leftarrow_{\$} (\mathbb{G}^2)^{N+1}$.

Recall that client NIZKs are simulated, so it is not a problem that $\mathbf{x}_C$ no longer has a correct witness. Additionally, the responses (V'_0, V'_1) that S computes

from $\mathbf{G}$ are not decrypted and used by $\mathbf{C}$, so it is not a problem that these responses are formed from random G ciphertexts.

Given an environment $\mathcal{Z}$ that can distinguish $\mathbf{G}_3$ from $\mathbf{G}_4$, we construct a reduction $\mathcal{R}$ that can solve DDH. The input to $\mathcal{R}$ is tuple $(A, B, C) \in \mathbb{G}^3$. $\mathcal{R}$ emulates game $\mathbf{G}_3$ to $\mathcal{Z}$ with the following modifications to the code of $\mathbf{C}$:

- Instead of sampling $\text{pk}_{\mathbf{C}} \leftarrow_{\$} \mathbb{G}$, $\mathcal{R}$ samples $\delta \leftarrow_{\$} \mathbb{Z}_p$ and sets $\text{pk}_{\mathbf{C}} := A \cdot g^\delta$.
- Instead of encrypting by $d_{G_j} \leftarrow_{\$} \mathbb{Z}_p$, $G_j \leftarrow \text{ElG.Enc}_{\text{pk}_{\mathbf{C}}}(g_j; d_{G_j})$, $\mathcal{R}$ samples $\mu, \nu \leftarrow_{\$} \mathbb{Z}_p$ and sets $G_j := (g^\mu \cdot B^\nu, g_j \cdot (A^\mu \cdot C^\nu) \cdot (g^\mu \cdot B^\nu)^\delta)$.

Here we apply the well-known DDH randomized self-reduction [27,22]. If (A, B, C) is a Diffie-Hellman tuple, then $(A, g^\mu \cdot B^\nu, A^\mu \cdot C^\nu)$ is a random Diffie-Hellman tuple. If not, then it is a uniformly random tuple. For every $\mathbf{C}$ session, we apply a “shift” of δ to get a freshly random $\text{pk}_{\mathbf{C}}$. If (A, B, C) is a Diffie-Hellman tuple, then each G_j ciphertext is a correct ElGamal encryption of g_j under $\text{pk}_{\mathbf{C}}$, so the view of $\mathcal{Z}$ is identical to that in $\mathbf{G}_3$. If (A, B, C) is not a Diffie-Hellman tuple, then each G_j ciphertext is independently random, so the view of $\mathcal{Z}$ is identical to that in $\mathbf{G}_4$.

Reduction $\mathcal{R}$ outputs 1 iff $\mathcal{Z}$ outputs 1. Thus, the advantage of $\mathcal{Z}$ at distinguishing between $\mathbf{G}_3$ and $\mathbf{G}_4$ is equal to the advantage of $\mathcal{R}$ in the DDH game, which is $\leq \text{Adv}_{\text{DDH}}$.

$$\Delta(\mathbf{G}_3, \mathbf{G}_4) \leq \text{Adv}_{\text{DDH}}$$

Game $\mathbf{G}_5$ (server NIZKs are simulated): $\mathbf{G}_5$ is $\mathbf{G}_4$ with the following changes:

- ws is truncated to $(F, \cdot, s_S, \cdot, K, \cdot, \cdot, \cdot)$ when $\mathbf{S}$ calls $\mathcal{F}_{\text{NIZK}}[\mathcal{R}_S].\text{PROVE}$. Since honest servers never attempt to prove false statements, the check for $(\mathbf{x}, \mathbf{w}) \in \mathcal{R}_S$ is simply skipped, and the code of $\mathcal{F}_{\text{NIZK}}[\mathcal{R}_S].\text{VERIFY}$ is modified to always accept proofs with witnesses of this truncated form.
- In the previous game, $\mathbf{C}$ parses F , s_S , K , and d_K from ws (see $\mathbf{G}_3$), but d_K is now missing from the recorded witnesses of honestly generated server proofs. However, d_K is only used to check whether $H_d^{\text{sid}_{\mathbf{C}}}(K) = d_K$, and this condition always holds for honest servers, so the code of $\mathbf{C}$ is modified to simply skip that check when handling a witness of this truncated form.

The views of $\mathcal{Z}$ in $\mathbf{G}_4$ and $\mathbf{G}_5$ are identical.

$$\Delta(\mathbf{G}_4, \mathbf{G}_5) = 0$$

Game $\mathbf{G}_6$ (V'_0 and V'_1 are random when responding to unsuccessful dishonest clients): $\mathbf{G}_6$ is $\mathbf{G}_5$ with the following changes to $\mathbf{S}$ after verifying $\pi_{\mathbf{C}}$ and checking that $\text{sid}_{\mathbf{C}} = \text{sid}_S$:

- If $\text{w}_{\mathbf{C}} = \perp$ (i.e. $\pi_{\mathbf{C}}$ was generated by an honest $\mathbf{C}$, see $\mathbf{G}_2$), then proceed as before.
- Otherwise, parse $\text{w}_{\mathbf{C}} \rightarrow (\text{hpw}, s_{\mathbf{C}}, \cdot)$. If $F = f_{\text{Vand}}^*(\text{sid}_S, \text{hpw}, s_{\mathbf{C}} + s_S)$, then still proceed as before.

- Otherwise, do not compute V'_0 and V'_1 as normal. Instead, simply sample $V'_0, V'_1 \leftarrow_{\S} \mathbb{G}^2$.

Recall that server NIZKs are simulated, so it is not a problem that $\mathcal{S}$ no longer knows a witness for $\mathbb{x}_S$. Furthermore, we are only modifying the behavior of $\mathcal{S}$ in the case that $w_C \neq \perp$, in which case $\mathbf{G}$ must be correctly formed for $\text{sid}_S, \text{hpw}, s_C$ (by $\mathcal{F}_{\text{NIZK}}[\mathcal{R}_C]$ soundness). Therefore, the algebra underlying $\text{CKEM}_{\text{Vand}}$ implies that, if $F \neq f_{\text{Vand}}^*(\text{sid}_S, \text{hpw}, s_C + s_S)$, then V'_0 and V'_1 are two uniformly random ElGamal ciphertexts.

The views of $\mathcal{Z}$ in $\mathbf{G}_5$ and $\mathbf{G}_6$ are identical.

$$\Delta(\mathbf{G}_5, \mathbf{G}_6) = 0$$

Game $\mathbf{G}_7$ (V'_0 and V'_1 are random when responding to honest clients): $\mathbf{G}_7$ is $\mathbf{G}_6$ with only one change: if $w_C = \perp$, then $\mathcal{S}$ samples $V'_0, V'_1 \leftarrow_{\S} \mathbb{G}^2$ instead of computing V'_0 and V'_1 as before.

Unlike the previous game change, this one does not result in an identical view. Nevertheless, since $\mathcal{S}$ should be computing V'_0 and V'_1 under a public key pk_C whose secret key is unknown to $\mathcal{Z}$, it should not be possible for $\mathcal{Z}$ to distinguish the replacement of V'_0 and V'_1 with random ciphertexts.

Given an environment $\mathcal{Z}$ that can distinguish $\mathbf{G}_6$ from $\mathbf{G}_7$, we construct a reduction $\mathcal{R}$ that can solve DDH. The input to $\mathcal{R}$ is tuple $(A, B, C) \in \mathbb{G}^3$. $\mathcal{R}$ emulates game $\mathbf{G}_6$ to $\mathcal{Z}$ with the following modifications:

- In $\mathbf{C}$, instead of sampling $\text{pk}_C \leftarrow_{\S} \mathbb{G}$, $\mathcal{R}$ samples $\delta \leftarrow_{\S} \mathbb{Z}_p$ and sets $\text{pk}_C := A \cdot g^\delta$. δ is recorded in association with proof π_C .
- In $\mathbf{S}$, instead of encrypting by $d_{V_i} \leftarrow_{\S} \mathbb{Z}_p$, $V_i \leftarrow \text{ElG.Enc}_{\text{pk}_C}(K; d_{V_i}) \cdot (\dots)$ (for $i \in \{0, 1\}$), $\mathcal{R}$ samples $\mu_i, \nu_i \leftarrow_{\S} \mathbb{Z}_p$ and sets $V_i := (g^{\mu_i} \cdot B^{\nu_i}, K \cdot (A^{\mu_i} \cdot C^{\nu_i}) \cdot (g^{\mu_i} \cdot B^{\nu_i})^\delta) \cdot (\dots)$, where δ is as recorded for π_C .

Again we utilize DDH randomized self-reduction with a random δ “shift” applied to each pk_C . If (A, B, C) is a Diffie-Hellman tuple, then the V_0, V_1 ciphertexts (and consequently the V'_0, V'_1 ciphertexts) are formed exactly as in $\text{CKEM}_{\text{Vand}}$ and the view of $\mathcal{Z}$ is identical to that in $\mathbf{G}_6$. If (A, B, C) is not a Diffie-Hellman tuple, then the V_0, V_1 ciphertexts (and consequently the V'_0, V'_1 ciphertexts) are independently random and the view of $\mathcal{Z}$ is identical to that in $\mathbf{G}_7$.

Reduction $\mathcal{R}$ outputs 1 iff $\mathcal{Z}$ outputs 1. Thus the advantage of $\mathcal{Z}$ at distinguishing between $\mathbf{G}_6$ and $\mathbf{G}_7$ is equal to the advantage of $\mathcal{R}$ in the DDH game, which is $\leq \text{Adv}_{\text{DDH}}$.

$$\Delta(\mathbf{G}_6, \mathbf{G}_7) \leq \text{Adv}_{\text{DDH}}$$

Game $\mathbf{G}_8$ (SK is random unless responding to successful dishonest clients): $\mathbf{G}_8$ is $\mathbf{G}_7$ with the following changes:

- In $\mathbf{S}$, if $w_C \neq \perp$, then parse $w_C \rightarrow (\text{hpw}, s_C, \cdot)$. If $F = f_{\text{Vand}}^*(\text{sid}_S, \text{hpw}, s_C + s_S)$, then proceed as before.

- Otherwise (i.e. $w_C = \perp$ or $F \neq f_{\text{Vand}}^*(\text{sid}_S, \text{hpw}, s_C + s_S)$), do not compute $d_K := H_d^{\text{sid}_S}(K)$ and $\text{SK} := H_{\text{SK}}^{\text{sid}_S}(\text{ssid}_S, K)$ as normal. Instead sample $d_K \leftarrow_{\mathbb{S}} \mathbb{Z}_p$ and $\text{SK} \leftarrow_{\mathbb{S}} \{0, 1\}^\kappa$. Also, record $(\text{ssid}_S, \text{SK})$ in association with proof π_S , and omit K from the truncated witness w_S (see G_5). Finally, add (sid_S, K) to set W .
- In C , if w_S is a truncated (i.e. simulated) witness (i.e. π_S was generated by an honest S , see G_5) and $F = f_{\text{Vand}}^*(\text{sid}_C, \text{hpw}, s_C + s_S)$ (see G_3), then retrieve the $(\text{ssid}_S, \text{SK})$ recorded for π_S . Instead of computing $\text{SK}' := H_{\text{SK}}^{\text{sid}_C}(\text{ssid}_C, K)$, if $\text{ssid}_C = \text{ssid}_S$ then set $\text{SK}' := \text{SK}$. Otherwise, sample $\text{SK} \leftarrow_{\mathbb{S}} \{0, 1\}^\kappa$.

Intuitively, the ideal adversary should learn nothing about the SK sampled by S unless S is responding to a dishonest client who supplies a successful guess of hpw, s_C . In this game change, we consider the cases that S is either responding to an honest client or to an unsuccessful dishonest client. In those cases, S in G_8 samples K and commits to it in C_K , but does nothing else with K . In particular, SK is an independently random value of which no information whatsoever is leaked to the ideal adversary.

Define KQUERIED to be the event that $\mathcal{Z}$ queries $H_d^{\text{sid}_S}(K)$ or $H_{\text{SK}}^{\text{sid}_S}(\text{ssid}, K)$ (with any ssid) for any $(\text{sid}_S, K) \in W$. Unless KQUERIED occurs, the views of $\mathcal{Z}$ in G_7 and G_8 are identical, because $\mathcal{Z}$ never observes that d_K and SK were not derived from H_d and H_{SK} . Thus, $\Delta(G_7, G_8) \leq \Pr[\text{KQUERIED}]$. Furthermore, the only information $\mathcal{Z}$ sees about K is the commitment C_K , so the hiding property of ElGamal commitment should imply that it is hard for $\mathcal{Z}$ to compute K and trigger KQUERIED . Note that $\Pr[\text{KQUERIED}]$ is equal in G_7 and G_8 , because the two games are identical up until the time KQUERIED occurs.

Given an environment $\mathcal{Z}$ that can trigger KQUERIED , we construct a reduction $\mathcal{R}$ that can solve DDH. The input to $\mathcal{R}$ is tuple $(A, B, C) \in \mathbb{G}^3$. $\mathcal{R}$ emulates game G_8 to $\mathcal{Z}$ with the following modifications:

- In S , if $w_C \neq \perp$, then $\mathcal{R}$ parses $w_C \rightarrow (\text{hpw}, s_C, \cdot)$. If $F = f_{\text{Vand}}^*(\text{sid}_S, \text{hpw}, s_C + s_S)$, then $\mathcal{R}$ proceeds as before.
- Otherwise (i.e. $w_C = \perp$ or $F \neq f_{\text{Vand}}^*(\text{sid}_S, \text{hpw}, s_C + s_S)$), instead of sampling $K \leftarrow_{\mathbb{S}} \mathbb{G}$, $C_K := \text{ElG.Commit}(K; d_K)$, $\mathcal{R}$ samples $\delta, \mu, \nu \leftarrow_{\mathbb{S}} \mathbb{Z}_p$, $c_3 \leftarrow_{\mathbb{S}} \mathbb{G}$, and sets $C_K := (A \cdot g^\delta, g^\mu \cdot B^\nu, c_3)$. Also, $\mathcal{R}$ computes $K^* := c_3 / (A^\mu \cdot C^\nu) \cdot (g^\mu \cdot B^\nu)^\delta$ and adds (sid_S, K^*) to set W .

Reduction $\mathcal{R}$ outputs 1 iff $\mathcal{Z}$ ever performs a query $H_d^{\text{sid}_S}(K^*)$ or $H_{\text{SK}}^{\text{sid}_S}(\text{ssid}, K^*)$ (with any ssid) for any $(\text{sid}_S, K^*) \in W$.

The views of $\mathcal{Z}$ in $\mathcal{R}$ and G_8 are identical. In both games, each C_K is a commitment to a random K that is never otherwise used (referring only to the cases of responding to an honest client or to an unsuccessful dishonest client). In G_8 , the game samples these K values and commits to them outright. In $\mathcal{R}$ the reduction uses (A, B, C) to create the commitments (again applying DDH randomized self-reduction).

If (A, B, C) is a Diffie-Hellman tuple, then the committed K is K^* , so the KQUERIED event is exactly the same as the condition for $\mathcal{R}$ to output 1. Thus,

$\Pr[1 \leftarrow \mathcal{R} \mid (A, B, C) \in DH] = \Pr[\text{KQUERIED}]$. If (A, B, C) is not a Diffie-Hellman tuple, the committed K and K^* are independently random, and no information about K^* is exposed to $\mathcal{Z}$. At most q_S (sid_S, K^*) tuples are recorded, so $\Pr[1 \leftarrow \mathcal{R} \mid (A, B, C) \notin DH] \leq (q_S q_H)/p$. The absolute difference between these two probabilities is $\leq \text{Adv}_{\text{DDH}}$.

$$\Delta(\mathbf{G}_7, \mathbf{G}_8) \leq \text{Adv}_{\text{DDH}} + \frac{q_S q_H}{p}$$

Game $\mathbf{G}_9$ ($\mathbf{H}_{\text{SK}}$ is programmed with SK when responding to successful dishonest clients): $\mathbf{G}_9$ is $\mathbf{G}_8$ with the following changes to $\mathbf{S}$:

- If $w_C = \perp$, then proceed as before.
- Otherwise, parse $w_C \rightarrow (\text{hpw}, s_C, \cdot)$. If $F \neq f_{\text{Vand}}^*(\text{sid}_S, \text{hpw}, s_C + s_S)$, then still proceed as before.
- Otherwise, do not compute C_K , V'_0 , and V'_1 as normal. Instead, sample $\text{SK} \leftarrow_{\mathbb{S}} \{0, 1\}^\kappa$, $K'_0, K'_1 \leftarrow_{\mathbb{S}} \mathbb{G}$, and $\beta \leftarrow_{\mathbb{S}} \{0, 1\}$. Then, compute $V'_0 \leftarrow \text{ElG.Enc}_{\text{pk}_C}(K'_0)$, $V'_1 \leftarrow \text{ElG.Enc}_{\text{pk}_C}(K'_1)$, and $C_K \leftarrow \text{ElG.Commit}(K'_\beta; \text{H}_d^{\text{sid}_S}(K'_\beta))$.

Finally, program $\mathbf{H}_{\text{SK}}^{\text{sid}_S}(\text{ssid}_S, K'_\beta) := \text{SK}$.

The change from $\mathbf{G}_8$ to $\mathbf{G}_9$ is mostly conceptual and merely shifts the order in which values are sampled and computed. The only actual difference from the previous game is that SK is sampled independently of K and then programmed into $\mathbf{H}_{\text{SK}}$. Consequently, the views of $\mathcal{Z}$ in $\mathbf{G}_8$ and $\mathbf{G}_9$ are identical unless $\mathcal{Z}$ queries $\mathbf{H}_{\text{SK}}^{\text{sid}_S}(\text{ssid}_S, K'_\beta)$ before the game samples $K'_\beta \leftarrow_{\mathbb{S}} \mathbb{G}$ and programs it. Only one $\mathbf{S}$ session runs per ssid_S , so only one $\mathbf{H}_{\text{SK}}$ point is programmed per ssid_S , and the environment's probability of having guessed one of them is negligible.

$$\Delta(\mathbf{G}_8, \mathbf{G}_9) \leq \frac{q_H}{p}$$

Game $\mathbf{G}_{10}$ (the simulated world): The change from $\mathbf{G}_9$ to $\mathbf{G}_{10}$ is purely conceptual. All interactions between the simulator and the honest parties now match the code of $\mathcal{F}_{\text{CKEM}}$, so one can imagine the monolithic simulator now cleanly split into the two components $\mathcal{F}_{\text{CKEM}}$ and $\text{SIM}_{\text{CKEM-Vand}}$.

$$\Delta(\mathbf{G}_9, \mathbf{G}_{10}) = 0$$

$$\text{Exp}_{\mathcal{Z}}[\mathbf{G}_{10}] = \text{Ideal}_{\mathcal{Z}}[\mathcal{F}_{\text{CKEM}}, \text{SIM}_{\text{CKEM-Vand}}]$$

Summing up the distinguishing advantage bounds for each incremental game change yields an overall bound on $\mathcal{Z}$'s distinguishing advantage between the real and simulated worlds.

$$\Delta(\mathbf{G}_0, \mathbf{G}_{10}) \leq 3 \cdot \text{Adv}_{\text{DDH}} + \frac{q_C^2 + q_S^2 + q_S q_H + q_H}{p}$$

$p = |\mathbb{G}|$ is an exponential function of the security parameter. If $\mathcal{Z}$ is efficient, then q_C , q_S , and q_H are polynomial functions of the security parameter. Furthermore, Adv_{DDH} is a negligible function of the security parameter under the DDH assumption. Thus, the distinguishing advantage of any efficient $\mathcal{Z}$ between the real and simulated worlds is negligible.

Protocol $\text{CKEM}_{\text{Vand}}$ realizes $\mathcal{F}_{\text{CKEM}}[\text{SSEM}[f_{\text{Vand}}^*, \text{ElG}], \text{SSEM}_{\text{Adv}}[f_{\text{Vand}}^*]]$.

H Proof of CKEM_{PoNR} Security

In this section we prove Theorem 9, i.e. that protocol CKEM_{PoNR} realizes

$$\mathcal{F}_{\text{CKEM}}[\text{SSEM}[f_{\text{PoNR}}^*, \text{EIG}], \text{SSEM}_{\text{Adv}}[f_{\text{PoNR}}^*]].$$

Specifically, for any efficient adversary $\mathcal{A}^*$ against protocol CKEM_{PoNR}, there exists a simulator SIM such that no efficient environment $\mathcal{Z}$ can distinguish the view of $\mathcal{A}^*$ interacting with the real CKEM_{PoNR} protocol and the view of SIM interacting with the ideal functionality $\mathcal{F}_{\text{CKEM}}[\text{SSEM}[f_{\text{PoNR}}^*, \text{EIG}], \text{SSEM}_{\text{Adv}}[f_{\text{PoNR}}^*]]$ with advantage better than $3 \cdot \text{Adv}_{\text{DDH}} + \text{Adv}_{\text{DL}} + \frac{qc^2 + qs^2 + qsq_H + q_H}{p}$ where there are at most q_C client sessions, there are at most q_S server sessions, there are at most q_H queries to H_d and H_{SK} , and Adv_{DDH} (resp. Adv_{DL}) is a bound on the DDH (resp. DL) advantage of any efficient algorithm.

Proof. For any adversary $\mathcal{A}^*$, we construct simulator $\text{SIM}_{\text{CKEM-PoNR}}$ as shown in Figures 16 and 17. Without loss of generality, we assume that $\mathcal{A}^*$ is a “dummy” adversary that merely passes messages to and from the environment $\mathcal{Z}$.

We now show that, for any efficient (i.e. PPT) $\mathcal{Z}$, the distinguishing advantage of $\mathcal{Z}$ between the real and simulated worlds is negligible. We denote the probability that $\mathcal{Z}$ outputs 1 in the interaction with the real world game or the simulated game as resp. $\mathbf{Real}_{\mathcal{Z}}[\text{CKEM}_{\text{PoNR}}]$ and $\mathbf{Ideal}_{\mathcal{Z}}[\mathcal{F}_{\text{CKEM}}, \text{SIM}_{\text{CKEM-PoNR}}]$. We denote the probability that $\mathcal{Z}$ outputs 1 in an interaction with security game G_i as $\text{Exp}_{\mathcal{Z}}[G_i] = \Pr[1 \leftarrow \mathcal{Z}^{G_i}(1^\kappa)]$, and we define $\Delta(G_i, G_j) = |\text{Exp}_{\mathcal{Z}}[G_i] - \text{Exp}_{\mathcal{Z}}[G_j]|$. The argument proceeds by a series of game changes, starting with game G_0 s.t. $\text{Exp}_{\mathcal{Z}}[G_0] = \mathbf{Real}_{\mathcal{Z}}[\text{CKEM}_{\text{PoNR}}]$ and ends with game G_{12} s.t. $\text{Exp}_{\mathcal{Z}}[G_{12}] = \mathbf{Ideal}_{\mathcal{Z}}[\mathcal{F}_{\text{CKEM}}, \text{SIM}_{\text{CKEM-PoNR}}]$. We must show that $\Delta(G_0, G_{12})$ is negligible.

Game G_0 (the real world): The distinguisher $\mathcal{Z}$ interacts with CKEM_{PoNR} (Figure 8) in the role of the honest parties and in the role of the adversary. Functionalities $\mathcal{F}_{\text{COT}}$ (Figure 9) and $\mathcal{F}_{\text{NIZK}}$ (Figure 1) interact with CKEM_{PoNR} and with $\mathcal{Z}$ via the adversarial interface. H_x , H_d , and H_{SK} are true random oracles.

$$\text{Exp}_{\mathcal{Z}}[G_0] = \mathbf{Real}_{\mathcal{Z}}[\text{CKEM}_{\text{PoNR}}]$$

Game G_1 (statements never collide): G_1 is G_0 with the following changes:

- If two C sessions ever pick the same pk_C , the game immediately aborts.
- If two S sessions ever pick the same K , the game immediately aborts.

Per the code of $\mathcal{F}_{\text{NIZK}}$, it is possible for a single statement-proof pair (x, π) to correspond to more than one witness w iff statement x is proven more than once by honest parties. This game change ensures that the x_C and x_S statements proven by honest parties never repeat. Therefore, in future games, we can retrieve the one w for any (x, π) without ambiguity.

Every pk_C (resp. K) is sampled uniformly at random from $\mathbb{G}$. Therefore, the probability of a collision is upper-bounded by qc^2/p (resp. qs^2/p). If no collision occurs, G_0 and G_1 are identical.

$$\Delta(G_0, G_1) \leq \frac{qc^2 + qs^2}{p}$$

- Program H_d and H_{SK} outputs uniformly at random, except where special H_{SK} programming is noted below.
- On $(\text{CEVAL}, \text{ssid}_C, C, (\text{sid}_C, C_F))$ from $\mathcal{F}_{\text{CKEM}}$:
 - Sample $\text{pk}_C \leftarrow_{\mathbb{G}} \mathbb{G}$, $\mathbf{G} \leftarrow_{\mathbb{G}} (\mathbb{G}^2)^N$, and $\hat{G} \leftarrow_{\mathbb{G}} \mathbb{G}^2$.
 - Sample $\mathbf{m} \leftarrow_{\mathbb{G}} ((\mathbb{Z}_p^N)^t)^2$ and $\mathbf{d}_m \leftarrow_{\mathbb{G}} ((\mathbb{Z}_p)^t)^2$. For each $j \in [t]$, compute $C_{m,j,0} \leftarrow \text{Ped}^N((m_{i,j,0})_{i \in [N]}; d_{m,j,0})$ and $C_{m,j,1} \leftarrow \text{Ped}^N((m_{i,j,1})_{i \in [N]}; d_{m,j,1})$.[†] Define $\mathbf{C}_m \leftarrow (C_{m,j,0}, C_{m,j,1})_{j \in [t]}$.
 - Send $(\text{PROVE}, C, \text{ssid}_C, (\text{sid}_C, \text{pk}_C, \mathbf{G}, \hat{G}, \mathbf{C}_m))$ to $\mathcal{A}^*$ as if from $\mathcal{F}_{\text{NIZK}}[\mathcal{R}_C]$, await response $(\text{PROOF}, \text{ssid}_C, \pi_C)$, and save record $(\mathcal{R}_C, (\text{sid}_C, \text{pk}_C, \mathbf{G}, \hat{G}, \mathbf{C}_C), \perp, \pi_C, \text{ssid}_C)$.
 - Send $(\text{sid}_C, \text{pk}_C, \mathbf{G}, \hat{G}, \pi_C)$ to $\mathcal{A}^*$ as if from C to S .
 - For each $j \in [t]$, save record $(\text{SEND}, (\text{ssid}_C|j), (m_{i,j,0})_{i \in [N]}, d_{m,j,0}, (m_{i,j,1})_{i \in [N]}, d_{m,j,1})$ marked fresh and send $(\text{SEND}, (\text{ssid}_C|j), C, C_{m,j,0}, C_{m,j,1})$ to $\mathcal{A}^*$ as if from $\mathcal{F}_{\text{COT}}$.
 - Await (C_K, V, π_S) from $\mathcal{A}^*$ to C .
 - Define $\mathbf{x}_S := (C_F, \text{pk}_C, \mathbf{G}, \hat{G}, \mathbf{C}_m, C_K, V)$. If there exists no record $(\mathcal{R}_S, \mathbf{x}_S, \mathbf{w}_S, \pi_S, \text{ssid}_S)$ then send $(\text{VERIFY}, C, \text{ssid}_C, \mathbf{x}_S, \pi_S)$ to $\mathcal{A}^*$ as if from $\mathcal{F}_{\text{NIZK}}[\mathcal{R}_S]$, await response $(\text{WITNESS}, \text{ssid}_C, \mathbf{w}_S)$, and save record $(\mathcal{R}_S, \mathbf{x}_S, \mathbf{w}_S, \pi_S, \perp)$.
 - If $\mathbf{w}_S \neq \perp$ then parse $\mathbf{w}_S \rightarrow (F, d_F, s_S, \cdot, \cdot, K, d_K, \cdot)$, and...
 - * If $(\mathbf{x}_S, \mathbf{w}_S) \notin \mathcal{R}_S \vee H_d^{\text{sid}_C}(K) \neq d_K$: Send $(\text{ADVSEVAL}, \text{ssid}_C, \perp, 0^\kappa)$ to $\mathcal{F}_{\text{CKEM}}$.
 - * Else: Send $(\text{ADVSEVAL}, \text{ssid}_C, (\text{sid}_C, F, s_S, d_F), H_{SK}^{\text{sid}_C}(\text{ssid}_C, K))$ to $\mathcal{F}_{\text{CKEM}}$.
 - If $\mathbf{w}_S = \perp$, then...
 - * If $\text{ssid}_C \neq \text{ssid}_S$: Send $(\text{ADVSEVAL}, \text{ssid}_C, \perp, 0^\kappa)$ to $\mathcal{F}_{\text{CKEM}}$.
 - * Else: Send $(\text{COUT}, \text{ssid}_C)$ to $\mathcal{F}_{\text{CKEM}}$.
- On $(\text{SEVAL}, \text{ssid}_S, S, (\text{sid}_S, C_F))$ from $\mathcal{F}_{\text{CKEM}}$:
 - Await $(\text{sid}_C, \text{pk}_C, \mathbf{G}, \hat{G}, \pi_C)$ from $\mathcal{A}^*$ to S .
 - For each $j \in [t]$, send $(\text{RCV}, (\text{ssid}_S|j), S)$ to $\mathcal{A}^*$ as if from $\mathcal{F}_{\text{COT}}$.
 - For each $j \in [t]$, await $(\text{COMPLETE}, \text{ssid}'_j, (\text{ssid}_S|j))$ from $\mathcal{A}^*$ to $\mathcal{F}_{\text{COT}}$ where there exists record $(\text{SEND}, \text{ssid}'_j, (m_{i,j,0})_{i \in [N]}, d_{m,j,0}, (m_{i,j,1})_{i \in [N]}, d_{m,j,1})$ marked fresh. Mark it completed.
 - Define $\mathbf{C}_m \leftarrow (\text{Ped}^N((m_{i,j,0})_{i \in [N]}; d_{m,j,0}), \text{Ped}^N((m_{i,j,1})_{i \in [N]}; d_{m,j,1}))_{j \in [t]}$.
 - Define $\mathbf{x}_C := (\text{sid}_S, \text{pk}_C, \mathbf{G}, \hat{G}, \mathbf{C}_m)$. If there exists no record $(\mathcal{R}_C, \mathbf{x}_C, \mathbf{w}_C, \pi_C, \text{ssid}_C)$ then send $(\text{VERIFY}, S, \text{ssid}_S, \mathbf{x}_C, \pi_C)$ to $\mathcal{A}^*$ as if from $\mathcal{F}_{\text{NIZK}}[\mathcal{R}_C]$, await response $(\text{WITNESS}, \text{ssid}_S, \mathbf{w}_C)$, and save record $(\mathcal{R}_C, \mathbf{x}_C, \mathbf{w}_C, \pi_C, \perp)$.[†]
 - If $\text{sid}_C \neq \text{sid}_S \vee (\mathbf{w}_C \neq \perp \wedge (\mathbf{x}_C, \mathbf{w}_C) \notin \mathcal{R}_C)$: Send $(\text{ADVCEVAL}, \text{ssid}_S, \perp)$ to $\mathcal{F}_{\text{CKEM}}$, receive response $(\text{COUT}, \text{ssid}_S, \perp)$, and break (i.e. don't run the following code).
 - If $\mathbf{w}_C \neq \perp$: Parse $\mathbf{w}_C \rightarrow (\text{hpw}, s_C, \cdot, \cdot, \cdot, \cdot)$, send $(\text{ADVCEVAL}, \text{ssid}_S, (\text{sid}_S, \text{hpw}, s_C))$ to $\mathcal{F}_{\text{CKEM}}$, and receive response $(\text{COUT}, \text{ssid}_S, \text{SK})$.
 - If $\mathbf{w}_C = \perp$ then define $\text{SK} := \perp$, and...
 - * If $\text{ssid}_S \neq \text{ssid}_C$: Send $(\text{ADVCEVAL}, \text{ssid}_S, \perp)$ to $\mathcal{F}_{\text{CKEM}}$ and receive response $(\text{COUT}, \text{ssid}_S, \perp)$.
 - * Else: Send $(\text{SOUT}, \text{ssid}_S)$ to $\mathcal{F}_{\text{CKEM}}$.
 - Sample $K' \leftarrow_{\mathbb{G}} \mathbb{G}$, and compute $V \leftarrow \text{ElG.Enc}_{\text{pk}_C}(K')$.
 - If $\text{SK} \neq \perp$ then compute $C_K \leftarrow \text{ElG.Commit}(K'; H_d^{\text{sid}_S}(K'))$, and program $H_{SK}^{\text{sid}_S}(\text{ssid}_S, K') := \text{SK}$. Else $(\text{SK} \stackrel{\text{def}}{=} \perp)$ sample $C_K \leftarrow_{\mathbb{G}} \mathbb{G}^3$.
 - Define $\mathbf{x}_S := (C_F, \text{pk}_C, \mathbf{G}, \hat{G}, \mathbf{C}_m, C_K, V)$. Send $(\text{PROVE}, S, \text{ssid}_S, \mathbf{x}_S)$ to $\mathcal{A}^*$ as if from $\mathcal{F}_{\text{NIZK}}[\mathcal{R}_S]$, await response $(\text{PROOF}, \text{ssid}_S, \pi_S)$, and save record $(\mathcal{R}_S, \mathbf{x}_S, \perp, \pi_S, \text{ssid}_S)$.
 - Send (C_K, V, π_S) to $\mathcal{A}^*$ as if from S to C .

Fig. 16. $\text{SIM}_{\text{CKEM-PoNR}}$: Simulator for protocol $\text{CKEM}_{\text{PoNR}}$, part 1: Honest C and S

- On (SEND, ssid, m_0, d_0, m_1, d_1) from $\mathcal{A}^*$ to $\mathcal{F}_{\text{COT}}^\dagger$:
 - Save record (SEND, ssid, m_0, d_0, m_1, d_1) marked **fresh**.
 - Send (SEND, ssid, $\mathcal{A}^*$, $\text{Ped}^N(m_0; d_0), \text{Ped}^N(m_1; d_1)$) to $\mathcal{A}^*$ as if from $\mathcal{F}_{\text{COT}}$.
- On (RCV, ssid, σ) from $\mathcal{A}^*$ to $\mathcal{F}_{\text{COT}}$:
 - Send (RCV, ssid, $\mathcal{A}^*$) to $\mathcal{A}^*$ as if from $\mathcal{F}_{\text{COT}}$.
 - Await (COMPLETE, ssid', ssid) from $\mathcal{A}^*$ to $\mathcal{F}_{\text{COT}}$ where there exists record (SEND, ssid', m_0, d_0, m_1, d_1) marked **fresh**. Mark it **completed**.
 - Send (COMPLETE, ssid, $m_\sigma, d_\sigma, \text{Ped}^N(m_{1-\sigma}, d_{1-\sigma})$) to $\mathcal{A}^*$ as if from $\mathcal{F}_{\text{COT}}$.

If $\mathcal{Z}$ ever supplies two different openings $C = \text{Ped}^N(m; d) = \text{Ped}^N(m^*; d^*)$, $m \neq m^*$ to a Ped^N commitment, immediately abort the game. There are three particular avenues by which these commitment openings could arise, and they are marked by symbol † in the simulator. First, honest clients produce and output Ped^N commitments. Second, $\mathcal{Z}$ supplies commitment openings to $\mathcal{F}_{\text{COT}}$ when acting as a COT sender. Third, $\mathcal{Z}$ supplies commitment openings in $\mathcal{R}_C$ witnesses, each of which includes (for every $j \in [t], \sigma \in \{0, 1\}$) a decommitment $d_{m_j, \sigma}$, as well as $s_C[j], (\delta_{i,j})_{i \in [N]}$, which imply message $(m_{i,j, \sigma})_{i \in [N]}$.

Fig. 17. $\text{SIM}_{\text{CKEM-PoNR}}$: Simulator for protocol $\text{CKEM}_{\text{PoNR}}$, part 2: Adversarial $\mathcal{F}_{\text{COT}}$ interactions

Game G_2 (client NIZKs are simulated): G_2 is G_1 with only one change: w_C is omitted when C calls $\mathcal{F}_{\text{NIZK}}[\mathcal{R}_C].\text{PROVE}$. Since honest clients never attempt to prove false statements, the check for $(x, w) \in \mathcal{R}_C$ is simply skipped, and instead of a witness a special symbol $w := \perp$ is recorded to represent that the produced proof is correct. The code of $\mathcal{F}_{\text{NIZK}}[\mathcal{R}_C].\text{VERIFY}$ is modified to always accept proofs for records with $w = \perp$.

The views of $\mathcal{Z}$ in G_1 and G_2 are identical.

$$\Delta(G_1, G_2) = 0$$

Game G_3 (clients do not decrypt V): G_3 is G_2 with the following changes to C :

- Instead of sampling $(sk_C, pk_C) \leftarrow \text{ElG.KG}$, only sample $pk_C \leftarrow_{\$} \mathbb{G}$.
- After verifying π_S , do not decrypt V . Instead, parse $w_S \rightarrow (F, \cdot, s_S, \cdot, \cdot, K, d_K, \cdot)$. If $H_d^{\text{sidc}}(K) = d_K \wedge F = f_{\text{PoNR}}^*(\text{sid}_C, \text{hpw}, s_C \oplus s_S)$, then output $\text{SK}' := H_{\text{SK}}^{\text{sidc}}(\text{ssid}_C, K)$. Otherwise, output $\text{SK}' \leftarrow_{\$} \{0, 1\}^\kappa$.

Notice that C 's changed code only runs if π_S verifies. $\mathcal{R}_S$ guarantees that the received V and C_K are consistent with correct server behavior on w_S , e.g. $C_K = \text{ElG.Commit}(K; d_K)$. Therefore it is not necessary for C to refer to V and C_K directly. In the previous game, C will decrypt K from V iff $F = f_{\text{PoNR}}^*(\text{sid}_C, \text{hpw}, s_C + s_S)$, and if additionally $H_d^{\text{sidc}}(K) = d_K$ then C will use K to derive SK' . If either of those conditions does not occur, then C will output a random SK' . The behavior of C in this game is exactly the same.

The views of $\mathcal{Z}$ in G_2 and G_3 are identical.

$$\Delta(G_2, G_3) = 0$$

Game G_4 (G and $\hat{G}$ are random): G_4 is G_3 with only one change: instead of computing $(g_i)_{i \in [N]}$, $G = (G_i)_{i \in [N]}$, $\hat{g}$, and $\hat{G}$ as in $\text{CKEM}_{\text{PoNR}}$, C simply samples $G \leftarrow_{\S} (\mathbb{G}^2)^N$ and $\hat{G} \leftarrow_{\S} \mathbb{G}^2$.

Recall that client NIZKs are simulated, so it is not a problem that $\mathbb{x}_C$ no longer has a correct witness. Additionally, the response V that S computes from G and $\hat{G}$ is not decrypted and used by C , so it is not a problem that these responses are formed from random G_i and $\hat{G}$ ciphertexts.

Given an environment $\mathcal{Z}$ that can distinguish G_3 from G_4 , we construct a reduction $\mathcal{R}$ that can solve DDH. The input to $\mathcal{R}$ is tuple $(A, B, C) \in \mathbb{G}^3$. $\mathcal{R}$ emulates game G_3 to $\mathcal{Z}$ with the following modifications to the code of C :

- Instead of sampling $\text{pk}_C \leftarrow_{\S} \mathbb{G}$, $\mathcal{R}$ samples $\delta \leftarrow_{\S} \mathbb{Z}_p$ and sets $\text{pk}_C := A \cdot g^\delta$.
- Instead of encrypting by $d_{G_i} \leftarrow_{\S} \mathbb{Z}_p$, $G_i \leftarrow \text{ElG.Enc}_{\text{pk}_C}(g_i; d_{G_i})$, $\mathcal{R}$ samples $\mu, \nu \leftarrow_{\S} \mathbb{Z}_p$ and sets $G_i := (g^\mu \cdot B^\nu, g_i \cdot (A^\mu \cdot C^\nu) \cdot (g^\mu \cdot B^\nu)^\delta)$.
- Instead of encrypting by $d_{\hat{G}} \leftarrow_{\S} \mathbb{Z}_p$, $\hat{G} \leftarrow \text{ElG.Enc}_{\text{pk}_C}(\hat{g}; d_{\hat{G}})$, $\mathcal{R}$ samples $\mu, \nu \leftarrow_{\S} \mathbb{Z}_p$ and sets $\hat{G} := (g^\mu \cdot B^\nu, \hat{g} \cdot (A^\mu \cdot C^\nu) \cdot (g^\mu \cdot B^\nu)^\delta)$.

Here we apply the well-known DDH randomized self-reduction [27,22]. If (A, B, C) is a Diffie-Hellman tuple, then $(A, g^\mu \cdot B^\nu, A^\mu \cdot C^\nu)$ is a random Diffie-Hellman tuple. If not, then it is a uniformly random tuple. For every C session, we apply a “shift” of δ to get a freshly random pk_C . If (A, B, C) is a Diffie-Hellman tuple, then each G_i (resp. $\hat{G}$) ciphertext is a correct ElGamal encryption of g_i (resp. $\hat{g}$) under pk_C , so the view of $\mathcal{Z}$ is identical to that in G_3 . If (A, B, C) is not a Diffie-Hellman tuple, then each G_i and $\hat{G}$ ciphertext is independently random, so the view of $\mathcal{Z}$ is identical to that in G_4 .

Reduction $\mathcal{R}$ outputs 1 iff $\mathcal{Z}$ outputs 1. Thus, the advantage of $\mathcal{Z}$ at distinguishing between G_3 and G_4 is equal to the advantage of $\mathcal{R}$ in the DDH game, which is $\leq \text{Adv}_{\text{DDH}}$.

$$\Delta(G_3, G_4) \leq \text{Adv}_{\text{DDH}}$$

Game G_5 (m is random): G_5 is G_4 with only one change: instead of computing $m_{i,j,0}$ and $m_{i,j,1}$ (for all $i \in [N], j \in [t]$) as in $\text{CKEM}_{\text{PoNR}}$, C simply samples $m \leftarrow_{\S} ((\mathbb{Z}_p^N)^t)^2$.

As in the previous game change, we recall that client NIZKs are simulated, so it is not a problem that $\mathbb{x}_C$ no longer has a correct witness. Additionally, the response V that S computes from the received m values is not decrypted and used by C , so it is not a problem that these m values are random.

In this game, $\mathcal{Z}$ sees a Pedersen commitment to $(m_{i,j,0})_{i \in [N]}$ and $(m_{i,j,1})_{i \in [N]}$ for every $j \in [t]$. If playing the role of a dishonest server, $\mathcal{Z}$ then sees an opening for exactly one of the two commitments. Pedersen commitments (Section 2) are perfectly hiding, and for any one revealed $(m_{i,j,\sigma})_{i \in [N]}$ there exists $(\delta_{i,j})_{i \in [N]}$ that could have been chosen in G_4 to produce an identical view.

The views of $\mathcal{Z}$ in G_4 and G_5 are identical.

$$\Delta(G_4, G_5) = 0$$

Game G_6 (C_m commitments are perfectly binding): G_6 is G_5 with only one change: if $\mathcal{Z}$ ever supplies two different openings $C = \text{Ped}^N(m; d) = \text{Ped}^N(m^*; d^*)$, $m \neq m^*$ to a Ped^N commitment, immediately abort the game. There are three particular avenues by which these commitment openings could arise. First, honest clients produce and output Ped^N commitments. Second, $\mathcal{Z}$ supplies commitment openings to $\mathcal{F}_{\text{COT}}$ when acting as a COT sender. Third, $\mathcal{Z}$ supplies commitment openings in $\mathcal{R}_C$ witnesses, each of which includes (for every $j \in [t]$, $\sigma \in \{0, 1\}$) a decommitment $d_{m_j, \sigma}$, as well as $s_C[j]$, $(\delta_{i,j})_{i \in [N]}$, which imply message $(m_{i,j,\sigma})_{i \in [N]}$.

Future games will assume that the OT messages m^S received by S are consistent with the witness w_C of the corresponding client proof π_C . $\mathcal{F}_{\text{NIZK}}$ guarantees the soundness of π_C , but if $\mathcal{Z}$ provides different openings of a commitment to $\mathcal{F}_{\text{NIZK}}$ and $\mathcal{F}_{\text{COT}}$, then these future games will not behave correctly.

Unless this binding-break abort occurs, the views of $\mathcal{Z}$ in G_5 and G_6 are identical. Given an environment $\mathcal{Z}$ that can trigger this abort, we construct a reduction $\mathcal{R}$ that can solve DL. $\mathcal{R}$ is the straightforward and well-known “DL relation” reduction (e.g. [18]) that embeds the DL challenge in the Ped^N bases $h_1, \dots, h_N$. The probability of this reduction’s success is $\leq \text{Adv}_{\text{DL}}$.

$$\Delta(G_5, G_6) \leq \text{Adv}_{\text{DL}}$$

Game G_7 (server NIZKs are simulated): G_7 is G_6 with the following changes:

- w_S is truncated to $(F, \cdot, s_S, \cdot, \cdot, \cdot, K, \cdot, \cdot)$ when S calls $\mathcal{F}_{\text{NIZK}}[\mathcal{R}_S].\text{PROVE}$. Since honest servers never attempt to prove false statements, the check for $(x, w) \in \mathcal{R}_S$ is simply skipped, and the code of $\mathcal{F}_{\text{NIZK}}[\mathcal{R}_S].\text{VERIFY}$ is modified to always accept proofs with witnesses of this truncated form.
- In the previous game, C parses F , s_S , K , and d_K from w_S (see G_3), but d_K is now missing from the recorded witnesses of honestly generated server proofs. However, d_K is only used to check whether $H_d^{\text{sidc}}(K) = d_K$, and this condition always holds for honest servers, so the code of C is modified to simply skip that check when handling a witness of this truncated form.

The views of $\mathcal{Z}$ in G_6 and G_7 are identical.

$$\Delta(G_6, G_7) = 0$$

Game G_8 (V is random when responding to unsuccessful dishonest clients): G_8 is G_7 with the following changes to S after verifying π_C and checking that $\text{sid}_C = \text{sid}_S$:

- If $w_C = \perp$ (i.e. π_C was generated by an honest C , see G_2), then proceed as before.

- Otherwise, parse $w_C \rightarrow (\text{hpw}, s_C, \cdot, \cdot, \cdot, \cdot)$. If $F = f_{\text{PoNR}}^*(\text{sid}_S, \text{hpw}, s_C \oplus s_S)$, then still proceed as before.
- Otherwise, do not compute V as normal. Instead, simply sample $V \leftarrow_{\$} \mathbb{G}^2$.

Recall that server NIZKs are simulated, so it is not a problem that S no longer knows a witness for x_S . Furthermore, we are only modifying the behavior of S in the case that $w_C \neq \perp$, in which case G , $\hat{G}$, and m^S must be correctly formed for $\text{sid}_S, \text{hpw}, s_C$ (by $\mathcal{F}_{\text{NIZK}}[\mathcal{R}_C]$ soundness and C_m binding, see G_6). Therefore, the algebra underlying $\text{CKEM}_{\text{PoNR}}$ implies that, if $F \neq f_{\text{PoNR}}^*(\text{sid}_S, \text{hpw}, s_C \oplus s_S)$, then V is a uniformly random ElGamal ciphertext.

Thus, the views of $\mathcal{Z}$ in G_7 and G_8 are identical.

$$\Delta(G_7, G_8) = 0$$

Game G_9 (V is random when responding to honest clients): G_9 is G_8 with only one change: if $w_C = \perp$, then S samples $V \leftarrow_{\$} \mathbb{G}^2$ instead of computing V as before.

Since S should be computing V under a public key pk_C whose secret key is unknown to $\mathcal{Z}$, it should not be possible for $\mathcal{Z}$ to distinguish the replacement of V with a random ciphertext.

Given an environment $\mathcal{Z}$ that can distinguish G_8 from G_9 , we construct a reduction $\mathcal{R}$ that can solve DDH. The input to $\mathcal{R}$ is tuple $(A, B, C) \in \mathbb{G}^3$. $\mathcal{R}$ emulates game G_8 to $\mathcal{Z}$ with the following modifications:

- In C , instead of sampling $\text{pk}_C \leftarrow_{\$} \mathbb{G}$, $\mathcal{R}$ samples $\delta \leftarrow_{\$} \mathbb{Z}_p$ and sets $\text{pk}_C := A \cdot g^\delta$. δ is recorded in association with proof π_C .
- In S , instead of encrypting by $d_V \leftarrow_{\$} \mathbb{Z}_p$, $V \leftarrow \text{ElG.Enc}_{\text{pk}_C}(K; d_V) \cdot (\dots)$, $\mathcal{R}$ samples $\mu, \nu \leftarrow_{\$} \mathbb{Z}_p$ and sets $V := (g^\mu \cdot B^\nu, K \cdot (A^\mu \cdot C^\nu) \cdot (g^\mu \cdot B^\nu)^\delta) \cdot (\dots)$, where δ is as recorded for π_C .

Again we utilize DDH randomized self-reduction with a random δ “shift” applied to each pk_C . If (A, B, C) is a Diffie-Hellman tuple, then ciphertext V is formed exactly as in $\text{CKEM}_{\text{PoNR}}$ and the view of $\mathcal{Z}$ is identical to that in G_8 . If (A, B, C) is not a Diffie-Hellman tuple, then ciphertext V is independently random and the view of $\mathcal{Z}$ is identical to that in G_9 .

Reduction $\mathcal{R}$ outputs 1 iff $\mathcal{Z}$ outputs 1. Thus the advantage of $\mathcal{Z}$ at distinguishing between G_8 and G_9 is equal to the advantage of $\mathcal{R}$ in the DDH game, which is $\leq \text{Adv}_{\text{DDH}}$.

$$\Delta(G_8, G_9) \leq \text{Adv}_{\text{DDH}}$$

Game G_{10} (SK is random unless responding to successful dishonest clients): G_{10} is G_9 with the following changes:

- In S , if $w_C \neq \perp$, then parse $w_C \rightarrow (\text{hpw}, s_C, \cdot, \cdot, \cdot, \cdot)$. If $F = f_{\text{PoNR}}^*(\text{sid}_S, \text{hpw}, s_C \oplus s_S)$, then proceed as before.

- Otherwise (i.e. $w_C = \perp$ or $F \neq f_{\text{PoNR}}^*(\text{sid}_S, \text{hpw}, s_C \oplus s_S)$), do not compute $d_K := H_d^{\text{sid}_S}(K)$ and $\text{SK} := H_{\text{SK}}^{\text{sid}_S}(\text{ssid}_S, K)$ as normal. Instead sample $d_K \leftarrow_{\mathbb{S}} \mathbb{Z}_p$ and $\text{SK} \leftarrow_{\mathbb{S}} \{0, 1\}^\kappa$. Also, record $(\text{ssid}_S, \text{SK})$ in association with proof π_S , and omit K from the truncated witness w_S (see G_7). Finally, add (sid_S, K) to set W .
- In C , if w_S is a truncated (i.e. simulated) witness (i.e. π_S was generated by an honest S , see G_7) and $F = f_{\text{PoNR}}^*(\text{sid}_C, \text{hpw}, s_C \oplus s_S)$ (see G_3), then retrieve the $(\text{ssid}_S, \text{SK})$ recorded for π_S . Instead of computing $\text{SK}' := H_{\text{SK}}^{\text{sid}_C}(\text{ssid}_C, K)$, if $\text{ssid}_C = \text{ssid}_S$ then set $\text{SK}' := \text{SK}$. Otherwise, sample $\text{SK} \leftarrow_{\mathbb{S}} \{0, 1\}^\kappa$.

Intuitively, the ideal adversary should learn nothing about the SK sampled by S unless S is responding to a dishonest client who supplies a successful guess of hpw, s_C . In this game change, we consider the cases that S is either responding to an honest client or to an unsuccessful dishonest client. In those cases, S in G_{10} samples K and commits to it in C_K , but does nothing else with K . In particular, SK is an independently random value of which no information whatsoever is leaked to the ideal adversary.

Define KQUERIED to be the event that $\mathcal{Z}$ queries $H_d^{\text{sid}_S}(K)$ or $H_{\text{SK}}^{\text{sid}_S}(\text{ssid}, K)$ (with any ssid) for any $(\text{sid}_S, K) \in W$. Unless KQUERIED occurs, the views of $\mathcal{Z}$ in G_9 and G_{10} are identical, because $\mathcal{Z}$ never observes that d_K and SK were not derived from H_d and H_{SK} . Thus, $\Delta(G_9, G_{10}) \leq \Pr[\text{KQUERIED}]$. Furthermore, the only information $\mathcal{Z}$ sees about K is the commitment C_K , so the hiding property of ElGamal commitment should imply that it is hard for $\mathcal{Z}$ to compute K and trigger KQUERIED . Note that $\Pr[\text{KQUERIED}]$ is equal in G_9 and G_{10} , because the two games are identical up until the time KQUERIED occurs.

Given an environment $\mathcal{Z}$ that can trigger KQUERIED , we construct a reduction $\mathcal{R}$ that can solve DDH. The input to $\mathcal{R}$ is tuple $(A, B, C) \in \mathbb{G}^3$. $\mathcal{R}$ emulates game G_{10} to $\mathcal{Z}$ with the following modifications:

- In S , if $w_C \neq \perp$, then $\mathcal{R}$ parses $w_C \rightarrow (\text{hpw}, s_C, \cdot, \cdot, \cdot)$. If $F = f_{\text{PoNR}}^*(\text{sid}_S, \text{hpw}, s_C \oplus s_S)$, then $\mathcal{R}$ proceeds as before.
- Otherwise (i.e. $w_C = \perp$ or $F \neq f_{\text{PoNR}}^*(\text{sid}_S, \text{hpw}, s_C \oplus s_S)$), instead of sampling $K \leftarrow_{\mathbb{S}} \mathbb{G}$, $C_K := \text{ElG.Commit}(K; d_K)$, $\mathcal{R}$ samples $\delta, \mu, \nu \leftarrow_{\mathbb{S}} \mathbb{Z}_p$, $c_3 \leftarrow_{\mathbb{S}} \mathbb{G}$, and sets $C_K := (A \cdot g^\delta, g^\mu \cdot B^\nu, c_3)$. Also, $\mathcal{R}$ computes $K^* := c_3 / (A^\mu \cdot C^\nu) \cdot (g^\mu \cdot B^\nu)^\delta$ and adds (sid_S, K^*) to set W .

Reduction $\mathcal{R}$ outputs 1 iff $\mathcal{Z}$ ever performs a query $H_d^{\text{sid}_S}(K^*)$ or $H_{\text{SK}}^{\text{sid}_S}(\text{ssid}, K^*)$ (with any ssid) for any $(\text{sid}_S, K^*) \in W$.

The views of $\mathcal{Z}$ in $\mathcal{R}$ and G_{10} are identical. In both games, each C_K is a commitment to a random K that is never otherwise used (referring only to the cases of responding to an honest client or to an unsuccessful dishonest client). In G_{10} , the game samples these K values and commits to them outright. In $\mathcal{R}$ the reduction uses (A, B, C) to create the commitments (again applying DDH randomized self-reduction).

If (A, B, C) is a Diffie-Hellman tuple, then the committed K is K^* , so the KQUERIED event is exactly the same as the condition for $\mathcal{R}$ to output 1. Thus,

$\Pr[1 \leftarrow \mathcal{R} \mid (A, B, C) \in DH] = \Pr[\text{KQUERIED}]$. If (A, B, C) is not a Diffie-Hellman tuple, the committed K and K^* are independently random, and no information about K^* is exposed to $\mathcal{Z}$. At most q_S (sid_S, K^*) tuples are recorded, so $\Pr[1 \leftarrow \mathcal{R} \mid (A, B, C) \notin DH] \leq (q_S q_H)/p$. The absolute difference between these two probabilities is $\leq \text{Adv}_{\text{DDH}}$.

$$\Delta(\mathbf{G}_9, \mathbf{G}_{10}) \leq \text{Adv}_{\text{DDH}} + \frac{q_S q_H}{p}$$

Game $\mathbf{G}_{11}$ ($\mathbf{H}_{\text{SK}}$ is programmed with SK when responding to successful dishonest clients): $\mathbf{G}_{11}$ is $\mathbf{G}_{10}$ with the following changes to $\mathbf{S}$:

- If $w_C = \perp$, then proceed as before.
- Otherwise, parse $w_C \rightarrow (\text{hpw}, s_C, \cdot, \cdot, \cdot, \cdot)$. If $F \neq f_{\text{PoNR}}^*(\text{sid}_S, \text{hpw}, s_C \oplus s_S)$, then still proceed as before.
- Otherwise, do not compute C_K and V as normal. Instead, sample $\text{SK} \leftarrow_{\$} \{0, 1\}^\kappa$ and $K' \leftarrow_{\$} \mathbb{G}$. Then, compute $V \leftarrow \text{ElG.Enc}_{\text{pk}_C}(K')$ and $C_K \leftarrow \text{ElG.Commit}(K'; \text{H}_d^{\text{sid}_S}(K'))$. Finally, program $\text{H}_{\text{SK}}^{\text{sid}_S}(\text{ssid}_S, K') := \text{SK}$.

The change from $\mathbf{G}_{10}$ to $\mathbf{G}_{11}$ is mostly conceptual and merely shifts the order in which values are sampled and computed. The only actual difference from the previous game is that SK is sampled independently of K' and then programmed into H_{SK} . Consequently, the views of $\mathcal{Z}$ in $\mathbf{G}_{10}$ and $\mathbf{G}_{11}$ are identical unless $\mathcal{Z}$ queries $\text{H}_{\text{SK}}^{\text{sid}_S}(\text{ssid}_S, K')$ before the game samples $K' \leftarrow_{\$} \mathbb{G}$ and programs it. Only one $\mathbf{S}$ session runs per ssid_S , so only one H_{SK} point is programmed per ssid_S , and the environment's probability of having guessed one of them is negligible.

$$\Delta(\mathbf{G}_{10}, \mathbf{G}_{11}) \leq \frac{q_H}{p}$$

Game $\mathbf{G}_{12}$ (the simulated world): The change from $\mathbf{G}_{11}$ to $\mathbf{G}_{12}$ is purely conceptual. All interactions between the simulator and the honest parties now match the code of $\mathcal{F}_{\text{CKEM}}$, so one can imagine the monolithic simulator now cleanly split into the two components $\mathcal{F}_{\text{CKEM}}$ and $\text{SIM}_{\text{CKEM-PoNR}}$.

$$\Delta(\mathbf{G}_{11}, \mathbf{G}_{12}) = 0$$

$$\text{Exp}_{\mathcal{Z}}[\mathbf{G}_{12}] = \text{Ideal}_{\mathcal{Z}}[\mathcal{F}_{\text{CKEM}}, \text{SIM}_{\text{CKEM-PoNR}}]$$

Summing up the distinguishing advantage bounds for each incremental game change yields an overall bound on $\mathcal{Z}$'s distinguishing advantage between the real and simulated worlds.

$$\Delta(\mathbf{G}_0, \mathbf{G}_{12}) \leq 3 \cdot \text{Adv}_{\text{DDH}} + \text{Adv}_{\text{DL}} + \frac{q_C^2 + q_S^2 + q_S q_H + q_H}{p}$$

$p = |\mathbb{G}|$ is an exponential function of the security parameter. If $\mathcal{Z}$ is efficient, then q_C , q_S , and q_H are polynomial functions of the security parameter. Furthermore, Adv_{DDH} and Adv_{DL} are negligible functions of the security parameter under the DDH assumption. Thus, the distinguishing advantage of any efficient $\mathcal{Z}$ between the real and simulated worlds is negligible.

Protocol $\text{CKEM}_{\text{PoNR}}$ realizes $\mathcal{F}_{\text{CKEM}}[\text{SSEM}[f_{\text{PoNR}}^*, \text{ElG}], \text{SSEM}_{\text{Adv}}[f_{\text{PoNR}}^*]]$.